University of Stuttgart
Germany
Institute of Industrial Automation and Software Engineering

isk-Annotated Spatial Scene Graph for Dynamic Risk Assessment in Autonomous Systems

Master Thesis TBD

Submitted at the University of Stuttgart by
Fabin Fizel Sha

Electrical Engineering

Examiner: Prof. Dr.-Ing. Andrey Morozov

Supervisor: Joachim Grimstad, M.Sc.

External Supervisor: ()

30 October 2026

Abstract

Conventional risk assessment mechanisms are limited when the control strategy or operating environment of an autonomous system changes after deployment. This limitation is particularly relevant for systems operating in dynamic environments or modified through adaptive software components and over-the-air updates.

This thesis extends the framework for online risk-annotated scene graph generation presented by André Schiel in the master's thesis *Online Risk-Annotated Scene Graphs for Autonomous Systems* [1] into a spatial scene graph that supports dynamic risk assessment in near real time. The scene graph represents objects together with their spatial context, semantic labels and risk-relevant object descriptions generated by vision-language models, and an uncertainty estimate of whether each object is still at its last observed position. The framework is architected as a modular ROS 2 system, which supports portability and integration with robotic platforms, and it builds on the Hydra spatial perception system to support large environments.

Development and optimisation were carried out on an NVIDIA Jetson AGX Orin using the office scene of the simulated uHumans2 dataset, and the system was subsequently ported to an NVIDIA Jetson AGX Thor. Qwen3.5-4B combined with an optimised prompt was selected for object labelling, reaching an accuracy of 88 % on 50 manually verified object crops. Additional tests on the uHumans2 apartment scene and on multiple recordings from the office scene of the real-world OpenLORIS-Scene dataset demonstrate portability across environments and across simulated and real sensor data. On the Jetson AGX Thor, the pipeline processes **[XX]** frames per second on **[dataset and configuration]**. This near-real-time rate is an improvement over the baseline framework and makes the risk-annotated scene graph available as environmental context during system operation.

Keywords: dynamic risk assessment, risk-annotated scene graph, ROS 2, Hydra, uncertainty-aware object representation, vision-language model, edge AI

Contents

Abstract ii

List of Figures vi

List of Tables ix

1. Introduction 1
- 1.1. Motivation . . . 1
- 1.2. Relevance . . . 2
- 1.3. Goal of the Thesis . . . 2
- 1.4. Contributions . . . 3

2. Theoretical Background 4
- 2.1. Autonomous Systems and Safety . . . 4
  - 2.1.1. Human-Robot Collaboration . . . 5
  - 2.1.2. Conventional Risk Assessment and Risk Reduction . . . 6
  - 2.1.3. Existing Safety Measures in HRC . . . 6
  - 2.1.4. Impact of Artificial Intelligence . . . 8
- 2.2. Existing Work on Spatial Scene Graphs . . . 9
  - 2.2.1. 3D Scene Graph by Armeni et al. . . . 9
  - 2.2.2. SceneGraphFusion . . . 10
  - 2.2.3. ConceptGraphs . . . 11
  - 2.2.4. OpenGraph . . . 13
- 2.3. Robot Operating System . . . 14
  - 2.3.1. ROS as Robotic Middleware . . . 14
  - 2.3.2. ROS Communication Mechanisms . . . 15
  - 2.3.3. ROS 1 and ROS 2 . . . 15
- 2.4. Scene Graphs . . . 16
- 2.5. Hydra-Based Spatial Scene Graph Generation . . . 18
  - 2.5.1. Input Data and Processing Overview . . . 19
  - 2.5.2. Layered 3D Scene Graph Representation . . . 19
  - 2.5.3. Incremental Construction of Spatial Layers . . . 20
  - 2.5.4. Loop Closure, Optimization, and Real-Time Architecture . . . 20
  - 2.5.5. Relevance to the Proposed Framework . . . 21

2.6. Risk-Annotated Scene Graph Framework Baseline 21
2.6.1. Information Flow in the Baseline Framework 22
2.6.2. FastAPI 23
2.6.3. Scene Graph Construction and Risk Annotation 23
2.6.4. Limitations of the Baseline Framework 24
2.7. Lightweight Promptable Segmentation with NanoSAM 25
2.7.1. Architecture of the Segment Anything Model 26
2.7.2. Knowledge Distillation and MobileSAM 27
2.7.3. NanoSAM 27
2.7.4. Limitations 28
2.8. Multi-Object Tracking and Data Association 28
2.8.1. The Data Association Problem 29
2.8.2. Object-Level SLAM with Fusion++ 29
2.8.3. Limitations of Fusion++ 30
2.8.4. Relevance for Persistent Scene Representations 31
2.9. Visual Retrieval and Vision-Language Models 31
2.9.1. Retrieval-Augmented Perception 31
2.9.2. Vision-Language Models 33
2.9.3. The Qwen Vision-Language Model Family 34
2.10. Embedded Computing Hardware: NVIDIA Jetson Platforms 35
2.10.1. NVIDIA Jetson AGX Orin Developer Kit 35
2.10.2. NVIDIA Jetson AGX Thor Developer Kit 36
2.11. Scene Graphs for Downstream Safety Reasoning 36

3. Methodology **38**
3.1. Framework Overview 38
3.1.1. Framework Architecture 38
3.1.2. Central Design Decisions 40
3.2. Data Acquisition and Preprocessing 42
3.3. Object Segmentation and Persistent Tracking 43
3.3.1. Object Proposals and 3D Observations 44
3.3.2. Evidence-Based Association 45
3.3.3. Track Representation and Local Segments 47
3.3.4. Transfer of Object Observations to Hydra 48
3.4. Semantic Labelling and Risk-Relevant Object Information 48
3.4.1. Selection and Presentation of the Object View 48
3.4.2. Structured Output and Risk-Relevant Object Information 49
3.5. Scene Graph Fusion and Uncertainty Modelling 50
3.5.1. Scene Graph Fusion 50
3.5.2. Uncertainty and Persistence Modelling 51
3.5.3. Treatment of Static and Dynamic Objects 52

3.6. Loop Closure and Multi-Session Consistency . . . 53
3.6.1. Consistency under Loop Closure . . . 53
3.6.2. Persistence across Sessions . . . 55

4. Implementation **56**
4.1. Data Collection and Substitute Datasets . . . 56
4.1.1. Planned Data-Collection Campaign . . . 56
4.1.2. Limitations Encountered . . . 57
4.1.3. Substitute Datasets . . . 57
4.2. Software Implementation . . . 58
4.2.1. Package Structure and Deployment . . . 58
4.2.2. Preprocessor . . . 60
4.2.3. Phase 1 Coordinator . . . 61
4.2.4. Hydra Integration and Session Persistence . . . 62
4.2.5. Scene Graph Fuser . . . 64
4.2.6. Portability and Extensibility . . . 66

5. Results **67**
5.1. Segmentation Backend and Configuration . . . 67
5.2. Object Tracking and Crop Selection . . . 70
5.3. VLM Model and Prompt Comparison . . . 72
5.4. Visual Results across Environments . . . 74
5.5. Runtime Performance . . . 75

6. Conclusions **79**

Bibliography **80**

A. Prompts of the VLM Comparison **86**
A.1. Zero-Shot Prompt (P1) . . . 86
A.2. Example-Driven Prompt (P2) . . . 86
A.3. Structural-Priority Prompt (P3) . . . 88
A.4. Deployed Prompt with Visual-State Description . . . 90

List of Figures

2.1. Main challenges in HRC[19]. 5
2.2. Illustrative representation of safety zones in a collaborative robot workspace. 7
2.3. Overview of the SceneGraphFusion pipeline, showing RGB-D input, incremental geometric segmentation, point-cloud segments as graph nodes, feature extraction, graph neural network-based relationship prediction, and temporal fusion into an incremental 3D scene graph. Reproduced from [22]. 10
2.4. Overview of the ConceptGraphs pipeline, showing open-vocabulary detection and segmentation from posed RGB-D observations, multi-view object association, object fusion, construction of a 3D scene graph, and downstream applications such as segmentation, grounding, navigation, and manipulation. Reproduced from [8]. 12
2.5. Overview of the OpenGraph pipeline, showing caption-enhanced object comprehension from RGB images, object-centric 3D map construction using posed point clouds, and hierarchical scene representation formation for large-scale outdoor environments. Reproduced from [23]. 13
2.6. Scene graph example. Reproduced from [26]. 17
2.7. Example of a layered 3D scene graph generated by Hydra, showing abstraction levels from metric-semantic 3D mesh to objects and agents, places, rooms, and buildings. Reproduced from [7]. 18
2.8. Information flow of the existing risk-annotated scene graph framework. Adapted from [1]. 22
2.9. Overview of the Segment Anything Model (SAM) architecture. The model consists of an image encoder, a prompt encoder, and a lightweight mask decoder that produces segmentation masks conditioned on input prompts. Reproduced from Kirillov et al. [30]. 26
2.10. Principle of visual retrieval: the crop embedding is compared with stored references, and the nearest label is returned if its distance lies below the threshold δ. 32
2.11. SafetyDetect pipeline showing the agent point of view, scene graph construction, prompt formation, and LLM-based anomaly classification. Reproduced from [9]. 37

3.1. Overview of the proposed framework, showing the synchronous per-frame path (solid) and the asynchronous per-track path (dashed). . . . 39
3.2. Preprocessing of the input streams into a prepared RGB-D-pose frame. . 42

4.1. Class overview of the implemented framework, showing the main ROS 2 nodes, internal processing stages, supporting classes, and configurable backends. 59
4.2. Sequence of one frame through the preprocessor. 60
4.3. Sequence of the Phase 1 coordinator. 63
4.4. Sequence of the modified Hydra pipeline with session persistence. . . . 64
4.5. Sequence of the scene graph fuser. 65

5.1. Example benchmark frame. Each colour is one ground-truth object; hatched regions are below 3500 pixels and ignored. 67
5.2. NanoSAM against SAM ViT-B on the benchmark: (a) PQ, (b) median latency (log scale), (c) RQ, (d) SQ. 68
5.3. Parameter study of NanoSAM (input scale 0.85, at most 8 masks): (a) grid density at threshold 0.80 and NMS IoU 0.10, (b) mask threshold with a 4×4 grid, (c) NMS IoU with a 4×4 grid and threshold 0.80. Overlap is the share of pixels covered by more than one mask. 69
5.4. Tracking results: (a) fragmentation over the tightening runs R1 to R6 of the second experiment; (b) new tracks, revisit associations and labelling calls over five sessions. 71
5.5. Comparison of the three prompts on models of two generations, Qwen3 (blue) and Qwen3.5 (orange and green), on the office sequence ($N \approx 50$ crops per run): (a) strict accuracy, (b) median inference latency. 73
5.6. Scene graphs of the uHumans2 office (a–d) and apartment (e, f) sequences and of the OpenLORIS-Scene office1-1 sequence (g, h) at the end of each run, displayed in RViz. From bottom to top: objects, places (+10 m) and rooms (+20 m). Objects are coloured by label (grey: unlabelled), their opacity follows the presence confidence, and a grey halo marks currently observed objects. Black lines: hierarchy and Hydra edges; amber double arrows: contact relations; blue dotted lines: identity relations. [**TODO: adapt to what the panels actually show.**] 76
5.7. Runtime at saturated input: (a) throughput (Thor: mean $\pm$ std of three runs), (b) Phase 1 latency (Orin indicative). 77
5.8. Mean Phase 1 frame time by stage group ("Other": message construction, crop scoring and minor stages; Orin indicative). 77

5.9. Object-labelling VLM (Qwen3.5-4B, Q4_K_M, prompt P3). (a) Time per call on Orin without segmentation load and on Thor within the running pipeline. (b) Split of a Thor call into prompt processing and token generation, taken from the server log. 78

List of Tables

2.1. Latency on Jetson AGX Orin and mean IoU on COCO 2017 validation with ground-truth box prompts, as reported in [33]. 28
2.2. Release history of the open-weight Qwen vision-language models. 34

3.1. Information exchanged between the components of the framework. . . . 40
3.2. Differences in the treatment of static and dynamic objects. 52

4.1. Limitations encountered during the Go1 data-collection campaign. . . . 57

5.1. Edges in the visualisation. *Completed by the Fuser where missing, if no wall lies between the nodes (Section 3.5.1). 75

1. Introduction

1.1. Motivation

Robotic systems are increasingly moving out of fenced industrial cells and into shared, dynamic environments such as warehouses, hospitals, offices, and public spaces. According to the International Federation of Robotics, almost 200,000 professional service robots were sold in 2024, an increase of 9% over the previous year [2]. The largest share of these, 102,900 units, were built for transportation and logistics and consist mainly of mobile robots that move goods through their surroundings [2]. At the same time, the global operational stock of industrial robots reached 4.66 million units in 2024 [3]. Together, these figures show that robots are becoming a common part of environments that are also used by people.

Alongside this growth, robotic systems are increasingly shaped by artificial intelligence. The International Federation of Robotics identifies physical, analytic, and generative AI as key technological trends in robotics [3]. AI-based components allow robots to perceive their surroundings, interpret scenes, and adapt their behaviour. At the same time, they make robot behaviour harder to predict fully during design-time safety analysis [4].

This development creates new challenges for safety and risk assessment. Conventional risk assessment is typically carried out before deployment and relies on assumptions about the system design, intended use, operating environment, and known hazards [5]. These methods remain essential, but they reach their limits when mobile robots operate in changing environments, receive over-the-air software updates, or use adaptive AI-based components. In such cases, the situation the robot actually faces may differ from the one assumed during the original assessment. For this reason, automated and continuous risk assessment has been proposed, in which the assessment is updated while the system is running [6].

Such dynamic risk assessment needs a structured description of the current environment. It must know which objects are present, where they are, how they relate to each other, and which of them may be relevant to risk. A spatial risk-annotated scene graph is a suitable way to provide this information, as it combines perceived objects, their spatial context, and risk-related properties in a single machine-readable representation. In this thesis, the term *risk-annotated scene graph* refers to a spatial scene graph whose object nodes are enriched with risk-relevant semantic and uncertainty information, including object labels, confidence values, mobility information, presence likelihood, and descriptive object properties. These annotations provide structured environmental context for downstream dynamic risk assessment rather than representing a complete risk assessment by themselves.

1.2. Relevance

Scene graphs have become an established way to represent robotic environments in a structured form. Hydra [7] builds a layered 3D scene graph of objects, places, and rooms in real time and keeps it consistent through loop closure. However, Hydra does not identify objects itself. Its object layer depends on per-pixel semantic labels, which come from ground-truth annotations in simulation or from a segmentation network trained on a fixed set of classes [7]. ConceptGraphs [8] removes this restriction by identifying objects with open-vocabulary foundation models, but it applies these models to every observation and represents objects through free-form descriptions and features rather than bounded, confidence-weighted information.

At the same time, work on safety reasoning shows that scene graphs are a useful input for detecting hazardous situations. SafetyDetect [9], for example, uses object relations from a scene graph to let a language model classify situations as normal or dangerous. The authors note that their approach is limited mainly by how the scene graph is created, since risk assessment can only consider what the scene graph actually contains. The quality of the underlying scene graph is therefore a key factor for any risk assessment built on top of it.

The existing risk-annotated scene graph framework [1] brings these two directions together by adding risk-related information to a scene graph generated from RGB-D data. It is, however, limited to local, frame-based scene understanding and is exposed through a service interface that is not designed for robotic middleware. It also has no mechanism for keeping objects consistent across a larger environment, and it provides only basic handling of perception uncertainty. There is therefore a need for a risk-annotated scene graph that combines the spatial consistency of systems like Hydra with flexible object identification and risk-relevant object information, and that is designed for integration into a mobile robotic system during operation.

1.3. Goal of the Thesis

The goal of this thesis is to extend the existing risk-annotated scene graph framework [1] into a 3D spatial scene graph that provides up-to-date environmental context for dynamic risk assessment. The scene graph should describe the objects in the environment, their spatial context, and their risk-relevant properties. The work focuses on four aspects: portability, spatial scalability, uncertainty awareness, and near-real-time operation.

The first goal is to integrate the baseline framework into ROS 2 [10], so that it can interact directly with robotic sensors, localisation, and mapping components. The second goal is to ground the scene graph in a spatial map, so that objects observed at different times and locations share a common reference frame and remain spatially consistent when areas are revisited. Object identification should support open-vocabulary semantic perception, removing the dependence on ground-truth semantic labels or a fixed set of object classes.

The third goal is to represent uncertainty associated with object identification, mobility, and continued presence in the environment. The fourth goal is to support near-real-time operation on embedded robotic hardware. Finally, the framework is evaluated on the simulated uHumans2 dataset [11] and the real-world OpenLORIS-Scene dataset [12], focusing on object tracking and labelling, scene graph consistency, and runtime behaviour.

1.4. Contributions

This thesis makes the following contributions:

1. **A ROS 2-based risk-annotated scene graph framework.** The baseline framework is adapted for ROS 2, enabling direct integration with the sensors and software components of a robotic system. This provides a foundation for using risk-relevant scene representations within a broader robotic software architecture.

2. **A spatial scene graph with open-vocabulary object identification.** The framework extends Hydra [7] with flexible object identification beyond fixed semantic classes. The resulting 3D scene graph combines spatial, semantic, and mobility information while maintaining a consistent representation of the environment over time.

3. **A persistent object representation with uncertainty information.** Objects are represented together with confidence information for their semantic identity, mobility, and continued presence in the environment. This allows the scene graph to express both what is currently believed about an object and the uncertainty associated with that information.

4. **A design for near-real-time operation on embedded robotic hardware.** The framework is designed to maintain spatial scene understanding while incorporating computationally demanding semantic information. This allows the scene representation to remain suitable for applications in which environmental information must be updated continuously during robot operation.

5. **An experimental evaluation on simulated and real-world data.** The extended framework is evaluated using the simulated uHumans2 dataset [11] and the real-world OpenLORIS-Scene dataset [12]. The evaluation examines object tracking and labelling, spatial scene graph consistency, and runtime behaviour to identify both the capabilities and limitations of the proposed approach.

2. Theoretical Background

2.1. Autonomous Systems and Safety

Autonomous systems (AS) can be understood as systems that are capable of making decisions independently from direct human control [13]. In robotic systems, this autonomy is typically achieved through a combination of sensing, perception, planning, decision-making, and actuation. For example, an autonomous mobile robot may perceive obstacles, select an alternative path, and continue its task without requiring continuous human input. This ability allows autonomous robots to operate more flexibly in dynamic and partially unknown environments. However, it also makes safety assurance more complex, because the system behaviour is no longer determined only by predefined commands, but also by runtime perception, environmental conditions, and decision-making processes.

Safety is a fundamental requirement in AS because robots interact with humans, physical objects, infrastructure, and the surrounding environment. In dependability theory, safety is commonly defined as the absence of catastrophic consequences for users and the environment [14]. In the context of autonomous robotic systems, safety therefore refers not only to whether the robot performs its intended task, but also to whether it performs the task without creating unacceptable risk. This distinction is important because a robot may be functionally successful while still being unsafe. For example, a robot arm in a shared workspace may complete a pick-and-place task correctly, but the system cannot be considered safe if its motion exposes a nearby human operator to an unacceptable collision risk.

From a risk assessment perspective, safety is closely related to the identification of hazards, the estimation of risk, and the implementation of suitable risk reduction measures [5]. In robotic applications, this requires considering both the severity of possible harm and the likelihood that a hazardous situation may occur. Robot-specific safety standards further define requirements and protective measures for industrial and collaborative robotic systems [15]–[17]. However, safety in AS is strongly dependent on operational context. A motion, speed, or decision that is safe in one situation may become hazardous when a human enters the workspace, when an object is misplaced, or when perception uncertainty increases. Therefore, safety in autonomous robotic systems must be understood as a property that depends on both the technical system and the changing environment in which it operates.

2.1.1. Human-Robot Collaboration

Industry 5.0 is shifting industrial development toward a more human-centric, sustainable, and resilient model, while continuing to support productivity and worker well-being [18]. Within this perspective, Human-Robot Collaboration (HRC) becomes an important requirement for using robotic technologies to support and empower workers rather than simply replacing human labour [18]. Collaborative robotic systems can assist humans in repetitive, strenuous, or hazardous tasks, thereby improving workplace flexibility, ergonomics, and the distribution of work between humans and machines. In industrial settings, HRC has also been discussed as a response to skill gaps, skilled-worker retention, and the need to make manufacturing work more attractive to younger generations [19]. This makes HRC a relevant foundation for future autonomous systems, where robots increasingly need to interpret human presence, environmental changes, and task-specific context during operation.

However, HRC also introduces important challenges in industrial settings. Industrial HRC must address three closely related aspects: ensuring human safety, maintaining trust in automation, and preserving productivity [19]. Among these, human safety remains a primary concern because close physical interaction increases the possibility of unintended human-robot collisions, unexpected robot motion, or interruptions in the robot's behaviour. Such situations can increase the risk of injury and may negatively affect the human operator's trust in the system. At the same time, safety measures must be balanced with productivity requirements, since overly restrictive measures such as excessive speed reductions, large separation distances, or frequent safety stops can reduce system efficiency and interrupt the intended collaborative workflow [19]. The relationship between safety, trust in automation, and productivity is illustrated in Figure 2.1. This trade-off shows that HRC cannot be addressed only as a productivity or automation problem; it also requires systematic risk assessment methods that can identify hazardous situations and define suitable risk reduction measures.

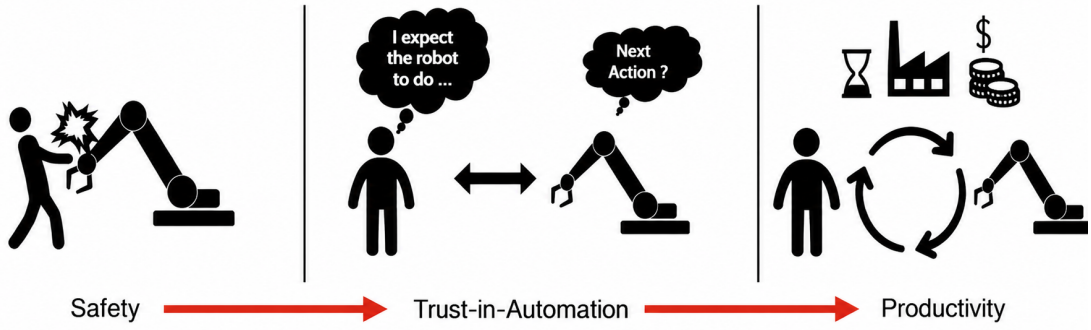


Figure 2.1.: Main challenges in HRC[19].

2.1.2. Conventional Risk Assessment and Risk Reduction

Conventional risk assessment in robotic systems is generally performed before deployment and is based on the intended use, foreseeable misuse, system limits, operating environment, and known hazards of the system. According to ISO 12100, the risk assessment process includes hazard identification, risk estimation, and risk evaluation, followed by the selection of suitable risk reduction measures [5]. In the context of industrial and collaborative robots, this process is supported by robot-specific safety standards that define requirements for robot systems, collaborative applications, and protective measures [15]–[17].

Risk reduction is usually implemented through a combination of inherently safe design, technical protective measures, and information for use. In HRC, this can include limiting robot speed and force, defining safe separation distances, using protective stops, and applying physical or electronic safeguards around the robot workspace. These measures are essential because they provide a systematic way to reduce the likelihood or severity of hazardous human-robot interactions before the system is put into operation.

However, conventional risk assessment is mainly based on assumptions made during design and deployment. These assumptions include expected human behaviour, known object configurations, defined workspaces, and predefined task conditions. As a result, conventional risk assessment is effective for identifying known and reasonably foreseeable hazards, but it becomes limited when the operating context changes significantly during runtime. This limitation is especially relevant for autonomous robotic systems, where the robot may operate in dynamic environments and make decisions based on sensor data and perception results.

2.1.3. Existing Safety Measures in HRC

In industrial HRC, risk reduction strategies are commonly implemented through two main approaches: the use of collaborative robots and the use of physical or electronic safeguarding measures [19]. Collaborative robots, also referred to as cobots, are designed to operate in direct cooperation with humans within a defined shared workspace. Their safety is supported through hardware and software features such as reduced momentum, force sensing, back-drivable actuators, and protective stop functions that interrupt robot motion when unexpected contact or resistance is detected.

The safeguarding-based approach focuses on monitoring the robot workspace so that robot motion can be adapted or stopped before hazardous human-robot collisions occur. In practice, this is achieved using physical and electronic safeguarding devices that define protected zones around the robot work cell. Examples include safeguarded perimeter door switches, light curtains, pressure-sensitive floors, 2-D scanning LiDAR, and 3-D safety vision systems. These mechanisms are important for reducing the risk of human injury, but they often rely on predefined safety zones, fixed distance thresholds, or conservative stopping rules [19].

A common method used for workspace monitoring is Speed and Separation Monitoring (SSM). In SSM, the distance between the human and the robot is monitored continuously, and the robot speed is reduced or stopped when the separation distance becomes too small. In a static SSM approach, the monitored zones are usually predefined before operation. For example, a robot may operate at full speed when no human is detected in the outer zone, slow down when a human enters a warning zone, and stop when the human enters a protective stop zone[19]. This approach is simple and practical, but it can be conservative because the safety zones are not always adapted to the actual robot motion, human motion, task state, or sensor uncertainty.



Figure 2.2.: Illustrative representation of safety zones in a collaborative robot workspace.

Dynamic SSM addresses this limitation by adapting the protective separation distance or the robot speed according to the current operating situation. Instead of relying only on fixed zones, dynamic SSM considers factors such as the actual distance between the human and the robot, robot velocity, human motion, stopping time, and uncertainty in the sensed position of obstacles or operators. Marvel discusses SSM performance in terms of both safety and productivity, representing collision potential through separation metrics and sensor uncertainty [20]. This shows that SSM is not only a binary stop-or-run mechanism, but also a method whose effectiveness depends on how accurately the system evaluates separation, uncertainty, and the productivity cost of safety actions.

Although dynamic SSM provides a more flexible basis for HRC safety than purely static safety zones, it also requires reliable perception, continuous monitoring, and suitable runtime decision-making. Therefore, existing safety measures show the importance of moving from fixed protective

assumptions toward context-aware risk assessment mechanisms that can consider the actual operational situation of the robot, humans, and surrounding environment.

2.1.4. Impact of Artificial Intelligence

The previous subsections show that conventional risk assessment and many existing HRC safety measures are based on pre-deployment assumptions, such as intended use, expected human behaviour, known object configurations, predefined workspaces, fixed safety zones, and foreseeable hazardous scenarios. These assumptions are necessary for systematic risk assessment, but they become less reliable when autonomous robots operate in changing environments. Human positions, object locations, obstacles, task conditions, and spatial relationships may vary during operation, meaning that a risk analysis performed before deployment may no longer fully represent the actual operating situation.

The integration of artificial intelligence into robotic perception, planning, or control further increases this limitation. AI-based components can enable object detection, scene interpretation, motion prediction, task planning, and adaptive decision-making. While these capabilities improve flexibility and autonomy, they also make robot behaviour harder to predict completely during design-time safety analysis. Safety assurance of AI-based systems is complicated by dataset dependence, black-box behaviour, explainability, and lifecycle assurance [4]. In robotic systems, this means that perception errors, low-confidence predictions, distribution shifts, previously unseen objects, changing human behaviour, or software updates after deployment can affect system decisions.

As a result, conventional risk assessment should be understood as a necessary baseline rather than a complete solution for adaptive autonomous robotic systems. If the deployed system operates under conditions that were not fully represented during the original assessment, the selected safety measures may become insufficient. Therefore, dynamic and context-aware risk assessment is required to incorporate runtime information such as robot state, human presence, object locations, spatial relationships, perception uncertainty, and environmental changes. Automated and continuous risk assessment has been proposed for software-defined robotic systems, where changes in software or system behaviour may require risk assessment to be updated during operation [6].

At the same time, AI can also support dynamic risk assessment when its outputs are structured, bounded, and interpretable. AI-based perception and reasoning methods can help identify relevant objects, detect environmental changes, classify scene elements, and localise risk-relevant information around the robot. In this thesis, this motivates the use of a spatial risk-annotated scene graph, which can represent objects, spatial relations, uncertainty, and risk-related context in a machine-readable form to support dynamic risk assessment in autonomous robotic systems.

2.2. Existing Work on Spatial Scene Graphs

Spatial scene graphs extend image-level scene graph representations by grounding objects, relationships, and higher-level spatial concepts in 3D space. This is important for robotic systems because a robot must not only recognise objects, but also understand where they are located, how they are arranged, and how they relate to larger spatial structures such as rooms or buildings. Several previous works have proposed methods for generating 3D scene graphs from reconstructed environments, RGB-D observations, or open-vocabulary perception pipelines. Although these works do not directly focus on risk annotation, they provide important methodological foundations for the spatial scene graph generation problem addressed in this thesis.

2.2.1. 3D Scene Graph by Armeni et al.

Armeni et al. introduced one of the foundational formulations of 3D scene graphs for building-scale indoor scene understanding [21]. Their work extends the conventional scene graph concept from 2D image representations to 3D environments. Instead of representing only objects and pairwise relations in an image, the proposed 3D scene graph organises semantic and spatial information across multiple levels of abstraction, including buildings, rooms, objects, and cameras.

The method uses a reconstructed 3D mesh and registered panoramic images as input. Visual information from the images is associated with the 3D mesh, allowing semantic labels and object-level information to be grounded in physical space. The resulting graph represents not only which objects are present, but also where they are located and how they are connected to larger spatial entities such as rooms. This makes the representation more suitable for indoor spatial understanding than a purely image-level graph.

A key contribution of this work is the introduction of a hierarchical representation for complete indoor environments. Object nodes can be connected to room nodes, cameras can be connected to the regions they observe, and spatial relationships can be represented explicitly. This supports queries such as which objects are present in a room, how cameras observe the environment, or how semantic information is distributed across a building.

For the present thesis, Armeni et al. are mainly relevant as a conceptual foundation for spatial scene graph structure. Their work shows how semantic information can be organised across object, room, camera, and building levels. However, the method is primarily designed for building-scale scene understanding from reconstructed data, rather than online robotic operation. Therefore, its main contribution to this thesis is the idea of spatial hierarchy, while real-time construction, loop closure, uncertainty handling, and risk annotation require additional methods.

2.2.2. SceneGraphFusion

SceneGraphFusion by Wu et al. focuses on incremental 3D scene graph prediction from RGB-D sequences [22]. This makes it particularly relevant to robotic perception, because RGB-D cameras provide both colour and depth information and are commonly used for indoor robotic systems. Unlike methods that generate a scene graph from a single image or from a completed offline reconstruction, SceneGraphFusion builds and updates the graph incrementally as new RGB-D frames are observed.

The method first uses RGB-D input to support 3D reconstruction and instance-level scene understanding. As the camera observes the environment from different viewpoints, partial observations are fused into a growing 3D representation. Object instances are represented as graph nodes, while relationships between objects are represented as graph edges. The method then predicts object and relationship semantics using graph neural network-based reasoning, combining geometric, visual, and semantic information from the reconstructed scene.

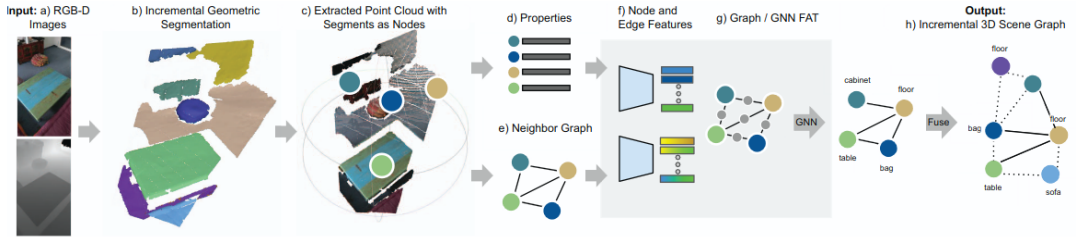


Figure 2.3.: Overview of the SceneGraphFusion pipeline, showing RGB-D input, incremental geometric segmentation, point-cloud segments as graph nodes, feature extraction, graph neural network-based relationship prediction, and temporal fusion into an incremental 3D scene graph. Reproduced from [22].

A key contribution of SceneGraphFusion is its incremental formulation. Robotic perception is naturally sequential: a robot observes only part of the environment at each time step, and the representation must be updated as new views become available. SceneGraphFusion addresses this problem by treating scene graph generation as a progressive RGB-D fusion task rather than as a single-frame prediction problem. This allows the graph to become more complete as additional observations are integrated.

An important part of the method is its temporal scene graph fusion strategy. Because the framework predicts the semantic labels of segments and relationships multiple times over a sequence, different observations may produce different or contradictory predictions. To handle this, SceneGraphFusion stores a probability distribution and a weight for each predicted class or relationship predicate. When a new prediction is available, the stored probability is updated using a weighted running average. Using $\hat{\mu}_t$ for the new prediction probability and $\hat{w}_t$ for its weight, the updated probability can be written as

$$\mu_t = \frac{\hat{\mu}_t \hat{w}_t + \mu_{t-1} w_{t-1}}{\hat{w}_t + w_{t-1}},$$

and the corresponding weight is updated as

$$w_t = \min(w_{\max}, \hat{w}_t + w_{t-1}),$$

where $w_{\max}$ is the maximum allowed weight [22]. This temporal fusion mechanism allows repeated observations to strengthen stable predictions while reducing the influence of isolated inconsistent predictions.

The temporal fusion strategy is relevant to this thesis as an example of how repeated observations can be combined to improve the stability of a scene graph over time. Instead of storing only a single deterministic label for each object or relationship, SceneGraphFusion preserves a probability distribution over possible classes or predicates. This allows the representation to retain information about semantic uncertainty and to reduce the effect of isolated inconsistent predictions. Such an approach is useful to study because dynamic risk assessment may depend not only on the selected object or relation label, but also on how confident the system is in that prediction.

For this thesis, SceneGraphFusion is therefore considered as an important reference for alternative approaches to incremental scene graph generation. It demonstrates how RGB-D observations can be fused over time and how semantic predictions for graph nodes and edges can be stabilized using temporal probability fusion. However, its main focus remains object-relation prediction in reconstructed indoor scenes. The present thesis investigates a different objective: extending risk-annotated scene graph generation toward spatial grounding, long-term object persistence, loop-closure-aware consistency, uncertainty representation, and dynamic risk assessment in robotic environments.

2.2.3. ConceptGraphs

ConceptGraphs by Gu et al. introduces open-vocabulary 3D scene graphs for perception and planning [8]. The work is motivated by the need for 3D scene representations that can be queried using flexible language-based concepts, rather than only a fixed set of predefined labels. This is relevant for robotic applications because the objects and categories encountered during operation may not always be known in advance. Instead of treating scene understanding only as a closed-set classification problem, ConceptGraphs connects 3D spatial mapping with language-aligned visual representations.

The method builds 3D scene graphs from posed RGB-D observations. Each RGB-D frame provides colour information, depth information, and a known camera pose. Object-level information is first extracted from individual 2D views using foundation-model-based perception. These detections

or segmentations provide candidate object regions in the image. The depth image and camera pose are then used to lift these 2D observations into 3D space. In this way, image-level object information is transformed into spatially grounded object candidates.

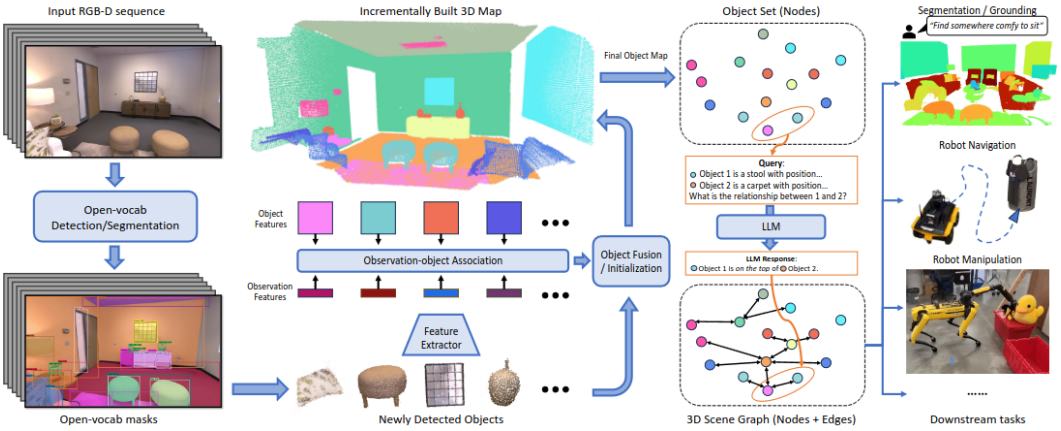


Figure 2.4.: Overview of the ConceptGraphs pipeline, showing open-vocabulary detection and segmentation from posed RGB-D observations, multi-view object association, object fusion, construction of a 3D scene graph, and downstream applications such as segmentation, grounding, navigation, and manipulation. Reproduced from [8].

A central part of the method is multi-view association. Since the same physical object can be visible in several RGB-D frames, the system must decide whether a newly detected object corresponds to an existing object in the 3D map or whether it should be inserted as a new object. ConceptGraphs performs this association using spatial and visual information, so that repeated observations of the same object can be fused into a single 3D object node. This reduces duplicate object creation and allows the graph to become more consistent as more views are processed.

The resulting graph contains object nodes that are grounded in 3D space while also carrying language-aligned features. These features allow the graph to be queried using natural-language descriptions. For example, instead of searching only for a fixed class label, a downstream module can use more flexible text queries related to object identity, appearance, or function. This makes the representation useful for tasks such as object grounding, navigation, manipulation, and language-conditioned planning.

For this thesis, ConceptGraphs is relevant because it shows one approach for combining 3D scene graph generation with foundation-model-based semantic understanding. This is related to the proposed risk-annotated scene graph framework, where visual-language reasoning is also used to enrich the scene representation. However, the focus of ConceptGraphs is mainly open-vocabulary perception and planning, whereas the present thesis focuses on generating a spatial scene graph that can support risk annotation, uncertainty representation, and dynamic risk assessment. Therefore, ConceptGraphs is useful as a reference for semantic flexibility in 3D

scene graphs, while the risk-oriented interpretation and safety relevance remain the focus of the proposed work.

2.2.4. OpenGraph

OpenGraph by Deng et al. proposes an open-vocabulary hierarchical 3D graph representation for large-scale outdoor environments [23]. While ConceptGraphs focuses on open-vocabulary 3D scene graphs from posed RGB-D observations, OpenGraph addresses outdoor scenes where the environment is larger, less structured, and contains many object instances. This makes the work relevant as an example of how open-vocabulary graph representations can be extended beyond indoor rooms or small-scale manipulation settings.

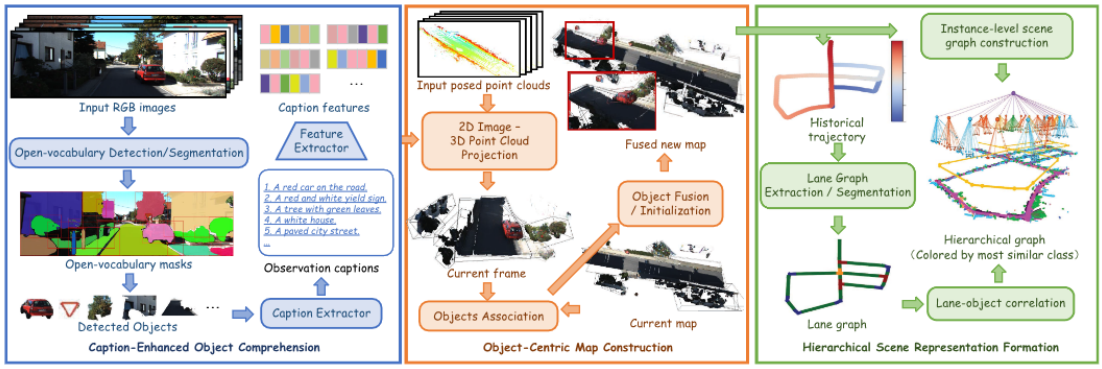


Figure 2.5.: Overview of the OpenGraph pipeline, showing caption-enhanced object comprehension from RGB images, object-centric 3D map construction using posed point clouds, and hierarchical scene representation formation for large-scale outdoor environments. Reproduced from [23].

The method first extracts object instances and textual captions from visual images. These captions are then encoded to support language-based reasoning about the detected instances. The 2D information is connected to 3D space by projecting image-based instance information onto LiDAR point clouds using multi-sensor calibration. In this way, OpenGraph builds an incremental object-centric 3D map in which objects are associated with both geometric information and language-related features.

A key feature of OpenGraph is its hierarchical organisation. The representation is not limited to isolated object nodes. Instead, the environment is structured using a hierarchy that includes object-level information and larger spatial regions derived from lane graph connectivity. This is useful in large outdoor environments, where direct object-level retrieval alone may be insufficient for navigation or spatial querying. The hierarchical graph structure allows the system to represent both local object information and broader environmental organisation.

For this thesis, OpenGraph is relevant because it shows how open-vocabulary semantic information can be combined with 3D graph representations in a large-scale robotic setting. Although the target environment is outdoor and the sensing setup relies on camera-LiDAR data rather than RGB-D input, the underlying idea is closely related: semantic information from visual-language models is grounded in 3D space and organised as a graph. This supports the motivation for using graph-based spatial representations when semantic reasoning must be connected to physical environments.

At the same time, OpenGraph has a different focus from the present thesis. Its main objective is open-vocabulary understanding, object retrieval, and hierarchical representation in outdoor environments. The present thesis instead focuses on spatial risk-annotated scene graph generation for robotic risk assessment. Therefore, OpenGraph is useful as a reference for large-scale and open-vocabulary graph construction, while the risk-related interpretation, uncertainty handling, and indoor RGB-D robotic integration remain separate contributions of this work.

2.3. Robot Operating System

The Robot Operating System (ROS) is an open-source software framework for developing and integrating robotic systems. Despite its name, ROS is not a traditional operating system; it functions as a middleware layer that provides communication mechanisms, software libraries, development tools, and conventions for building modular robot software [24]. In this thesis, ROS is relevant because the risk-annotated scene graph framework must be integrated with continuous RGB-D sensor streams, pose information, SLAM modules, system status information, and downstream risk assessment components. Therefore, ROS provides the conceptual and technical basis for replacing the service-oriented baseline interface with a robot-native integration approach.

2.3.1. ROS as Robotic Middleware

A ROS-based robotic system is typically organised as a collection of nodes. A node is an executable software component that performs a specific function, such as publishing camera data, processing sensor input, estimating robot pose, performing SLAM, planning motion, controlling actuators, or visualising system state. This node-based structure allows complex robotic systems to be decomposed into smaller modules that can be developed, tested, and replaced independently. For example, an RGB-D camera node can publish sensor data, a SLAM node can estimate the robot pose and map, and a scene graph node can use this information to update a structured representation of the environment.

Communication between nodes is performed using messages. A message defines the structure of the exchanged data, such as an image, pose, velocity command, point cloud, or status value. ROS provides standard message types for common robotic data, while also allowing custom message

definitions for application-specific information. This is important for robotic integration because different modules can exchange data in a predictable and structured way.

2.3.2. ROS Communication Mechanisms

ROS provides several communication mechanisms for different interaction patterns. Topics are used for continuous data streams and follow a publish-subscribe model. A node publishes messages to a topic, and one or more other nodes subscribe to that topic. This is suitable for sensor data such as RGB images, depth images, LiDAR scans, odometry, IMU data, and system status. In the context of this thesis, RGB-D observations, pose information, and scene graph updates can naturally be represented as topic-based communication because they are produced repeatedly during robot operation.

Services provide request-response communication. A service is useful when one node needs to request a specific operation from another node and receive a result, such as resetting a scene graph, saving the current graph, or requesting the current system state. Unlike topics, services are not intended for continuous streaming, but for discrete interactions where a response is expected.

Actions are used for long-running tasks that may require feedback or cancellation. An action consists of a goal, feedback during execution, and a final result. This mechanism is commonly used for navigation tasks, where a robot receives a goal pose, reports progress, and eventually returns whether the goal was reached. In a scene graph framework, actions could be useful for longer operations such as processing a selected dataset sequence or generating a scene graph for a defined exploration segment.

ROS also uses parameters to configure node behaviour. Parameters can define topic names, thresholds, model settings, update rates, frame identifiers, or other runtime options. This allows the same node to be reused in different experimental setups without changing the source code. Launch files are then used to start multiple nodes together with their required parameters, making it easier to reproduce complete robotic system configurations.

2.3.3. ROS 1 and ROS 2

ROS 1 was the first widely adopted version of ROS and became an important standard for robotics research and prototyping. In ROS 1, node discovery is coordinated through a central ROS Master, which allows publishers and subscribers to find each other before communicating. This architecture supported rapid development and experimentation, but it was mainly designed for research-oriented systems and did not fully address several requirements of modern robotic deployment, such as reliable distributed execution, multi-robot communication, real-time behaviour, security, and production-oriented system design.

ROS 2 was developed to address these limitations and to support more robust robotic applications [10]. A major architectural difference is that ROS 2 uses the Data Distribution Service (DDS) as its communication middleware. DDS enables decentralised discovery, configurable Quality of Service (QoS), and improved support for distributed robotic systems. This means that ROS 2 nodes can discover each other without relying on a central master, which improves suitability for multi-robot systems and distributed deployment.

Quality of Service is an important feature of ROS 2 communication. QoS settings allow developers to control how messages are delivered, for example whether communication should prioritize reliability or low latency. This is important because different robotic data streams have different requirements. A camera stream may prefer recent data and tolerate dropped frames, while system status or safety-related information may require more reliable delivery. ROS 2 also improves support for lifecycle management, embedded platforms, real-time control, and security compared with ROS 1 [10]. At the same time, practical studies of ROS 2 robotic control show that latency and jitter still need to be evaluated carefully in distributed and time-sensitive applications [25].

In the context of this thesis, ROS 2 is relevant because the risk-annotated scene graph framework must operate as part of a robotic system rather than as an isolated external service. RGB-D data, pose information, system status, and scene graph updates need to be exchanged continuously between different software components. ROS 2 provides a robot-native communication structure for this purpose, allowing the framework to be integrated with perception sensors, SLAM modules, robot state estimation, visualization tools, and downstream risk assessment mechanisms.

2.4. Scene Graphs

A scene graph is a structured representation of a visual scene in which objects are represented as nodes and relationships between objects are represented as edges [26]. Instead of describing an image or environment only as a collection of detected objects, a scene graph also captures how these objects are related to each other. For example, a scene may contain nodes such as *person*, *chair*, and *table*, while edges may describe relations such as *sitting on*, *next to*, or *under*. Johnson et al. demonstrated the usefulness of scene graphs for image retrieval by representing image content through objects, attributes, and relationships rather than relying only on low-level image features or isolated object labels [26].

The main advantage of a scene graph is that it provides a machine-readable representation of both entities and context. Object detections alone can indicate which objects are present, but they do not fully describe the structure of the scene. In contrast, a scene graph can represent object attributes, spatial arrangements, and semantic relationships in a compact graph format [26]. This makes it useful for tasks that require reasoning about the environment, because downstream modules can query not only whether an object exists, but also how it is positioned or related to other objects.

Scene graphs have been used in several computer vision and language-vision tasks, including image retrieval, visual relationship understanding, visual question answering, and image cap-

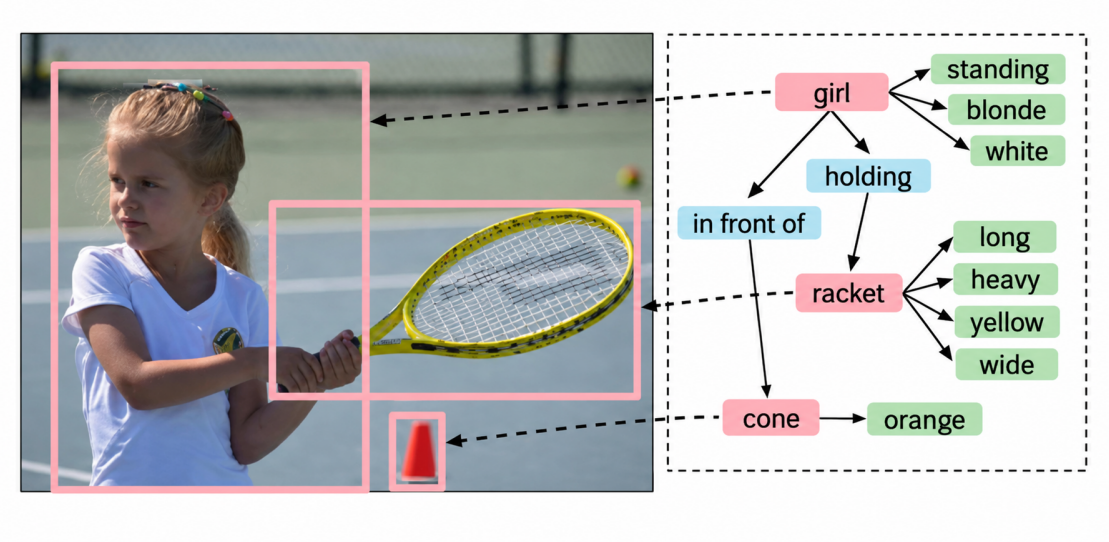


Figure 2.6.: Scene graph example. Reproduced from [26].

tioning. Datasets such as Visual Genome support this direction by providing dense annotations of objects, attributes, relationships, region descriptions, scene graphs, and question-answer pairs [27]. Beyond 2D images, scene graphs have also been extended to 3D environments, where objects, rooms, cameras, and their relationships can be represented in a unified spatial structure [21]. This makes scene graphs relevant not only for image-level understanding, but also for robotic and spatial perception tasks.

Scene graphs are also advantageous because they provide an interpretable bridge between perception and higher-level reasoning. Raw sensor data such as images or point clouds is difficult for symbolic reasoning modules to use directly. A scene graph abstracts this raw data into nodes, edges, and attributes that can be inspected, updated, and processed algorithmically. This is especially useful in robotic systems, where perception results must often be connected to planning, navigation, manipulation, or risk assessment.

In the context of this thesis, scene graphs are relevant because dynamic risk assessment requires more than object detection. Risk may depend on object identity, spatial proximity, support relations, containment, occlusion, human presence, and the current configuration of the environment. A graph-based representation allows such information to be stored in a structured form. When extended with spatial information, uncertainty estimates, and risk annotations, the scene graph becomes a suitable basis for representing the current operational context of an autonomous robotic system.

2.5. Hydra-Based Spatial Scene Graph Generation

Hydra is a real-time spatial perception system for constructing and optimizing 3D scene graphs from robotic sensor data [7]. It is relevant to this thesis because it addresses key requirements for spatial risk-annotated scene graph generation, including incremental construction, scalability to larger environments, loop closure correction, and near-real-time operation. While the baseline risk-annotated scene graph framework focuses on semantic and risk-related interpretation of RGB-D observations, Hydra provides a spatial backbone for building persistent environment-level representations.

Hydra represents the environment as a layered 3D scene graph rather than only as a set of detected objects. The graph connects metric geometry, objects, agents, places, rooms, and building-level structure. This allows the system to maintain both low-level geometric information and higher-level semantic and topological abstractions, as illustrated in Figure 2.7.

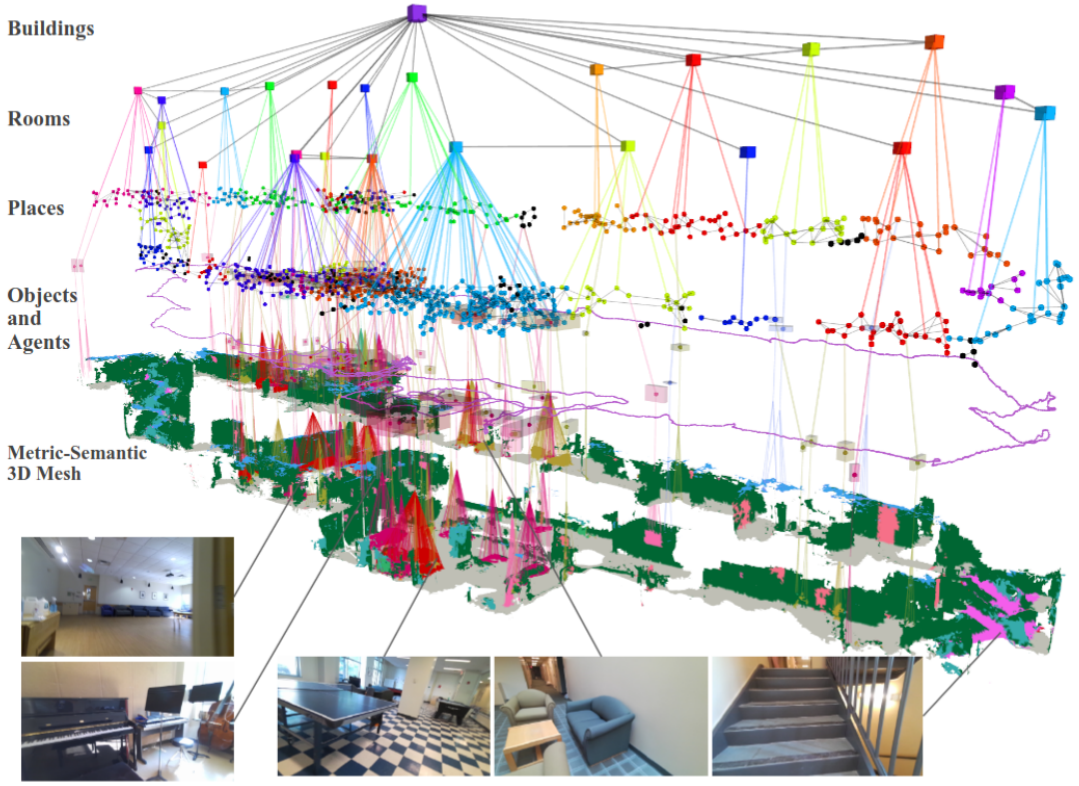


Figure 2.7.: Example of a layered 3D scene graph generated by Hydra, showing abstraction levels from metric-semantic 3D mesh to objects and agents, places, rooms, and buildings. Reproduced from [7].

2.5.1. Input Data and Processing Overview

Hydra operates on continuous sensor data from a moving robot. In the architecture described by Hughes et al., the raw inputs are images and IMU measurements, which are processed by low-level perception modules to estimate robot motion, reconstruct depth, and obtain semantic segmentation [7]. The motion estimate provides the transformations required to place observations into a common spatial reference frame, while depth information provides the 3D structure of the environment.

Semantic information is provided before the 3D scene graph layers are constructed. In simulation, these semantic labels can come from ground-truth annotations generated by the simulated environment. In real-world experiments, Hydra uses HRNet for semantic segmentation with labels from the ADE20K scene parsing dataset [7], [28], [29]. These semantic labels are combined with depth information to construct a metric-semantic 3D representation, where reconstructed geometry is associated with semantic classes such as wall, floor, furniture, or other object categories.

The overall processing pipeline transforms sensor-level data into graph-level spatial knowledge. Low-level perception estimates motion, depth, and semantic labels. Mid-level perception constructs the metric-semantic mesh, object layer, and places graph. High-level perception performs loop closure detection, scene graph optimization, and room detection. In this way, Hydra integrates observations over time to build a persistent layered representation of both the current surroundings and previously explored parts of the environment.

2.5.2. Layered 3D Scene Graph Representation

The main output of Hydra is a layered 3D scene graph, where nodes represent spatial concepts at different levels of abstraction and edges represent relationships between them. The lowest layer is the metric-semantic 3D mesh, which represents the physical geometry of the environment together with semantic labels. Above this, the object and agent layer represents detected objects and the robot trajectory. Object nodes can include semantic labels, centroids, and bounding boxes, while the agent is represented by a pose graph describing the robot's motion [7].

The places layer provides a sparse topological representation of obstacle-free space. A place represents a traversable location in the environment, and an edge between two places indicates straight-line traversability. The room layer groups places into larger spatial regions, while the building layer provides the highest-level abstraction. This hierarchy is useful because different robotic tasks require different levels of detail: metric geometry supports local navigation, places support topological reasoning, and objects and rooms support semantic interpretation.

Hydra separates scene graph construction into a frontend and a backend. The frontend incrementally assembles an odometric scene graph from the latest mesh, object, places, and agent-pose information. The backend corrects this representation when loop closures are detected by optimizing selected graph elements and propagating the correction to the remaining layers [7].

This separation allows continuous local graph construction while still supporting slower global correction.

2.5.3. Incremental Construction of Spatial Layers

Hydra constructs the scene graph incrementally rather than rebuilding the full environment in a batch process. A key design choice is the use of a local active window around the robot. Within this active window, Hydra maintains volumetric spatial representations such as a Truncated Signed Distance Field (TSDF) and an Euclidean Signed Distance Function (ESDF). This limits memory usage because the system does not need to maintain an ESDF of the entire environment at all times [7].

The metric-semantic mesh is extracted from the local volumetric representation. Object nodes are then generated by clustering mesh vertices that belong to the same semantic class. For each putative object, Hydra retains the semantic label inherited from the metric-semantic mesh and estimates geometric properties such as the centroid and bounding box. New observations can be merged with existing object nodes if they overlap spatially and belong to the same semantic class; otherwise, they are inserted as new object nodes.

The places layer is extracted from the ESDF using a Generalized Voronoi Diagram (GVD). The ESDF stores the distance from each voxel, or small 3D map cell, to the nearest obstacle. Voxels that are approximately equidistant from multiple obstacles form the GVD, which lies near the medial axis of free space. In indoor environments, this means that GVD voxels tend to lie near the centre of corridors, rooms, and passages rather than close to walls or obstacles. Hydra sparsifies this structure into a graph of places by selecting representative GVD voxels as place nodes and connecting them with edges when straight-line traversal is possible. Therefore, places are not object detections; they are topological free-space locations extracted from the geometry of the environment [7].

Room detection is then performed using the places graph. The key idea is that rooms can be identified by analysing how free-space connectivity changes when obstacles are virtually inflated. When the obstacle dilation distance increases, narrow openings such as doors gradually close. This causes the places graph to split into connected components, which can correspond to individual rooms. Hydra evaluates multiple dilation distances, selects a stable partition of the places graph, and assigns remaining unlabeled places using graph clustering based on modularity. In this way, room-level structure is extracted from the topology of free space rather than from direct image-based room classification or from processing the complete dense map [7].

2.5.4. Loop Closure, Optimization, and Real-Time Architecture

As a robot moves through an environment, odometry errors can accumulate and cause drift in the estimated map and scene graph. Hydra addresses this through loop closure detection and

scene graph optimization. Loop closure detection uses hierarchical descriptors derived from multiple graph layers, including visual appearance, nearby object semantics, and places geometry. Candidate loop closures are then geometrically verified using visual features or object-based registration [7].

Once a loop closure is accepted, Hydra optimizes the scene graph using an embedded deformation graph. This deformation graph contains selected elements such as the agent pose graph, mesh control points, and places layer. The optimization corrects accumulated drift and propagates the correction to the rest of the scene graph. Object centroids and bounding boxes can be recomputed after deformation, overlapping nodes can be merged, and rooms can be updated from the corrected places graph.

Hydra is organized as a highly parallelized architecture with low-level, mid-level, and high-level perception processes. Low-level perception operates close to sensor rate and includes visual-inertial odometry, semantic segmentation, and depth reconstruction. Mid-level perception constructs local spatial structures such as the mesh, object layer, places graph, and scene graph frontend. High-level perception performs loop closure detection, backend optimization, and room detection. This separation prevents slower global operations from blocking faster local perception, which is important for near-real-time robotic operation.

2.5.5. Relevance to the Proposed Framework

Hydra provides a strong methodological reference for this thesis because it demonstrates how a robot can construct a persistent spatial scene graph in real time. The most relevant ideas are the layered graph representation, incremental construction of spatial layers, local active-window mapping, places extraction from free-space topology, room detection from the places graph, loop closure correction, and separation between fast local processing and slower global optimization.

In this thesis, Hydra is useful as a spatial backbone for extending the existing risk-annotated scene graph framework. The baseline framework provides semantic and risk-oriented interpretation, while Hydra provides spatial consistency, scalability, and loop closure support. Combining these ideas motivates a spatial risk-annotated scene graph that can represent risk-relevant objects, semantic relations, uncertainty, and spatial context across larger robotic environments.

2.6. Risk-Annotated Scene Graph Framework Baseline

The framework developed in this thesis extends the risk-annotated scene graph framework presented by Schiel [1], which is referred to as the baseline in the following. This section outlines its processing pipeline and external interface, explains how the scene graph is constructed and annotated with risk-relevant information, and identifies the limitations that motivate the present work. The focus lies on the conceptual design of the framework rather than on its implementation.

The underlying segmentation, retrieval, and vision-language techniques are introduced separately in Sections 2.7 and 2.9.

2.6.1. Information Flow in the Baseline Framework

The baseline framework follows a modular, worker-based processing pipeline in which RGB-D data is received, processed, interpreted, and integrated into a risk-annotated scene graph. The input to the system is an RGB-D observation of the environment, consisting of a colour image and its corresponding depth image, which is either provided through a dataset or submitted through the FastAPI-based interface described in Section 2.6.2. This design separates data input and client communication from the perception and scene graph generation modules. The resulting information flow is illustrated in Figure 2.8.

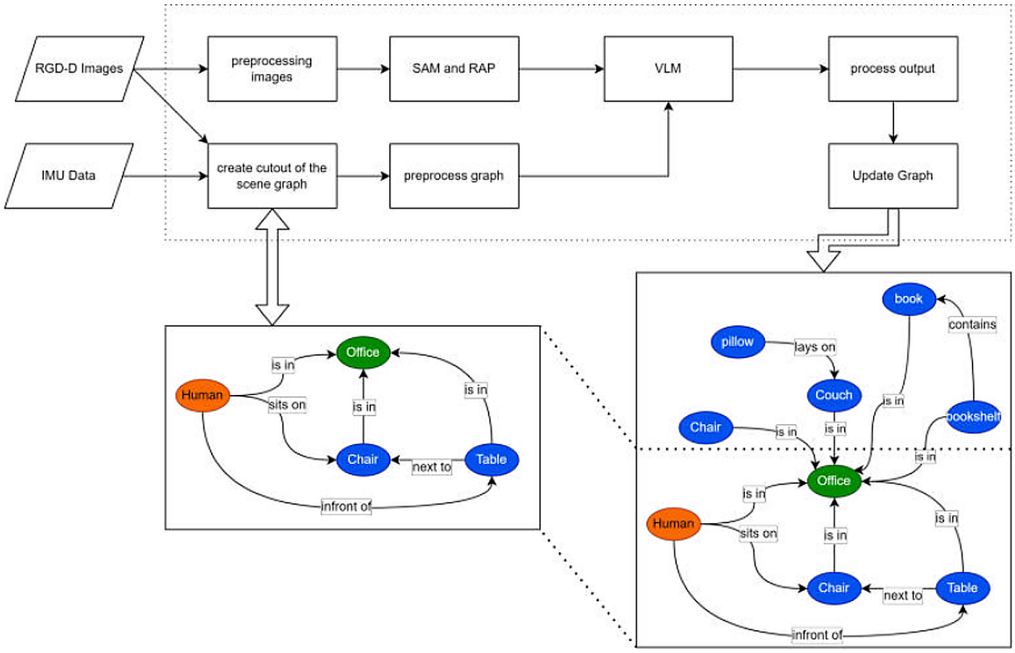


Figure 2.8.: Information flow of the existing risk-annotated scene graph framework. Adapted from [1].

Each received sample is handled as a task in the processing pipeline. The colour image is first segmented with the Segment Anything Model (SAM) [30] to obtain class-agnostic object candidates. For every retained candidate, the framework stores an image crop, a bounding box, a binary mask, and a segmentation quality score. These candidates carry no semantic meaning, so their identities must be determined in the subsequent stages. The depth image, camera parameters, and pose information are used for the later spatial processing, including coordinate conversion, visibility checking, and the spatial grounding of graph elements.

The object crops are then passed to the Retrieval-Augmented Perception (RAP) module, which returns label candidates from a visual knowledge base of previously stored observations (Section 2.9.1). In parallel, the existing scene graph is reduced to a cutout that contains only the nodes and edges likely to be visible in the current camera view, determined from the current pose and depth image. The RGB image, the scene-graph cutout, and the RAP label candidates are subsequently passed to a vision-language model (VLM), which returns an updated graph for the current view (Section 2.9.2). Finally, this output is grounded in 3D and merged into the complete scene graph, as described in Section 2.6.3.

2.6.2. FastAPI

FastAPI is a Python web framework for building application programming interfaces (APIs), through which external clients communicate with a software system via network-accessible endpoints, typically using HTTP requests [31]. The expected format of incoming requests can be defined explicitly, which suits complex inputs such as RGB-D images, timestamps, and pose information, and asynchronous operation allows several requests to be handled concurrently [31]. In the baseline framework, FastAPI exposes the scene graph generation pipeline as an external service and keeps client applications separated from the internal perception, segmentation, VLM, and scene graph modules. For this external communication, the internal graph is converted into a dictionary-based representation in which node and edge information are stored separately.

This interface is well suited to dataset-based testing and to request-response communication with external clients [31]. Robotic software, however, relies on continuous high-frequency sensor streams, synchronised multi-modal data, distributed nodes, and predictable timing behaviour, for which a REST-based interface is less natural. FastAPI is therefore not ideal as the primary communication mechanism for deployment on a robotic platform, which motivates the port of the framework to ROS 2 [10].

2.6.3. Scene Graph Construction and Risk Annotation

The baseline framework represents the scene graph as a directed graph, where entities in the environment are modelled as nodes and relationships between entities are modelled as edges [1]. The implementation uses NetworkX, a Python library for network analysis and graph manipulation, which supports incremental graph operations such as adding, updating, and removing nodes and edges [32].

Each node stores semantic, spatial, and risk-related information. Important node attributes include a persistent identifier, object label, semantic description, semantic pose, 3D position, risk metadata, appearance score, confidence score, and layer type. The semantic description describes the visual identity of the entity, while the semantic pose describes its current physical state, such as standing, lying, open, closed, or moving. Edges store relationships between nodes, including the source node, target node, relationship label, distance between the connected entities, and an appearance score.

After each VLM inference step, the returned graph is converted from image-space information into 3D coordinates using the depth image and camera pose. The converted output is then compared with the scene-graph cutout that was originally sent to the VLM, and the two are combined following an update-or-insert strategy. For each node in the VLM output, the framework checks whether a node with the same identifier already exists in the complete scene graph. If it exists, its attributes are updated using the latest VLM output and its appearance score is increased. Otherwise, it is inserted as a new node with an initial appearance score. This allows newly detected objects or humans to be added while persistent entities are reinforced over time.

Nodes that were present in the VLM input cutout but are missing from the VLM output are interpreted as not being detected in the current update. Instead of being removed immediately, their appearance score is decreased, and a node is removed only when its appearance score falls below a predefined threshold. This prevents isolated missed detections from immediately deleting objects from the graph.

The same principle is applied to edges. If an edge returned by the VLM already exists between the corresponding source and target nodes, its relationship label and distance are updated and its appearance score is increased; otherwise, a new edge is added with an initial appearance score. The edge distance is calculated from the 3D positions of the connected nodes, so that spatial relationships are stored together with semantic relationship labels. Edges that were present in the input cutout but are missing from the VLM output are weakened by decreasing their appearance score and are removed once it falls below the threshold. Temporal consistency is therefore handled through appearance-based persistence: repeatedly observed nodes and edges become more stable, while missing or transient entities are gradually removed.

2.6.4. Limitations of the Baseline Framework

When considered for deployment on a mobile robot, the baseline framework has limitations in three areas: its integration into the robotic system and the spatial extent it can represent, the reliability of the information stored in the scene graph, and its computational cost.

The first limitation arises from the service-oriented interface of the framework, which is not aligned with robot-native communication patterns (Section 2.6.2) and therefore makes deployment on real robotic platforms less straightforward. The spatial extent of the representation is likewise restricted, as the scene graph is updated based only on the current frame, camera pose, and visible graph elements. This is suitable for local scene understanding, but it provides no mechanism for keeping the graph spatially consistent across multiple rooms, corridors, or revisited locations. Since the framework also lacks loop closure, an area revisited after accumulated pose drift may lead to revisited objects being inserted as new observations, or to updates based on spatial information that is no longer globally consistent. Anchoring the scene graph in a SLAM-based spatial map with loop closure, such as the one provided by Hydra [7], is therefore necessary to extend the framework beyond local frame-by-frame scene understanding.

Beyond these integration aspects, the information stored in the scene graph is only partially reliable. Uncertainty is expressed solely through confidence values and appearance scores. Appearance-based persistence distinguishes repeatedly observed entities from briefly observed ones, but it maintains neither a probability distribution over object classes or relationship labels nor a probabilistic estimate of object existence or spatial position, although such information is required for downstream risk assessment (Section 2.8.4). The relationships stored on the edges are similarly limited. They are inferred by the VLM from a single two-dimensional view, and several of them, such as *in front of* or *behind*, are defined relative to the observer rather than to the scene. An object that is behind another object from one viewpoint may appear beside or in front of it from another, so the same pair of objects can receive contradictory labels as the robot moves, and each update overwrites the previously stored label. Such egocentric relations do not describe a stable property of the three-dimensional scene. Moreover, the scene graph depends directly on the VLM output and therefore inherits the failure modes described in Section 2.9.2. Since the parsing and graph-combination steps mitigate only some of these effects, the VLM output must be bounded, checked, and converted into a consistent graph representation before it can serve as reliable input for risk assessment.

Finally, the runtime performance of the framework is constrained by its per-frame inference load. Segmenting every frame with SAM in automatic mode and invoking the VLM on every frame limits the achievable update rate of the scene graph on embedded hardware, for the reasons given in Sections 2.7.1 and 2.9.2. Together, these limitations motivate the extensions proposed in this thesis: porting the framework to ROS 2, anchoring the scene graph in a SLAM-based spatial map with loop closure, introducing a probabilistic object representation, deriving object relationships from the three-dimensional geometry of the scene, constraining and validating the VLM output, and reducing the per-frame inference load to enable near-real-time operation.

2.7. Lightweight Promptable Segmentation with NanoSAM

The baseline framework uses the Segment Anything Model (SAM) to generate class-agnostic object candidates from the RGB component of each observation [1], [30]. Although SAM achieves strong zero-shot segmentation quality, its image encoder is computationally expensive. On an embedded robotic platform, segmentation must share the available compute and memory with spatial mapping, retrieval, and vision-language model inference, which makes the original model unsuitable for near-real-time operation. In this thesis, SAM is therefore replaced by NanoSAM, a distilled SAM variant developed by NVIDIA for real-time inference on Jetson devices using NVIDIA TensorRT [33], [34]. This section introduces the architecture of SAM that NanoSAM inherits, the knowledge distillation approach used to obtain NanoSAM, its deployment and performance characteristics, and its use in the proposed framework.

2.7.1. Architecture of the Segment Anything Model

SAM formulates segmentation as a promptable task: given an image and a prompt that indicates what to segment, the model returns a valid segmentation mask, even if the prompt is ambiguous [30]. The model is class-agnostic, meaning that it delineates image regions without assigning semantic labels. It consists of three components. The image encoder is a Vision Transformer (ViT) [35], pre-trained as a masked autoencoder [36], which processes an input image resized and padded to 1024×1024 pixels. The image is divided into 16×16 pixel patches, which are processed by transformer blocks using mostly windowed and occasionally global self-attention, and a convolutional neck compresses the result into a dense image embedding of $64 \times 64 \times 256$.

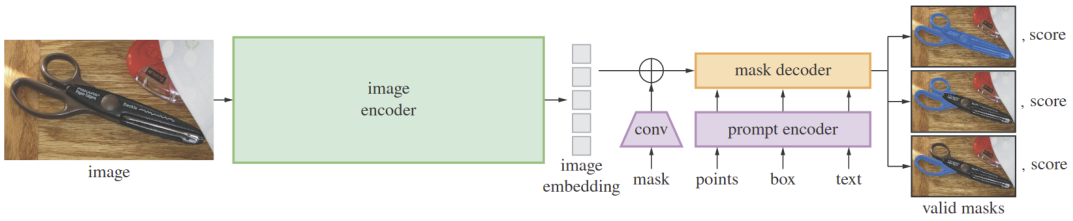


Figure 2.9.: Overview of the Segment Anything Model (SAM) architecture. The model consists of an image encoder, a prompt encoder, and a lightweight mask decoder that produces segmentation masks conditioned on input prompts. Reproduced from Kirillov et al. [30].

In this work, the model is prompted exclusively with single foreground points placed on a regular grid. The prompt encoder maps each point to a 256-dimensional token by combining a random Fourier positional encoding of its image coordinates with a learned embedding identifying it as a foreground point. Since the same positional encoding is applied to the locations of the image embedding, the prompt token and the image features share a common coordinate representation, which allows the decoder to relate the prompt to the corresponding image region. The mask decoder is a lightweight two-way transformer, in which the prompt token and a set of learned output tokens attend to each other and to the image embedding, while the image embedding in turn attends to the tokens. After upsampling the updated embedding, each output mask token is mapped to a per-pixel classifier that yields mask logits, and a dedicated output token predicts the intersection-over-union (IoU) of each mask with the target object [30].

Two design properties of SAM are particularly relevant for this thesis. First, the computational cost is distributed asymmetrically: the image encoder accounts for the large majority of the computation but is executed only once per image, after which the embedding can be reused for an arbitrary number of prompts at low additional cost. Second, since a single point can be ambiguous, for example when it lies on a part of an object as well as on the object as a whole, the decoder predicts several candidate masks at different levels of granularity together with their predicted IoU. For automatic segmentation without user input, SAM prompts the image with a regular grid of foreground points and filters the resulting masks by predicted IoU, mask stability, and non-maximum suppression (NMS). This automatic mode multiplies the decoder cost by

the number of grid points and, combined with the large ViT encoder, is not compatible with near-real-time operation on embedded hardware.

2.7.2. Knowledge Distillation and MobileSAM

Knowledge distillation is a model compression technique in which a compact student network is trained to reproduce the outputs or intermediate representations of a larger teacher network [37]. Since the student is supervised by the teacher rather than by ground-truth annotations, a substantial part of the teacher's behaviour can be transferred to a model with considerably lower computational cost.

MobileSAM applies this principle to SAM [38]. Zhang et al. observed that jointly distilling the image encoder and the mask decoder is difficult, because both components are optimised in a coupled manner. They therefore proposed decoupled distillation, in which only the image encoder is distilled: a lightweight TinyViT-based encoder [39] is trained to regress the image embeddings of SAM's ViT-H encoder. Because the student produces embeddings in the same format as the teacher, the original prompt encoder and mask decoder can be reused without retraining. In this way, MobileSAM reduces the image encoder from more than 600 million to approximately five million parameters while retaining segmentation behaviour comparable to the original model [38].

2.7.3. NanoSAM

NanoSAM extends the decoupled distillation strategy of MobileSAM by a further compression step targeted at embedded NVIDIA hardware [33]. The MobileSAM image encoder acts as the teacher, and a convolutional network acts as the student. The student uses an ImageNet-initialised ResNet-18 backbone [40], which processes the 1024×1024 input through a stem and four stages of residual blocks, progressively halving the spatial resolution and doubling the number of channels until a feature map of $32 \times 32 \times 512$ is obtained. A convolutional neck then reduces the channels, upsamples the features to 64×64 using a transposed convolution, and projects them to 256 channels. Finally, a learned positional embedding of $64 \times 64 \times 256$ is added. This positional embedding compensates for the largely translation-equivariant nature of convolutions, which would otherwise prevent the student from reproducing the position-dependent component of the transformer teacher's embedding.

The student is trained on unlabelled images from the COCO 2017 training set [41] by regressing the teacher embedding with a Huber loss [42]. Since no segmentation annotations are required, the distillation relies entirely on the teacher's representations. Because the embedding format is preserved, NanoSAM reuses the MobileSAM prompt encoder and mask decoder without modification. The resulting pipeline therefore consists of the distilled ResNet-18 encoder followed by the unchanged MobileSAM decoder.

For deployment, both components are compiled into TensorRT engines. TensorRT converts a trained network into a hardware-specific engine by applying, among other techniques, layer fusion, reduced-precision execution, and automatic selection of kernel implementations for the target GPU [34]. The NanoSAM encoder is executed in half precision (FP16), whereas the decoder is executed in single precision (FP32) [33]. Since the engines are specific to the GPU architecture and TensorRT version, they must be built on, or for, the target platform. Table 2.1 summarises the latency and accuracy reported by the developers, where accuracy is measured by prompting the model with ground-truth bounding boxes from the COCO 2017 validation set.

Table 2.1.: Latency on Jetson AGX Orin and mean IoU on COCO 2017 validation with ground-truth box prompts, as reported in [33].

	Latency (ms)		mIoU		
Model	Encoder	Full pipeline	All	Small	Large
MobileSAM	35.0	39.0	0.728	0.658	0.804
NanoSAM (ResNet-18)	4.2	8.1	0.706	0.624	0.796

2.7.4. Limitations

NanoSAM has two limitations that affect its use in the proposed framework. First, although NanoSAM uses the same decoder architecture as SAM, its TensorRT-based predictor only exposes prompt-based inference and does not include the automatic mask generation pipeline of SAM, which therefore has to be implemented separately as a post-processing wrapper around the model. In particular, object candidates must be obtained through externally generated prompts, and duplicate masks arising from neighbouring prompts must be removed by the application. Second, the distillation reduces segmentation accuracy, particularly for small objects, which may nevertheless be safety-relevant in cluttered indoor environments. Consequently, NanoSAM masks must be regarded as uncertain object proposals that require subsequent filtering, tracking, and semantic interpretation.

2.8. Multi-Object Tracking and Data Association

Segmentation, as discussed in Section 2.7, describes the objects visible in a single image, but it does not establish whether a segmented region corresponds to an object that has already been observed. For a mobile robot operating over an extended period, this distinction is essential. As the robot moves, the same physical object reappears from different viewpoints, at different distances, and

under varying degrees of occlusion. If every observation were treated independently, the scene representation would accumulate duplicate objects, and information gathered over time could not be attributed to the same physical entity. Multi-object tracking addresses this problem by maintaining persistent object representations over time and relating each new observation to them.

2.8.1. The Data Association Problem

The central task in multi-object tracking is data association: deciding which current observation corresponds to which previously established object. If an observation is sufficiently consistent with an existing object representation, it is associated with that object and contributes to its refinement. Otherwise, it is treated as a previously unseen instance and used to initialise a new object representation. Conversely, an existing object that is not observed in the current frame is not necessarily absent, since it may be occluded, outside the field of view, or missed by the perception system.

The reliability of this decision depends on the evidence used to compare observations with existing objects. Image-space evidence, such as the position or overlap of regions in the image, changes substantially when the camera moves, even if the observed object remains stationary. A table, for example, can occupy entirely different image regions when viewed from opposite sides of a room. For a mobile robot, apparent motion in the image is therefore caused by both object motion and ego-motion. When depth measurements and camera poses are available, observations can instead be expressed in a common three-dimensional reference frame, in which the position and extent of a static object remain approximately constant across viewpoints. Three-dimensional information therefore provides more stable evidence for associating observations that are far apart in time or taken from very different viewpoints.

Two types of association error must be considered. A false association merges observations of distinct physical objects into a single representation, while a missed association splits one physical object into duplicate representations. Both errors propagate directly into any scene representation built on top of the tracking result.

2.8.2. Object-Level SLAM with Fusion++

A relevant precedent for persistent, object-centric scene representation is Fusion++, an object-level SLAM system proposed by McCormac et al. [43]. Instead of representing the environment exclusively through a single monolithic geometric model, Fusion++ builds a persistent map composed of individual object instances. The system operates on RGB-D input and uses Mask R-CNN instance segmentation [44] to detect object regions in incoming frames. The corresponding depth measurements are used to initialise a separate volumetric TSDF model for each newly

detected object, with the voxel resolution of each model chosen according to the object's size. A coarse TSDF of the background is maintained in parallel to support camera tracking [43].

Data association in Fusion++ couples the persistent 3D models with the 2D segmentation output. Existing object models are rendered into the current camera view, and each new instance mask is compared with these renderings using their intersection-over-union. A mask is associated with an existing object if the overlap exceeds a threshold. Otherwise, it initialises a new object [43]. When an object is observed again, the new depth measurements are fused into its existing TSDF rather than creating an independent copy. Since an object is rarely fully visible from a single viewpoint, repeated observations from different viewpoints gradually complete its geometry. Persistent association therefore serves two purposes: it preserves object identity, and it allows the spatial representation of each object to be refined over time.

Fusion++ also maintains uncertainty-related quantities for each object. Semantic identity is not fixed at initialisation. Instead, the class probabilities predicted by Mask R-CNN are fused over successive observations, so that the class estimate of an object can change as further evidence becomes available, independently of the object's identity in the map. In addition, each object carries an existence probability, which is derived from how often the object is detected compared with how often it is expected to be visible but is not detected. Objects whose existence probability falls too low are removed, which suppresses spurious detections without discarding objects after a single missed observation [43]. Finally, the object instances are organised in a six-degree-of-freedom pose graph and used as landmarks for tracking, relocalisation, and loop-closure detection. As a result, objects become persistent elements of the spatial map rather than transient detections tied to individual frames.

2.8.3. Limitations of Fusion++

Despite these properties, Fusion++ is not suitable as the basis of the proposed framework. Its semantic perception is closed-set, since objects are detected and classified by Mask R-CNN, which is trained on a fixed set of categories [43]. Objects outside this set are not represented as distinct entities, which conflicts with the requirement of open-vocabulary identification of risk-relevant objects. Furthermore, Fusion++ produces an object-level map rather than a scene graph. It contains no relationships between objects, descriptive object properties, or higher-level spatial structures such as rooms, and therefore lacks the relational context on which downstream safety reasoning depends.

In addition, Fusion++ assumes a static environment [43]. Its existence probability indicates whether an object is repeatedly detected, but not whether a previously observed object is still at its last observed position, even though this uncertainty is itself risk-relevant in dynamic environments. Finally, maintaining a separate volumetric model for every object, combined with a two-stage segmentation network, results in memory and computational costs that grow with the

number of objects. This conflicts with the requirement of near-real-time operation on embedded hardware.

2.8.4. Relevance for Persistent Scene Representations

Despite these limitations, the central idea behind Fusion++ remains relevant to this work. Image observations can be combined with depth to build persistent object representations, and new observations can be linked back to them over time. For a scene graph that supports risk assessment, this means that each node should represent a physical object rather than a single detection. Information collected at different times, such as labels, properties, or positions, can then be gathered in one place instead of being spread across duplicate entries. The semantic label of an object should also be treated as an estimate that can improve with more observations, rather than as a fixed value assigned when the object is first seen.

In the same way, the presence of an object is better described by a probability than by a simple yes or no. This makes it possible to tell apart objects that have been seen many times, objects that were seen only briefly, and objects that have not been observed for some time. Such distinctions matter for risk assessment, since an object that may have moved should be treated differently from one whose position is well known. These ideas, however, need to be applied under conditions that Fusion++ does not consider, including open-vocabulary object identification, relations between objects, changing object positions, and near-real-time operation on embedded hardware.

2.9. Visual Retrieval and Vision-Language Models

Class-agnostic segmentation produces object masks without semantic meaning. Labels can be assigned to such masks by visual retrieval, which reuses the label of a previously seen, similar object, or by a vision-language model (VLM), which names an object directly from its appearance. Retrieval is computationally cheap but limited to what has been observed before, whereas a VLM can describe arbitrary objects at a considerably higher computational cost. The baseline framework combines both approaches [1], and its concept forms the starting point for the pipeline developed in this thesis.

2.9.1. Retrieval-Augmented Perception

Visual retrieval requires an image encoder that converts an image into a fixed-length vector, referred to as an embedding, such that similar images obtain similar embeddings. A widely used encoder is the image encoder of Contrastive Language–Image Pre-training (CLIP) [45]. In its ViT-B/32 variant, the image is resized to 224×224 pixels, divided into 32×32 pixel patches, and

processed by a Vision Transformer [35]. A summary token that collects information from all patches is projected to a vector of 512 values and normalised to unit length.

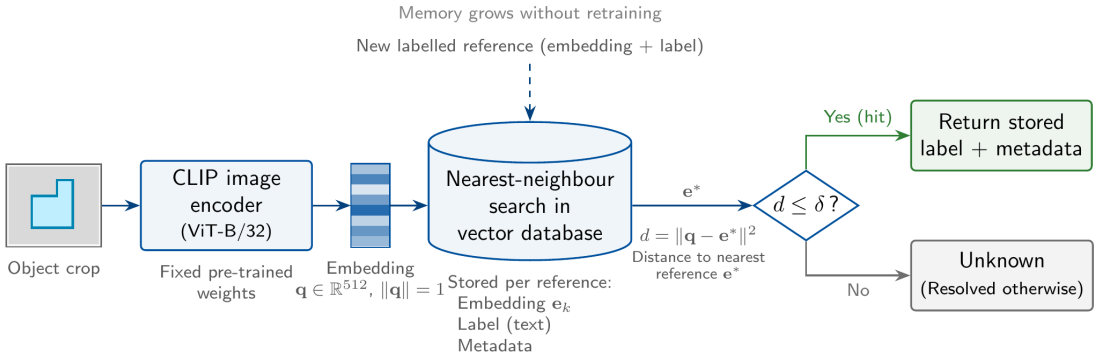


Figure 2.10.: Principle of visual retrieval: the crop embedding is compared with stored references, and the nearest label is returned if its distance lies below the threshold δ.

The meaning of these embeddings results from the encoder weights, which were determined once during pre-training and remain fixed afterwards. CLIP was trained on approximately 400 million images with their captions; during training only, a separate text encoder maps each caption into the same space, and both networks are optimised so that each image lies close to its own caption and far from all others [45]. To achieve this, the image encoder must extract features that describe objects, materials, and scenes. Images with similar content must lie close to similar captions and are therefore also close to each other. After training, images are compared using the image encoder alone. To compare a new image with a stored reference image, their embeddings are compared. Let $\mathbf{q}$ denote the embedding of the new image (the query) and $\mathbf{e}$ the embedding of a stored reference. Their squared Euclidean distance $\|\mathbf{q} - \mathbf{e}\|^2$, i.e. the sum of the squared differences of all 512 values, is directly related to their cosine similarity $\mathbf{q}^\top \mathbf{e}$. Since both embeddings have unit length,

$$\|\mathbf{q} - \mathbf{e}\|^2 = \|\mathbf{q}\|^2 + \|\mathbf{e}\|^2 - 2\,\mathbf{q}^\top \mathbf{e} = 2\left(1 - \mathbf{q}^\top \mathbf{e}\right). \tag{2.1}$$

The distance is therefore zero for identical embeddings and increases as the cosine similarity decreases, so both measures rank stored references in the same order.

Retrieval-augmented methods use stored knowledge to support the interpretation of new input [46]. In visual retrieval, a memory stores the embedding of each labelled reference image together with its label as plain text and optional metadata. Vector databases such as ChromaDB organise these embeddings in an approximate nearest-neighbour index for efficient search [47]. A new image is embedded with the same encoder, and if the distance to the nearest reference lies below a threshold, the stored label is returned; otherwise, the image remains unknown. The memory thus does not interpret images itself but determines whether an image closely resembles one that has already been labelled. New objects can be added at any time without retraining, since the learning takes place in the stored references rather than in the encoder.

In the baseline framework, this principle is implemented as Retrieval-Augmented Perception (RAP) [1]. For each frame, the object cutouts generated by the Segment Anything Model are embedded with CLIP and compared with a visual knowledge base in ChromaDB. The labels of sufficiently similar references are passed to the VLM as label candidates, which guide its interpretation and help to keep labels consistent across observations. RAP therefore supports the VLM in every inference step but does not replace it. The same principle could also be used to resolve an object directly when a sufficiently similar reference exists, so that the VLM would only be required for the remaining objects.

Retrieval is fast, and repeated observations of the same object receive consistent labels. However, it can only return labels that are already stored, an incorrect entry is propagated to all similar observations, and the threshold trades the number of resolved objects against the number of accepted errors. Moreover, the encoder summarises the entire image including its background, because nothing indicates which pixels belong to the object. Large uniform surfaces such as walls and ceilings in the same room can therefore appear more similar to each other than to the same surface type elsewhere.

2.9.2. Vision-Language Models

A VLM consists of a vision encoder, which converts image patches into feature vectors, a connector, which maps these features into the input space of a large language model, and the language model, which generates the answer token by token from the visual input and a text prompt [48]. Since each token requires one forward pass, the latency grows with the length of the answer. A pre-trained VLM can be applied to new tasks without retraining, as the task is specified in the prompt.

Compared with conventional object detectors and segmentation networks, which are trained on a fixed set of classes, VLMs are open-vocabulary: they can name objects, materials, and states that were never defined as classes in advance, and they can describe properties beyond the object category, such as the condition of an object or its relation to its surroundings. This capability results from pre-training on large and diverse image–text data. At the same time, the generated text is not restricted to what is visible in the image. A well-documented failure mode is object hallucination, in which a model describes objects or attributes that are not present, often because they frequently co-occur with the visible content in the training data [49]. The flexibility of VLMs is therefore accompanied by the risk of plausible but incorrect answers.

In the baseline framework, the VLM is the central reasoning component [1]. For each frame, it receives the RGB image, the part of the existing scene graph expected to be visible, and the label candidates from RAP, and returns an updated scene graph with labels, descriptions, states, confidence values, risk-relevant properties, and relationships. The output is parsed, converted into 3D coordinates, and merged into the scene graph. The framework is model-interchangeable, and the study reports formatting errors, repetitions, hallucinated relationships, and unstable location output as typical difficulties [1]. Since the VLM is called for every frame, its inference time also limits the update rate of the scene graph.

In general, VLM output cannot be treated as ground truth. A reported confidence value is generated text rather than a calibrated probability, and models often state high confidence for incorrect answers [50]. VLM output should therefore be validated, for example against confidence thresholds, before it is stored in a persistent environment representation.

2.9.3. The Qwen Vision-Language Model Family

The baseline study evaluated three VLMs for scene graph generation: LLaVA 1.6 Mistral 7B, Qwen2.5-VL 7B, and Phi-4 Multimodal [1]. The models were compared with respect to the compatibility of their input and output formats, their tendency towards repetition and hallucination, and the stability of the generated object locations. Based on this comparison, the Qwen series was identified as the most suitable model family, and later tests of the baseline framework used the newer Qwen3-VL generation, taking advantage of the model-interchangeable design of the framework [1]. This thesis builds on this result. Instead of repeating the comparison between model families, it continues with the best-performing Qwen family and updates the model choice to the latest available generations, with a focus on small models that are suited to embedded hardware.

The Qwen models of Alibaba Cloud are published with open weights in several sizes and can therefore be executed locally on embedded hardware. Table 2.2 lists the relevant generations.

Table 2.2.: Release history of the open-weight Qwen vision-language models.

Generation	Release	Small dense sizes
Qwen2-VL	Aug. 2024	2B, 7B
Qwen2.5-VL	Jan. 2025	3B, 7B
Qwen3-VL	Oct. 2025	2B, 4B, 8B
Qwen3.5	Mar. 2026	0.8B, 2B, 4B, 9B

Qwen2-VL introduced image processing at native resolution [51], and Qwen2.5-VL improved efficiency, object localisation, and document understanding [52]. Qwen3-VL builds on the Qwen3 language models and supports contexts of up to 256K tokens [53]. Qwen3.5 is trained natively on mixed image and text data instead of attaching a vision encoder to a separately trained language model, and by default generates a reasoning trace before its answer, which can improve accuracy but increases latency [54].

The released models differ in size, architecture, and operating mode. Dense models use all of their parameters for every token, whereas mixture-of-experts models activate only a subset of their parameters per token, which reduces computation but not the memory required to store the full model [53]. In addition, Qwen3-VL is released in an instruction-following variant, which answers directly, and a thinking variant, which reasons explicitly before answering [53], while Qwen3.5 combines both behaviours in one model [54]. For embedded deployment, the memory footprint is decisive. The memory required for the weights is approximately the number of parameters

multiplied by the number of bits per parameter, so that an 8B model in 16-bit precision requires about 16 GB, whereas a 4B model stored with about 4 bits per parameter requires roughly 2 to 3 GB. Small dense models, which are commonly also distributed in such reduced-precision formats, are therefore the most suitable candidates when the VLM has to run alongside other perception and mapping components on the same device.

Since the baseline study, a new generation has appeared roughly every six to twelve months, and small models have improved noticeably: on the MMMU-Pro benchmark, Qwen3.5-4B scores 65.4 % compared with 52.0 % for Qwen3-VL-4B [55]. Although general benchmarks do not directly predict task-specific accuracy, this trend indicates that the small Qwen3-VL and Qwen3.5 models are promising candidates for the present work, and that future generations may improve the results further. The model-interchangeable design of the baseline framework allows such newer models to be adopted without changes to the remaining pipeline.

2.10. Embedded Computing Hardware: NVIDIA Jetson Platforms

The proposed framework combines several computationally demanding components, including NanoSAM-based segmentation, retrieval-augmented perception, VLM reasoning, Hydra-based spatial mapping, and risk-annotated scene graph generation. For onboard robotic deployment, these components must run on embedded hardware with sufficient AI compute, memory capacity, and energy efficiency. This section introduces the two NVIDIA Jetson platforms relevant to this work, the Jetson AGX Orin Developer Kit and the Jetson AGX Thor Developer Kit.

2.10.1. NVIDIA Jetson AGX Orin Developer Kit

NVIDIA Jetson AGX Orin is an embedded edge-AI computing platform designed for robotics, computer vision, and autonomous machines. Like other members of the Jetson family, it integrates CPU, GPU, memory, I/O, and AI acceleration into a compact module for onboard computation. Jetson AGX Orin is based on the NVIDIA Ampere architecture, combining a 2048-core GPU with 64 Tensor Cores and a 12-core Arm Cortex-A78AE CPU. It provides up to 275 TOPS of AI performance, 64 GB of memory, and a configurable power range of 15 W to 60 W [56].

Jetson AGX Orin was used as the target platform in the baseline framework [57]. Its compute capacity is sufficient for executing individual perception and reasoning components, such as segmentation and VLM inference with models in the 7B parameter class. However, executing the complete pipeline, consisting of segmentation, retrieval, VLM reasoning, and spatial mapping, concurrently on a single module can be expected to place considerable demands on the available memory and compute resources. Since the CPU and GPU share the same physical memory and power budget, these components compete for the same resources, which may limit the achievable update rate of the scene graph.

2.10.2. NVIDIA Jetson AGX Thor Developer Kit

NVIDIA Jetson AGX Thor is the successor platform within the Jetson family and is designed specifically for physical AI, robotics, and real-time multimodal perception. It is based on the NVIDIA Blackwell architecture, combining a 2560-core GPU with 96 fifth-generation Tensor Cores and a 14-core Arm Neoverse-V3AE CPU. It provides up to 2070 FP4 sparse TFLOPS of AI compute, 128 GB of memory, and a configurable power range of 40 W to 130 W [58]. According to the manufacturer, Jetson AGX Thor provides up to 7.5 times higher AI compute and 3.5 times better energy efficiency than Jetson AGX Orin [58].[1]

This additional capacity is potentially relevant for the proposed framework, as it may allow the perception, reasoning, and mapping components to be executed concurrently on the same embedded platform with fewer resource constraints. Jetson AGX Thor also offers improved support for transformer-based and multimodal workloads. Jetson AI Lab lists Qwen3-VL models, including Qwen3-VL 4B and Qwen3-VL 8B, as available for Jetson AGX Thor through Quick Start Runner and vLLM-based serving configurations [59]. Since the Qwen model series was identified as the best-performing option in the baseline framework [57], Jetson AGX Thor represents a suitable hardware basis for extending the system toward onboard robotic deployment without dependence on an external workstation for model inference.

2.11. Scene Graphs for Downstream Safety Reasoning

This section discusses how a generated scene graph can be used as structured input for downstream risk assessment. SafetyDetect by Mullen et al. is considered as an example of scene-graph-based safety reasoning [9]. The work proposes a dataset and LLM-based approach for detecting anomalous situations in home environments, such as food left outside storage, a stove left on, or harmful objects being accessible to children. In the proposed pipeline, object relations from a scene graph are provided to an LLM, which classifies situations as normal, dangerous, unsanitary, or dangerous for children.

The relevance of SafetyDetect lies in its demonstration that object relations can be important for safety-related reasoning. Many risks cannot be inferred from object identity alone, because the same object may be safe or unsafe depending on its relation to other objects, humans, or locations. A scene graph is therefore useful because it represents these relations explicitly and provides a structured intermediate representation for a downstream reasoning module.

At the same time, SafetyDetect highlights the importance of the scene graph generation stage. The authors state that their method is limited by perception, particularly by how the scene graph is created [9]. For example, a scene graph may represent that an object is inside a room and on the floor, but it may not indicate whether the object is blocking a doorway and creating a

[1] The compute figures of the two platforms are specified at different numerical precisions (FP4 sparse TFLOPS for Jetson AGX Thor and TOPS for Jetson AGX Orin). The comparison therefore reflects the manufacturer's specification rather than a measured result.

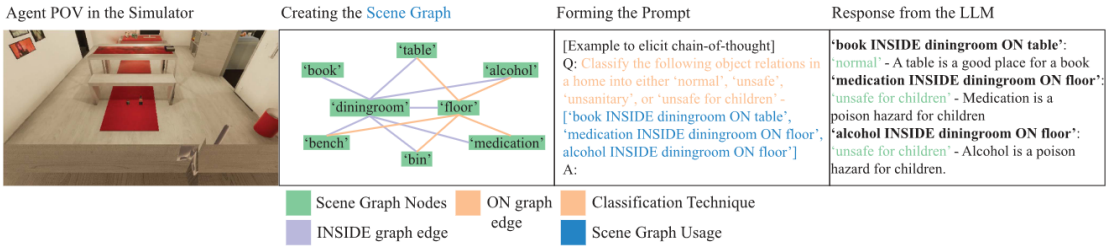


Figure 2.11.: SafetyDetect pipeline showing the agent point of view, scene graph construction, prompt formation, and LLM-based anomaly classification. Reproduced from [9].

tripping hazard. This shows that the downstream safety assessment depends not only on the reasoning capability of the LLM, but also on whether the scene graph contains the required objects, relations, spatial details, and obstruction information.

This limitation is important for the present thesis. SafetyDetect mainly focuses on reasoning after a scene graph has already been generated. In simulation, the scene graph is provided by the VirtualHome environment, while in the real-world TurtleBot experiment an existing scene graph generation method is used before the graph is passed to the LLM. Therefore, SafetyDetect is more relevant as an example of downstream safety reasoning than as a method for constructing a persistent spatial scene graph.

Consequently, SafetyDetect can be understood as evidence for the downstream value of scene-graph-based representations in safety reasoning. However, it also indicates that the effectiveness of such reasoning depends strongly on the quality of the generated scene graph. This supports the central motivation of the present thesis: improving scene graph generation, spatial grounding, persistence, and uncertainty representation is a necessary step toward more reliable dynamic risk assessment in robotic systems.

3. Methodology

This chapter describes the concept of the proposed framework and the reasoning behind its design, whereas software details and parameter values are deferred to Chapter 4. Section 3.1 presents the overall architecture and its central design decisions. The subsequent sections follow the flow of data through the framework: input preparation (Section 3.2), segmentation and persistent object tracking (Section 3.3), semantic labelling (Section 3.4), fusion with the spatial scene graph (Section 3.5), uncertainty representation (Section 3.5.2), and the handling of loop closures and revisited environments (Section 3.6).

3.1. Framework Overview

The structure of the framework follows from the four aspects defined in Section **??**, **??**, namely portability, spatial scalability, uncertainty awareness, and near-real-time operation, and from the limitations of the baseline framework [1] identified in Section 2.6.4, *Limitations of the Baseline Framework*. In essence, it combines Hydra as a spatial backbone with a semantic pipeline that identifies objects and characterises their mobility, and joins the two through persistent object identities. Section 3.1.1 describes how the components are connected, and Section 3.1.2 explains why.

3.1.1. Framework Architecture

The framework consists of three ROS 2 nodes [10] developed in this thesis, the preprocessor, the Phase 1 semantic coordinator, and the scene graph fuser, and of three external software components that run as separate processes. Two of these are used entirely as provided: a Chroma vector database [47], which stores the visual memory used for retrieval, and local inference servers hosting the vision-language model Qwen3.5-4B [54], which is used for object labelling. The third, Hydra [7], builds the layered 3D scene graph with its mesh, object, place, and room layers. It is configured with a label space of numbered slots instead of semantic classes and minimally modified in three respects: objects segmented only after several frames can still form their own object nodes, the map can be saved and restored across sessions, and incoming frames are discarded rather than queued without limit when reconstruction falls behind. Figure 3.1 and Table 3.1 summarise the components and the data exchanged between them.

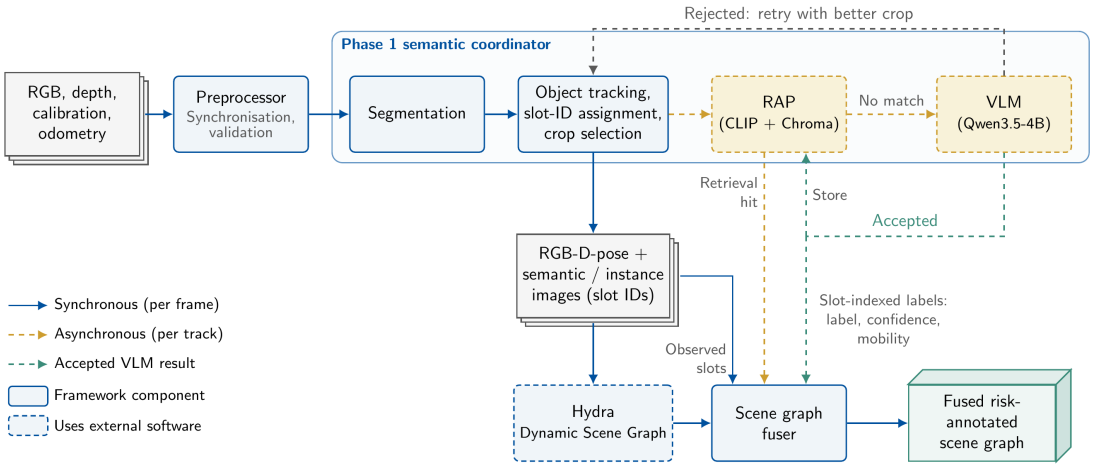


Figure 3.1.: Overview of the proposed framework, showing the synchronous per-frame path (solid) and the asynchronous per-track path (dashed).

The framework operates on two time scales. A per-frame path processes every observation at the rate of the camera and is responsible for geometry, tracking, and mapping, whereas an asynchronous path processes individual objects and is responsible for labels, confidence values, and mobility. The two paths are connected only through numeric slot IDs that identify persistent objects.

Data enters at the preprocessor, which associates the RGB, depth, calibration, and odometry streams in time, rejects frames without a consistent association or with largely invalid depth, converts depth into metric units, and outputs a single RGB-D-pose frame in the common map frame (Section 3.2). Synchronisation is thus resolved once for all downstream components.

On the per-frame path, the Phase 1 coordinator segments the colour image into class-agnostic masks and back-projects them with the depth image into 3D point sets with centroids and bounding boxes. These observations are associated with persistent tracks primarily on the basis of 3D evidence, such as position, extent, and accumulated footprint, with image overlap used only as supporting evidence; unmatched observations create new tracks (Section 3.3). Extended structures such as floors and walls are divided into local segments. Each new track or segment immediately receives a slot ID, and the current masks are rendered into semantic and instance images whose pixel values are these slot IDs. Hydra receives these images together with the RGB-D-pose frame. Since it clusters mesh vertices of the same class into object nodes (Section 2.5.3) and each slot acts as its own class, every Hydra object node corresponds to exactly one track or segment and inherits its persistent identity, even before any semantic label is known.

On the asynchronous path, the coordinator retains for each track the crop with the highest quality score, based on the size, completeness, and sharpness of the view. An eligible track is queued by its identifier, and its crop is taken only when it is dequeued, so that a better view arriving in the meantime is still used. Labelling is retrieval-first: the crop is embedded with CLIP [45] and compared with the visual memory in Chroma (Section 2.9.1), and the stored label, confidence,

and mobility class of a sufficiently similar reference are reused. Otherwise, a crop that emphasises the target object is passed to the VLM, which returns a label, a confidence value, and a mobility class in a fixed format (Section 3.4). The VLM thus only ever sees a single object, never the scene graph. Malformed or low-confidence results are rejected and the track is retried a limited number of times, whereas accepted results are written back into the memory. Labels are published as slot-indexed events, together with the slots observed in each frame, which serve as evidence for the presence model.

The scene graph fuser joins both paths. It caches the label events by slot ID and, at a bounded rate, applies them to every node of Hydra's Dynamic Scene Graph that carries the corresponding slot, so that labels arriving long after an object was mapped are applied as soon as they are available (Section 3.5). It also derives contact between objects and object-to-segment relations from the 3D bounding boxes, takes room and place membership from Hydra's layers, and adds missing object-to-place links where a geometric check against the mesh confirms them. For every slot, it maintains a presence confidence (Section 3.5.2). The result is the fused risk-annotated scene graph, which can be visualised in RViz and consumed by downstream risk assessment.

Two further mechanisms maintain consistency over time. After a loop-closure correction, Hydra deforms its own scene graph (Section 2.5.4), while the coordinator shifts its cached object geometry accordingly and merges tracks that were duplicated around the closure. In addition, the tracker state and Hydra's map can be saved at the end of a session and restored in the next, so that a revisited environment is not mapped a second time (Section 3.6).

Table 3.1.: Information exchanged between the components of the framework.

From	To	Content	Timing
Sensors	Preprocessor	RGB, aligned depth, calibration, odometry	per frame
Preprocessor	Phase 1	Synchronised RGB-D-pose frame	per frame
Phase 1	Hydra	RGB-D-pose frame with slot-ID label images	per frame
Phase 1	Chroma	Crop embeddings (queries, stored VLM results)	per track
Phase 1	VLM	Single-object crops and structured label responses	per track
Phase 1	Fuser	Label events (label, confidence, mobility class)	per track
Phase 1	Fuser	Slots observed in the current frame	per frame
Hydra	Fuser	Dynamic Scene Graph	continuous
Fuser	Downstream	Fused risk-annotated scene graph	bounded rate

3.1.2. Central Design Decisions

The architecture follows from a single underlying problem. Spatial mapping must keep pace with the camera, whereas open-vocabulary identification relies on a vision-language model whose inference takes far longer than the interval between two frames. The baseline coupled both in one per-frame loop, so that the slowest component determined the update rate of the entire scene graph. The following choices separate these two time scales without losing the connection between them.

The first choice concerns the source of the spatial representation. The baseline maintained a frame-based graph without a common map frame or loop closure. Extending it would have meant re-implementing incremental mapping, a mesh-grounded object layer, places, rooms, and loop-closure correction, all of which Hydra already provides in a real-time architecture evaluated on robotic data [7]. Other systems were less suitable: SceneGraphFusion [22] is restricted to a closed set of classes, and ConceptGraphs [8] applies foundation models to every observation, which is precisely the coupling that limits the update rate. Hydra was therefore adopted and deliberately kept close to its original form, with its mapping and optimisation algorithms unchanged. This focuses the contribution of this thesis on what Hydra does not offer, namely open-vocabulary semantics, mobility, and uncertainty, and keeps the framework compatible with future versions of Hydra.

Adopting Hydra brings one of its assumptions with it: object nodes are formed by clustering mesh vertices that share a label, so every pixel must be labelled before integration. Supplying class labels would have required either a closed set of classes, which contradicts open-vocabulary identification, or holding back Hydra's input until the vision-language model has answered, which reintroduces the coupling. Slot IDs resolve this conflict, because they are available immediately after tracking. Hydra can thus cluster objects as designed and at its native rate without ever seeing a semantic label, while the meaning of each slot is supplied later and independently.

Once geometry no longer depends on semantics, labelling can run at its own pace from bounded queues. A slow or unavailable inference server then delays labels but cannot stall segmentation, tracking, or mapping, and a full queue defers pending work instead of blocking the pipeline. The price is that a newly observed object appears in the map before it has a label. This is acceptable, since such objects are explicitly marked as unlabelled and are already geometrically present, so that downstream components can treat them as obstacles.

Decoupling makes the cost of the model tolerable, but it neither reduces this cost nor makes the output more reliable. Since the same objects are observed repeatedly in indoor environments and many resemble objects already labelled, retrieval resolves a growing share of the requests at a fraction of the cost. The fixed output format, in turn, replaces the free-form graph updates of the baseline, whose errors passed directly into the scene representation, with an output that can be validated while remaining open-vocabulary. Because an incorrect memory entry would propagate to every similar object, only validated results are stored.

The last choice concerns how semantic information enters the scene graph. Because labels arrive late and objects move out of view, the graph has to express how much it knows about each object. The fuser therefore never alters Hydra's scene graph and attaches all semantic and confidence information as derived metadata. Relationships are derived from 3D geometry rather than generated by the model, which makes them properties of the scene instead of viewpoint-dependent impressions such as *in front of* or *behind*, and relieves the model of inferring all spatial relationships of the visible scene in every step, as in the baseline. Where the baseline deleted objects once their appearance score fell below a threshold, the presence confidence is reset on observation and decays otherwise, slowly for static objects and considerably faster for humans, animals, and mobile robots, which have probably moved; the rates are configurable. Only tracks confirmed as dynamic are eventually withdrawn from the fused output, and the underlying map

is never pruned. Downstream risk assessment can thus distinguish objects whose state is well known from those whose state has become uncertain.

In summary, Hydra provides spatial consistency and scalability, slot IDs connect geometry and semantics without making either wait for the other, asynchronous and retrieval-first labelling keep open-vocabulary identification compatible with near-real-time operation, confidence-annotated fusion makes the remaining uncertainty explicit, and the use of ROS 2 and standard network interfaces allows the framework to be integrated into a robotic system without changes to its components.

3.2. Data Acquisition and Preprocessing

The components described in Section 3.1 require each observation to combine visual, geometric, and pose information that refers to the same instant. The sensor data, however, consists of independent timestamped streams of colour images, depth images, odometry, and camera calibration, which are produced at different rates. The preprocessing stage combines these streams into a single *prepared RGB-D-pose frame*, which constitutes the input contract of the framework. Since every downstream component consumes this frame, synchronisation is resolved once at the entry of the framework rather than separately in each component. The stage processes each observation as a sequence of steps, illustrated in Figure 3.2, in which a frame is passed on only if it satisfies both temporal and geometric consistency and is rejected otherwise.

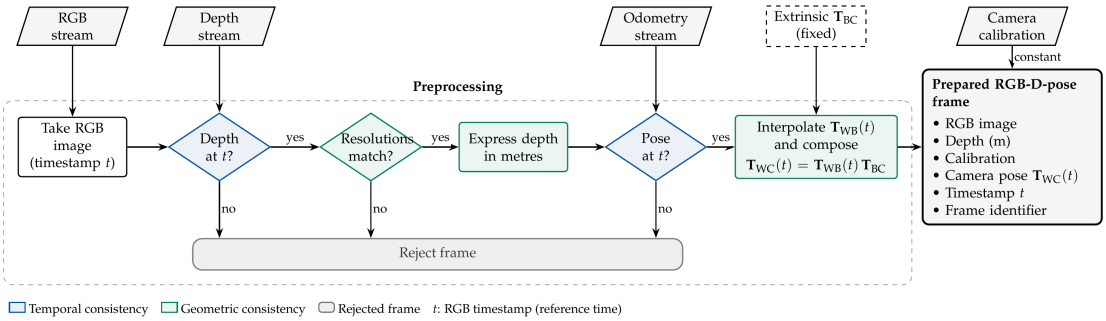


Figure 3.2.: Preprocessing of the input streams into a prepared RGB-D-pose frame.

Each frame is anchored to an RGB image, whose timestamp t serves as the reference time for all subsequent steps. The first step checks whether a depth image with approximately the same timestamp is available. If no such depth image exists, the frame is rejected, since the colour image cannot be related to the 3D structure of the scene. The second step checks geometric consistency between the two images, which must have identical resolution, so that each colour pixel corresponds to the same depth pixel and a segmentation mask computed on the RGB image can be lifted directly to 3D. A frame that violates this condition is likewise rejected. The depth image is then expressed in metric units, which are used throughout the framework; sources that provide depth in other units are converted at this stage. Depth values are, however, not filtered here, since range filtering is performed in Phase 1 (Section 3.3), where depth is first used.

The third step checks whether the pose of the robot at time t can be determined from the odometry. Since odometry is generally not sampled at the same instants as the camera, the base pose $\mathbf{T}_{\mathrm{WB}}(t)$ is interpolated to the RGB timestamp, which reduces the geometric error that would otherwise arise when an image captured during motion is projected with a pose from a slightly different time. If no sufficiently close odometry is available, the frame is rejected. This reflects a deliberate design choice: Hydra [7] integrates every received frame into a persistent map, so discarding an inconsistent frame is preferable to integrating geometrically inconsistent data. Since the odometry describes the pose of the robot base rather than that of the camera, the camera pose is obtained by composing the interpolated base pose with the fixed transform from the robot base to the camera:

$$\mathbf{T}_{\mathrm{WC}}(t) = \mathbf{T}_{\mathrm{WB}}(t)\,\mathbf{T}_{\mathrm{BC}}, \tag{3.1}$$

where $\mathbf{T}_{\mathrm{WB}}(t) \in SE(3)$ denotes the pose of the robot base in the world frame at time t, $\mathbf{T}_{\mathrm{BC}} \in SE(3)$ denotes the calibrated extrinsic transform from the robot base to the camera, and $\mathbf{T}_{\mathrm{WC}}(t)$ is the resulting camera pose. The world frame is the fixed reference frame of the odometry source. Since $\mathbf{T}_{\mathrm{BC}}$ is constant, the camera is assumed to be rigidly mounted on the robot.

Finally, a frame that has passed all checks is assembled into the prepared RGB-D-pose frame. It contains the RGB image, the depth image in metres, the intrinsic camera calibration, the camera pose $\mathbf{T}_{\mathrm{WC}}(t)$, the timestamp t, and a unique frame identifier. The calibration is not part of the sequential checks, as the intrinsic parameters remain constant during operation and are attached to every frame.

The preprocessing stage is kept intentionally minimal, as it lies on the per-frame path of the framework and any latency it introduces delays all subsequent components. It therefore performs only the operations needed to establish the consistency conditions described above. Furthermore, the method is independent of the data source: applying it to the simulated uHumans2 dataset [11] or the real-world OpenLORIS-Scene dataset [12] changes only the sensor-specific parameters, such as the base-to-camera transform and the depth unit, while the sequence of checks and Equation (3.1) remain unchanged .

3.3. Object Segmentation and Persistent Tracking

Class-agnostic segmentation of the frames prepared in Section 3.2 yields an unordered set of masks without identity. For each mask, the framework must decide whether it shows a new physical object or a further observation of a known one, and it must maintain a persistent three-dimensional estimate of every object. The two association errors identified in Section 2.8.1 have strongly asymmetric consequences here. If two objects are merged into one track, its geometry describes neither of them and the crop passed to the vision-language model shows parts of both, so the label is wrong for both; since label and geometry belong to the track as a whole, this cannot be repaired downstream. If one object is fragmented into several tracks, the representation becomes redundant, but each fragment remains correct and can, in principle, be merged later. The method therefore follows a single design principle: whenever the evidence is ambiguous, fragmentation is preferred over over-merging.

3.3.1. Object Proposals and 3D Observations

Object proposals are generated by a promptable model of the Segment Anything family [30], prompted with a regular grid of foreground points. A common interface returns a set of class-agnostic masks, so the remainder of the method is independent of the model behind it. Two interchangeable models are supported: the original SAM with a ViT-B encoder and NanoSAM (Section 2.7), which has a considerably lower latency on embedded hardware but reduced accuracy, particularly for small objects [33]. SAM uses its own automatic mask generation, whereas for NanoSAM the equivalent post-processing is realised as a wrapper (Section 2.7.4). In both cases, duplicate masks from neighbouring prompts are removed by non-maximum suppression, and implausibly small masks are discarded, as they rarely represent complete objects and provide too few depth measurements. To bound the cost of all subsequent stages, the number of proposals per frame is limited, retaining the masks with the highest predicted quality. Since the choice of model trades proposal quality against update rate, it is determined experimentally in Chapter **??**. The result is a set of masks $\mathcal{M}_t = \{m_1, \ldots, m_N\}$ for frame t.

Each mask is converted into a three-dimensional observation. Depth is taken only from the largest connected region of the mask, because masks often contain detached fragments and boundary regions whose depth belongs to the background. A pixel (u, v) with valid depth d is back-projected with the pinhole model and transformed into the map frame using the camera pose of Equation 3.1:

$$\mathbf{p}^c = d \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}, \qquad \mathbf{p}^m = \mathbf{R}_c^m \, \mathbf{p}^c + \mathbf{t}_c^m, \tag{3.2}$$

where f_x, f_y, c_x and c_y are the focal lengths and principal point, and $(\mathbf{R}_c^m, \mathbf{t}_c^m)$ is the transformation from the camera to the map frame. The resulting point set $\mathcal{P}_i$ is summarised as an observation $o_i = (\mathcal{P}_i, \mathbf{c}_i, \mathcal{B}_i, \mathbf{b}_i)$ with the axis-aligned three-dimensional bounding box $\mathcal{B}_i$ of the points in the map frame, its centre $\mathbf{c}_i$, which serves as the centroid of the observation, and the two-dimensional bounding box $\mathbf{b}_i$ of the mask in image coordinates. Since thin structures such as floors, ceilings and walls yield boxes of almost zero thickness, for which a volumetric overlap is not meaningful, every box dimension below a minimum thickness is padded to that thickness before $\mathbf{c}_i$ and the volume are computed. The three-dimensional quantities provide viewpoint-invariant evidence for association, while $\mathbf{b}_i$ is used for the image-space comparison and for extracting the object crop. A mask without reliable geometry, for instance because most of its depth values are invalid, is discarded for the current frame, and no observation, crop or track is created from it. The object is not ignored: it is expected to be observed again with meaningful depth as the robot moves, and is tracked then. Discarding the mask only keeps inaccurate data out of the persistent representation; moreover, an observation without geometry could not be associated by the three-dimensional evidence and would only open short-lived tracks.

Depth values outside a valid sensing range are excluded during back-projection, completing the filtering deferred in Section 3.2. This serves two purposes. First, depth accuracy decreases with distance: for a stereo camera, the error grows approximately quadratically, $\delta z \approx z^2 \, \delta_d / (f \, b)$, with depth z, focal length f, baseline b and disparity error δ_d [60]. Distant points therefore yield noisy centroids and inflated boxes, which weaken the association cues and, through the accumulated

extent (Section 3.3.3), could permanently distort a track. Measurements below the sensor's minimum operating distance are excluded for the same reason. Second, the restriction limits the computational cost per frame, since masks of distant objects are discarded before association, crop selection and labelling, so that the expensive vision-language model inference is applied only to objects near the robot. Distant objects are observed again once they enter the valid range, and nearby objects are generally the most relevant for immediate risk assessment.

3.3.2. Evidence-Based Association

Association is performed primarily in three dimensions, because the position and extent of a static object in the map frame are approximately constant across viewpoints, whereas its image appearance is not (Section 2.8.1). Fusion++, in contrast, relies on a single image-space overlap criterion [43]. An observation o is compared only with candidate tracks τ in its spatial neighbourhood, which are retrieved by a grid search over a uniform grid in the ground plane: each track is registered in the cells covered by its accumulated extent and by its centroid, and the candidates are the tracks registered in the cells covered by the observation's footprint or lying within the centroid acceptance radius. Since a track outside these cells can pass neither the overlap nor the centroid test, it cannot satisfy the acceptance condition below, so the grid search reduces the cost per observation without changing the association result. The evidence is organised hierarchically: the three-dimensional overlap is the primary evidence, while centroid distance and image overlap are secondary evidence that is consulted only when the overlap is not conclusive.

The primary evidence is the volumetric overlap between the observation's bounding box and the accumulated extent of the track, which reflects that a static object occupies the same region of the map from any viewpoint. The accumulated extent is used because successive views often capture different parts of an object, and overlap is required on both horizontal axes, so that boxes that merely touch do not count. Two levels are distinguished. A partial overlap shows a shared region but is ambiguous, since adjacent objects also overlap partially. A near-complete inclusion of the observation in the track's extent, referred to as containment, is the expected outcome of a re-observation, especially for large objects such as floors and walls that are seen only in part. In addition, a height test requires only the vertical extents to overlap or to be separated by at most a small gap, irrespective of horizontal position. A volumetric overlap therefore implies height compatibility, but not vice versa, since boxes at the same height may lie side by side.

The secondary evidence resolves the ambiguity of a partial overlap. The centroid distance separates objects whose boxes intersect but whose centres lie apart. It is mapped to a similarity by a Gaussian kernel whose width scales with object size, because the centroid of a large, partially observed object shifts more between views. The image overlap between $\mathbf{b}$ and the box of the track's last observation is independent of depth and pose errors and strongly indicates continuity between consecutive frames, but its value decreases quickly as the viewpoint changes.

Each test k yields a similarity $\sigma_k(o,\tau) \in [0,1]$ and a binary verdict $\pi_k(o,\tau) \in \{0,1\}$. With π_{con}, π_{ov}, π_{h}, π_{cen} and π_{img} denoting the verdicts for containment, partial overlap, height, centroid distance and image overlap, a match in association mode μ is accepted if

$$s^{(\mu)}(o,\tau) = \frac{\sum_k w_k^{(\mu)}\,\sigma_k(o,\tau)}{\sum_k w_k^{(\mu)}} \geq s_{\min}^{(\mu)} \quad \wedge \quad \neg\,\chi(o,\tau) \quad \wedge$$
$$\begin{cases} \text{no further condition} & \text{if } \pi_{\text{con}} = 1, \qquad \text{(Tier 1)} \\ \pi_{\text{cen}} \vee \pi_{\text{img}} & \text{if } \pi_{\text{ov}} = 1 \text{ and } \pi_{\text{con}} = 0, \qquad \text{(Tier 2)} \\ \pi_{\text{h}} \wedge \pi_{\text{cen}} \wedge \pi_{\text{img}} & \text{otherwise,} \qquad \text{(Tier 3)} \end{cases} \tag{3.3}$$

where $w_k^{(\mu)}$ are mode-dependent weights with the largest weight on the volumetric overlap, and the score rejects matches with only marginal support. Under containment (Tier 1), the overlap alone decides, based on the assumption that a near-complete inclusion in a track's extent indicates a re-observation of the same object; the cases in which this assumption fails, such as an extent enlarged by an earlier erroneous association or a small object stored inside a larger one, are examined in the evaluation. Under partial overlap (Tier 2), one secondary cue must confirm the match, and otherwise (Tier 3), both secondary cues and the height test must agree. Tier 3 mainly serves young tracks of small objects. Depth errors displace points along the viewing ray [60], which is approximately horizontal for a forward-facing camera, so the observed box is shifted mainly in the ground plane. Because the volumetric overlap is multiplicative across axes, small horizontal shifts can prevent a sufficient overlap for the same object, while its vertical extent is barely affected. The height test is therefore reliable in this situation, and the agreement of both secondary cues guards against horizontally adjacent objects. An image-only match that contradicts the three-dimensional evidence, such as an object in front of a wall, falls into Tier 3 and fails on the centroid. Since containment implies partial overlap, which implies height compatibility, the rule is equivalent to a quorum of three agreeing verdicts among the five tests, which is how it is implemented. Finally, the predicate χ rejects a match whose observation volume is grossly inconsistent with the smoothed volume of the track; as this reference is formed from individual observations, χ cannot detect an extent that has grown through an erroneous association.

Two association modes depend on the time $\Delta t = t - t_\tau$ since the track was last observed. In the recent mode, the viewpoint has changed little and image overlap is trusted. In the revisit mode, the object is seen again after a longer absence, possibly from a very different viewpoint, so much stronger image agreement is required and centroids are compared in the ground plane only, since the height of a partially observed object's centroid depends on the visible part.

Before any track is updated, all observations of a frame are evaluated against the track states at the beginning of the frame. Segmentation often emits a merged region together with its parts, so a mask largely covered by smaller masks already matched to distinct tracks is suppressed. Nested masks that are admissible for the same track describe the same visible surface twice; only one of them is retained, where a broader mask replaces a nested part if it adds new visible area with consistent three-dimensional support, so that the track can grow as more of the object becomes visible. Each remaining observation o_i is then assigned independently to its admissible track with the highest association score,

$$\tau^*(o_i) = \arg \max_{\tau_j \in \mathcal{E}(o_i)} s^{(\mu)}(o_i, \tau_j), \tag{3.4}$$

where $\mathcal{E}(o_i)$ denotes the set of tracks for which o_i satisfies Equation 3.3; if $\mathcal{E}(o_i)$ is empty, o_i starts a new track. Since all observations are assigned against the same frame-initial track states, the assignment does not depend on the order of the segmentation output. Unlike a one-to-one assignment, several non-nested observations may be assigned to the same track within one frame, so that an object divided into adjacent masks is not fragmented; these observations are integrated successively into the track and its local segments (Section 3.3.3). As a consequence, competition between observations does not prevent over-merging: an observation admissible for several tracks is assigned to the highest-scoring one irrespective of the other observations, and a new object admissible for an adjacent known track is absorbed into it. Protection against over-merging therefore rests entirely on the conservative acceptance condition of Equation 3.3.

3.3.3. Track Representation and Local Segments

Each track holds a persistent identity, a smoothed centroid and volume, an accumulated three-dimensional extent, and the image-space bounding box of its last observation. Centroid and volume are updated as exponential moving averages, for instance $\mathbf{c}_\tau \leftarrow \alpha\,\mathbf{c}_\tau + (1-\alpha)\,\mathbf{c}_o$ with $\alpha \in (0,1)$, which damps per-frame noise. The accumulated extent is the smallest axis-aligned box enclosing all associated observation boxes. It grows monotonically and thus approaches the full object over several partial views, but it never shrinks, so an erroneous association enlarges it permanently and facilitates further ones. The track centroid is deliberately not taken as the centre of this extent. Since an observation usually covers only part of an object, its centroid is the centre of the visible part, whereas the centre of the extent is the centre of everything observed so far. For a large object observed while the robot moves along it, the distance between the two grows with the distance travelled, so that a correct observation would fail the centroid test. Consecutive observations, in contrast, show largely the same part of the object, and the moving average follows them with a lag of approximately $\Delta c/(1-\alpha)$, where Δc denotes the displacement of the observed centroid between frames, independently of the size of the object. In addition, an outlier observation affects the moving average only temporarily, whereas it would enlarge the extent, and thus shift its centre, permanently. This compact state requires constant memory per track.

Extended structures such as floors, walls and ceilings cannot be represented by a single box, since its extent would cover a large part of the map and its centroid would lie far from most of the structure. Each track is therefore divided into local segments of bounded horizontal span. Every segment receives its own slot ID (Section 3.1) and thus its own object node in Hydra [7], while the track retains one identity and therefore one label for all its segments. An observation extends an open segment as long as the span limit is respected. A segment that reaches the limit is closed; subsequent observations either re-attach to it without enlarging its geometry or open a new segment. A single mask that already exceeds the limit is not split.

3.3.4. Transfer of Object Observations to Hydra

The tracking results of every frame are passed to Hydra [7] independently of the semantic labelling, so that the spatial representation is built up without delay. The masks are rendered into a semantic image and an instance image (Section 3.1.1), in which each pixel carries the slot ID of the assigned local segment, so that Hydra forms one object node per segment. Each mask is rendered in the filtered form from which its geometry was computed, and overlapping masks are drawn in order of decreasing area, so that every pixel carries a single slot ID and small objects remain visible in front of large structures. Pixels without a track or with depth outside the valid range receive a reserved background ID and invalid depth, and for real sensor data the slot ID of a new track can be withheld until it has been observed in several frames. All data are transmitted with the timestamp of the original observation, so that Hydra integrates geometry and object identity from the same instant.

3.4. Semantic Labelling and Risk-Relevant Object Information

Section 3.1 introduced the asynchronous, retrieval-first organisation of labelling and the principle of validating every result before it is stored. This section describes the two aspects that determine the quality of the semantic information within that organisation: how the view of an object is selected and presented to retrieval and to the VLM, and what the structured result contains beyond the label itself.

3.4.1. Selection and Presentation of the Object View

Since a track is labelled from a single image, the quality of that image largely determines the quality of the label. Each observation of a track is therefore assessed with three no-reference quality cues. The prominence P measures the number of object pixels and is normalised logarithmically, reflecting the diminishing gain in visual information as an object grows larger in the image. The sharpness S_{sharp} is estimated from the variance of the Laplacian response, a widely used focus measure [61]. The framing M expresses how little of the object mask touches the border of the crop, and thus penalises objects that are cut off by the image boundary. With all cues normalised to $[0, 1]$, the score of a candidate and the condition under which it replaces the stored crop are

$$S = w_P\, P + w_S\, S_{\text{sharp}} + w_M\, M, \quad w_P + w_S + w_M = 1, \qquad S_{\text{new}} > (1 + h)\, S_{\text{old}}, \tag{3.5}$$

where w_P, w_S and w_M weight the cues and h is a relative improvement margin.

The cues are combined additively rather than multiplicatively. In a product, a single weak cue suppresses the entire score, so that dissimilar candidates collapse to similarly low values; a sum preserves the contribution of the remaining cues and keeps candidates distinguishable. The margin h introduces hysteresis into the selection. Because view selection continues for as long as a track waits to be processed, consecutive frames supply many near-identical views whose scores

differ only by noise; requiring a clear improvement prevents their continual exchange while still admitting genuinely better views as the robot approaches an object.

Once a track is processed, the selected view is presented differently to retrieval and to the VLM. Retrieval receives the crop defined by the bounding box of the object, without further modification. The VLM receives a crop in which this bounding box is enlarged by a margin of surrounding context, so that the object can be interpreted in relation to its surroundings and parts of it that the segmentation mask has missed remain visible.

Within this enlarged crop, the target is marked by a boundary. The boundary is drawn along the outer contour of the segmentation mask rather than along the bounding box, and is rendered as a thin line in a conspicuous colour, while the pixels of the object itself remain unaltered. Since an enlarged crop frequently contains several objects, the boundary specifies which of them is to be named. The instruction given to the model refers explicitly to this boundary and states that everything outside it serves only as context, so that the model identifies the marked region rather than the most salient object in the crop.

3.4.2. Structured Output and Risk-Relevant Object Information

The structured result of the VLM comprises five fields: an open-vocabulary label, a label confidence, a mobility class, a mobility confidence, and a short description of the visual state of the object. The mobility class distinguishes three cases. An object is classified as dynamic only if it can change its position by its own means, which applies to humans, animals and self-propelled machines such as mobile robots. All other objects are classified as static, including the structural elements of a room, fixtures and furniture. The distinction thus refers to the capacity for self-propelled motion rather than to whether an object can be moved at all: an object that is occasionally relocated by a person remains static, since it does not change its position unless acted upon. The unknown class is used when the object itself could not be identified, so that no statement about its mobility can be made. An explicit no-result answer is provided for crops in which nothing can be identified, so that the model is not forced to name an object it cannot recognise.

The label states what an object is, whereas its risk relevance frequently depends on its current state. The visual-state description therefore records what is visibly particular about the instance at the time of observation, namely items placed on or with it, its condition, or whether it appears active or in use, such as water on a floor or a display that is switched on. It excludes identity, material and colour, which are covered by the label, and has an explicit value for the case that nothing notable is visible. Together with the mobility class, this description constitutes the risk-relevant object information that the scene graph provides to downstream risk assessment; the framework records these observations rather than computing a risk score itself. As open-ended text, it is not used to accept or reject a result. When a label is obtained by retrieval, however, the stored description belongs to the reference observation rather than to the current instance, so the description is most informative for objects labelled directly by the VLM.

Validation treats the fields asymmetrically. A result is rejected only on the basis of its label and label confidence, whereas a weak mobility decision is reduced to the unknown class without discarding an otherwise valid label, and a mobility class that contradicts an unambiguous label, for instance a person reported as static, is corrected for consistency. The instruction given to the model is organised as a structured decision procedure with priorities for ambiguous regions, such as surfaces carrying fixtures or transparent partitions, together with explicit guidance on calibrating the reported confidence. Its examples illustrate this procedure rather than enumerate permissible labels. The prompt variants considered during development are compared in Chapter **??**.

3.5. Scene Graph Fusion and Uncertainty Modelling

The semantic interpretation of objects (Sections 3.3 and 3.4) and the spatial scene graph maintained by Hydra [7] are produced independently and at different rates. This section describes how both are combined into a single risk-annotated scene graph (Section 3.5.1) and how the uncertainty associated with each object is represented within it (Section 3.5.2).

3.5.1. Scene Graph Fusion

The fusion stage combines the spatial scene graph of Hydra with the semantic results and observation history of the tracked objects. Its central design principle is that Hydra's graph is treated as the authoritative spatial structure and is never modified. Labels, relations and confidence values are added on top of this structure rather than written into it. They are not kept as a separate record that is updated step by step; instead, they are computed anew from Hydra's graph each time Hydra provides an update. As a consequence, structural changes made by Hydra, for example when object nodes are merged or corrected after loop closure (Section 3.6), are reflected automatically in the fused graph, and semantic information cannot drift out of alignment with the spatial map.

Semantic information is attached to Hydra's object nodes through their common slot. In the normal case, all nodes associated with a slot receive the same label, mobility class, descriptive object properties and associated confidence values. Where Hydra represents a single object by several overlapping nodes that share a slot, these nodes are consolidated into one node in the fused graph, and their edges are transferred to it, so that each physical object appears only once. Conflicting semantic results for the same slot, which can arise when Hydra merges or corrects nodes, are resolved by spatial proximity, so that each node receives the interpretation that was observed closest to it. Objects that have not yet been interpreted are represented explicitly as such, which allows the graph to distinguish between an object of unknown identity and the absence of an object.

Before relations between objects can be derived, it must be clear which graph nodes belong to the same physical object. A large structure, such as a long wall, may be represented by several local segments (Section 3.3), each of which appears as a separate node in the graph. If relations

were derived between these nodes directly, a single physical relationship, such as a table standing against the wall, would appear several times, once for each wall segment. To avoid this, nodes that share a common track identity are first grouped into a *main object*, and relations are always reasoned about at this level.

Two kinds of relations are derived between main objects. The *identity relation* connects the segments of one main object and states that they are parts of the same physical object. It follows directly from the tracking result and does not depend on geometry. The *contact relation* connects two different main objects and states that they touch or nearly touch. Contact is first decided for the two objects as a whole, and the resulting edges are then attached to those segments of the two objects that lie closest to each other. Since identity relations only link parts of the same object, while contact relations only link different objects, the two relation types never connect the same pair of nodes and therefore cannot contradict each other.

The spatial membership of objects and places is taken from Hydra wherever it is available and is completed only where it is missing. Associations between objects and places, and between places and rooms, are added only when the reconstructed geometry of the environment confirms that the two elements are mutually visible, so that no association is created through a wall or across floors. Where such geometric confirmation is not possible, no association is derived.

All derived relations therefore follow the same principle: they are added only when supported by geometric evidence in the shared three-dimensional frame. As a result, they describe properties of the scene itself and remain the same regardless of the viewpoint from which the robot observes it.

3.5.2. Uncertainty and Persistence Modelling

For each object, the framework represents three kinds of uncertainty separately: uncertainty about its semantic identity, about its mobility, and about its continued presence at the last observed position. These quantities answer different questions for downstream risk assessment and behave differently over time: identity and mobility are fixed once an object has been labelled, whereas the belief in its presence changes continuously.

Semantic identity and mobility are determined together, once per tracked object (Section 3.4). If a sufficiently similar object has been labelled before, its label, mobility class and object description are taken over through visual retrieval; otherwise, they are obtained directly from the VLM. Because tracks with conflicting labels are never merged (Section 3.3), each object retains a single label throughout its lifetime, and objects for which no usable label is obtained remain explicitly unclassified. The uncertainty of identity and mobility is represented by the confidence values attached to them, which are carried into the scene graph. For a retrieved label, this confidence is bounded both by the confidence with which the original object was labelled and by the similarity between the two objects. A label transferred from a weakly matching or uncertainly labelled object is therefore marked as correspondingly uncertain, which allows downstream risk assessment to distinguish confidently identified objects from those whose identity is doubtful.

Presence is represented by a confidence $p \in [0, 1]$, which expresses the belief that an object is still located where it was last observed. It is derived from the time of the most recent observation of the object, is equal to one while the object is being observed, and otherwise decays over time at a rate that depends on its mobility class m,

$$p(t) = 0.5^{(t-t_{\text{last}})/\tau_m}, \qquad \tau_{\text{dynamic}} \ll \tau_{\text{static}}, \tag{3.6}$$

where t_{last} is the time of the most recent observation and τ_m the half-life associated with the mobility class. The decay rates can be adapted to the dynamics of a given environment. How the mobility class shapes this decay, and what follows from it, is described in Section 3.5.3.

By separating what an object is, whether it moves, and whether it is still present, the representation enables downstream risk assessment to distinguish well-established objects from uncertain ones. These quantities are, however, confidence values and decay heuristics rather than calibrated probabilities. They inherit the uncalibrated nature of VLM-reported confidence (Section 2.9.2) and should therefore be interpreted as relative measures of reliability.

3.5.3. Treatment of Static and Dynamic Objects

The mobility class determines how an object is represented and maintained over time. An object is classified as *dynamic* only if it can move by itself, as humans, animals and mobile robots do. All other objects, including furniture that is occasionally moved by others, such as chairs, are classified as *static*. If the mobility of an object cannot be determined with sufficient confidence, it is classified as *unknown*. To reduce misclassifications, the mobility class proposed by the VLM is checked against the object category: clear labels of humans, animals or mobile robots enforce the dynamic class, whereas clear labels of structures and furniture enforce the static class (Section 3.4).

The two classes reflect two different assumptions about the environment. For a static object, the default assumption is that it remains where it was observed, even when it has not been seen for some time. For a dynamic object, the default assumption is the opposite: once it is no longer observed, it has most likely moved elsewhere. This difference affects three aspects of the representation, which are summarised in Table 3.2.

Table 3.2.: Differences in the treatment of static and dynamic objects.

Aspect	**Static (and unknown)**	**Dynamic**
Typical objects	Walls, floors, furniture, fixtures	Humans, animals, mobile robots
Spatial representation	Node created by Hydra from the accumulated reconstruction	Most recent observed position; added to the fused graph if Hydra has no node
Presence decay	Slow	Fast
When no longer observed	Retained with decreasing presence confidence	Withdrawn once presence has become unlikely

The first aspect is the spatial representation. Hydra creates object nodes from surfaces that are reconstructed consistently over time [7]. This works well for static objects, whose surfaces accumulate at the same location with every observation. A moving object, by contrast, does not remain in place long enough for its surface to accumulate, so that Hydra may never create a node for it, even though it has been reliably identified by the perception pipeline. Such dynamic objects are therefore added to the fused graph by the fusion stage itself, using only their most recent observation. The extent observed over the whole track is deliberately not used, since for a moving object it would cover the entire path travelled rather than the space the object currently occupies. Without this step, the objects that are most relevant for risk assessment would be missing from the scene graph precisely because they move.

The second aspect is the decay of the presence confidence (Equation 3.6). Because a dynamic object is expected to change its position, its presence confidence decays within a short time after it was last observed. The presence confidence of a static object decays slowly, so that furniture and structures remain credible long after leaving the field of view. Objects of unknown mobility are treated as static, since in the absence of evidence that an object moves, assuming that it has stayed in place is the safer choice.

The third aspect concerns what happens to an object that is no longer observed. A dynamic object is withdrawn from both the tracking state and the fused graph once its presence confidence has fallen below a threshold, since it has then most likely left its last observed position. This prevents outdated positions of people or robots from suggesting hazards where none exist. Static objects and objects of unknown mobility are never withdrawn. They remain in the scene graph with a correspondingly low presence confidence, so that downstream risk assessment can still take them into account while recognising that their position has not been confirmed recently. In no case is an object removed from Hydra's map. This asymmetry is deliberate: for risk assessment, wrongly removing a static obstacle that is still present is more harmful than retaining an object whose presence is uncertain.

3.6. Loop Closure and Multi-Session Consistency

The tracks of Phase 1 cache object geometry in the map frame and must remain consistent with Hydra's map whenever that map changes, whether through a drift correction within a session or through a restart between sessions. Otherwise, a re-observed object no longer matches its own track and creates a duplicate instead of being re-identified.

3.6.1. Consistency under Loop Closure

When the localisation back-end corrects accumulated drift, Hydra deforms its scene graph accordingly (Section **??**). The cache of the tracker is not part of this optimisation and would remain in the uncorrected frame. Re-observed objects would then fail the association tests of Section 3.3.2 and start new tracks, so that Phase 1 diverges from Hydra.

A correction becomes visible as a change in the transform $\mathbf{T}_{\mathrm{MO}}$ between the map frame and the odometry frame, and changes below a small threshold are ignored as jitter. This change, however, describes the correction at the current pose only. Since drift accumulates along the trajectory, geometry observed shortly before the closure carries almost the full error, whereas geometry observed at the beginning of the loop carries almost none. Applied rigidly to the entire cache, the change would be exact for the most recent objects but would displace the earliest ones, which are precisely those re-observed when the loop is closed. Hydra's optimisation, in contrast, distributes the correction along the trajectory and moves every object node individually.

The tracker therefore adopts Hydra's correction instead of estimating its own. Since every local segment carries its own slot ID and thus corresponds to exactly one object node (Section 3.3.3), the correction of a segment can be read directly from the movement of its node. Only segments for which Hydra has not yet created a node, typically those observed shortly before the closure, require a different treatment. Every point $\mathbf{x}$ of a segment s, that is, its centroid and the corners of its bounding box, is therefore moved to

$$\mathbf{x}' = \begin{cases} \mathbf{x} + (\mathbf{p}'_s - \mathbf{p}_s) & \text{if Hydra has a node for } s, \\ \mathbf{T}'_{\mathrm{MO}}\, \mathbf{T}_{\mathrm{MO}}^{-1}\, \mathbf{x} & \text{otherwise,} \end{cases} \tag{3.7}$$

where a prime denotes a quantity after the correction. The first case applies to segments that Hydra has already integrated into its graph. Here, $\mathbf{p}_s$ and $\mathbf{p}'_s$ are the positions of the node of s in the last graph before and the first graph after the correction, so that their difference is exactly the displacement that Hydra's optimisation has assigned to this object. Since both positions are expressed in the map frame, this displacement already contains the complete correction of the object, and the change in $\mathbf{T}_{\mathrm{MO}}$ is not applied in addition. Adding it to $\mathbf{x}$ moves the cached segment by the same amount as its node, and the tracker therefore inherits Hydra's non-rigid correction segment by segment without estimating any correction of its own.

The second case applies to segments without a node and is read from right to left. A cached point was originally obtained in the odometry frame and converted into the map frame with the transform valid at that time, $\mathbf{x} = \mathbf{T}_{\mathrm{MO}}\, \mathbf{x}_{\mathrm{O}}$, where $\mathbf{x}_{\mathrm{O}}$ denotes its odometry coordinates. A loop closure changes only this transform, not the odometry measurement itself. Multiplying by $\mathbf{T}_{\mathrm{MO}}^{-1}$ therefore recovers $\mathbf{x}_{\mathrm{O}}$, and multiplying by $\mathbf{T}'_{\mathrm{MO}}$ converts it into the map frame with the corrected transform, as if the correction had already been known when the point was observed. Both transforms are applied in homogeneous coordinates. Because $\mathbf{T}'_{\mathrm{MO}}$ compensates the drift accumulated up to the current pose, this case is only valid for geometry that carries the same drift, which holds for the recently observed segments to which it is applied.

Tracks that were nevertheless created twice around the closure are folded into their earlier identity by a subsequent merging step. A recently created track is merged into an older compatible track, that is, one without a conflicting label, if it lies within a search radius that scales with the magnitude of the correction, and pairs whose extents overlap strongly are merged as well. The older identity, together with its slot and label, survives. This is the later repair of fragmentation anticipated in Section 3.3.

3.6.2. Persistence across Sessions

A robot that is switched off and restarted should neither forget what it has mapped nor build a second copy of the environment. This work considers a restart in place: the last known pose of the robot is saved together with the map and used as the initial pose of the new session, which therefore continues in the same map frame.

The three components that hold state save it independently. The tracker saves its tracks, including geometry, local segments with their slot IDs, labels and mobility classes, together with its slot and identity counters, so that no identifier is issued twice. Hydra saves its scene graph and mesh, and the retrieval memory retains its references (Section 3.4). The fuser holds no state, since its output is recomputed from the other components (Section 3.5.1). Restored tracks take part in the association like any other track, and their labels are reused without querying retrieval or the VLM again.

Since Hydra can save its map but not resume from it, three changes were required. The saved scene graph and mesh are loaded as the initial map before new data are integrated. All restored nodes are marked as archived, that is, outside the active mapping window, because Hydra only considers archived nodes as merge candidates for new objects. Finally, new node identifiers continue after the highest restored one and new geometry is appended to the restored mesh, so that new and restored elements never collide. The previous trajectory is not restored.

Restored and newly mapped nodes are reconciled by Hydra's object merging, which joins spatially consistent nodes with the same label. Merging by label is normally risky, since many instances share a class, but is safe here: owing to the slot design (Section 3.1.2), labels are per-object slot IDs, so two distinct objects never share one.

4. Implementation

This chapter describes how the methodology of Chapter **??** was realised. Section 4.1 describes the data basis, from the planned data-collection campaign to the substitute datasets, Section 4.2 describes the software implementation, and Section **??** the measures taken to achieve near-real-time operation on embedded hardware.

4.1. Data Collection and Substitute Datasets

4.1.1. Planned Data-Collection Campaign

The publicly available datasets considered in this work were recorded with wheeled robots, handheld sensors or in simulation. To evaluate the framework on the type of platform for which it is intended, a dedicated real-world dataset was planned, recorded with a legged robot inside a university building.

An Intel RealSense D455 RGB-D camera [62] was mounted on a Unitree Go1 quadruped robot [63] with a custom 3D-printed base, and the transform between the robot base and the camera was defined manually as the fixed extrinsic $\mathbf{T}_{\mathrm{BC}}$ of Section 3.2. The camera was connected via USB to a Raspberry Pi onboard the robot, which ran the RealSense driver under ROS 1 Melodic, in line with the ROS 1-based software stack of the Go1. A separate onboard computer subscribed to the colour, depth and calibration messages over the internal network of the robot and recorded them at 640×480 pixels, together with the depth-to-colour extrinsics and the robot odometry, as ROS 1 bags. Depth was recorded in its native sensor geometry rather than aligned to the colour image, which kept the processing load on the Raspberry Pi low, while the recorded extrinsics retained the information required for a later alignment. Since the onboard storage was insufficient for RGB-D recordings, the bags were to be written to an external USB storage device.

The bags were then to be converted into the ROS 2 format and replayed offline through the ROS 2 pipeline [10] on the Jetson platform. A live deployment would require either a native ROS 2 interface on the robot or a ROS 1–ROS 2 bridge. The campaign was carried out but ultimately discarded for the reasons described in Section 4.1.2.

4.1.2. Limitations Encountered

Table 4.1.: Limitations encountered during the Go1 data-collection campaign.

No.	Limitation	Cause	Consequence
1	Onboard compute	No power supply or interface for the Jetson AGX Thor on the Go1	Offline processing only
2	USB bandwidth	USB 2.0 only; D455 at 5, 15 or 30 fps (640×480)	Frame loss; approx. 11.7 Hz colour and 10.7 Hz depth at 15 fps, varying between runs
3	Vibration	Gait vibrations amplified by the 3D-printed mount	High depth noise; camera transform not rigid
4	Reflections	Reflective floors and surfaces	Critical errors in active stereo depth

The limitations summarised in Table 4.1 fall into three categories. Limitation 1 concerns platform integration; it restricts the campaign to offline evaluation but does not prevent it. Limitation 2 concerns data recording and prevented the complete, temporally consistent streams that the frame preparation of Section 3.2 requires. Limitations 3 and 4 concern the quality of the depth measurements.

The depth-quality limitations were decisive, since the framework depends on metric depth at every stage of the per-frame path. Depth is used to back-project segmentation masks into 3D observations (Section 3.3.1), it provides the 3D evidence on which data association is based (Section 3.3.2), and it is integrated by Hydra into the volumetric reconstruction from which the mesh, the object nodes and the places are derived [7]. In addition, back-projection relies on the rigid camera transform of Equation (3.1), which the vibrations of limitation 3 invalidate. Noisy depth therefore propagates into the object geometry, the tracking and the map itself. Together with the incomplete recordings, this made the Go1 data unsuitable as the basis of the evaluation.

4.1.3. Substitute Datasets

The substitute datasets were selected according to the input contract of Section 3.2: registered colour and depth images, the camera calibration, odometry and a known base-to-camera extrinsic, recorded in indoor environments as replayable ROS bags.

The office sequences of the OpenLORIS-Scene dataset [12] serve as the real-world data and are the closest substitute for the planned recordings. They were captured with a RealSense D400-series camera, the sensor family of the D455, and provide depth aligned to the colour image, real wheel odometry and a calibrated base-to-camera transform. Since they were recorded on a wheeled robot with complete sensor streams, they avoid the recording and vibration problems of Section 4.1.2, so that the method can be evaluated on real sensor data without the results being dominated by these hardware-specific effects.

The simulated office and apartment scenes of the uHumans2 dataset [11], both with ground-truth odometry and metric depth, complement the real-world data. The office scene was used for development, optimisation and repeatable experiments, while the apartment scene, which contains a human, tests the portability of the framework to a second environment and its handling of a dynamic object. Switching between the datasets requires only configuration (Section 4.2.6). As a consequence of the substitution, effects of legged locomotion, such as gait-induced camera motion and depth noise on reflective floors, are not covered by the evaluation, and the uHumans2 experiments rely on ground-truth odometry (Chapter **??**).

4.2. Software Implementation

This section describes how the framework of Chapter **??** is realised in software. It follows the components in the order of the data flow, namely the preprocessor, the Phase 1 semantic coordinator with its tracker and labelling lifecycle, Hydra, and the scene graph fuser, and concentrates on their class structure, their concurrency, and the configuration values of the evaluated runs. The design rationale is given in Chapter **??** and is not repeated here.

4.2.1. Package Structure and Deployment

The framework is implemented as a single ROS 2 package, `rsg` [10], and runs as a chain of four components, shown together with their classes in Figure 4.1. The preprocessor turns the raw sensor streams into prepared frames and publishes them as the custom message `RsgFrame`. The Phase 1 semantic coordinator segments and tracks the objects in each frame and passes slot-ID images to Hydra [7] on its standard image topics. Hydra builds the scene graph and publishes it as `hydra_msgs/DsgUpdate` to the scene graph fuser, which also receives the label events, heartbeats, and deletion events of the asynchronous semantic path as JSON over `std_msgs/String`. The preprocessor and the coordinator are written in Python; the fuser is written in C++, since it traverses Hydra's complete scene graph through the `spark_dsg` library. Hydra and the llama.cpp server [64] that hosts the VLM run as separate processes.

Within each node, the ROS 2 node class handles only subscriptions, publications, and threads, and delegates the processing to separate classes. In the coordinator, this takes the form of a stage-based architecture: each step of the Phase 1 pipeline is a stage class in its own module, for example `Phase1SegmentationStage`, `Phase1VlmDispatchStage`, or `Phase1PublishingStage`. The coordinator composes the stages at start-up and keeps only the state they share, namely the tracker and the queues between its threads. Models are reached through backend interfaces selected by factory functions, so that a stage or a model can be exchanged without affecting the rest of the pipeline (Section 4.2.6).

All settings that differ between datasets or models are kept in configuration files rather than in code. A run is defined by a pipeline file, whose `preprocessing` and `phase1` sections are read by the preprocessor and the coordinator, a fuser file, and Hydra's input file, dataset file, and slot-only

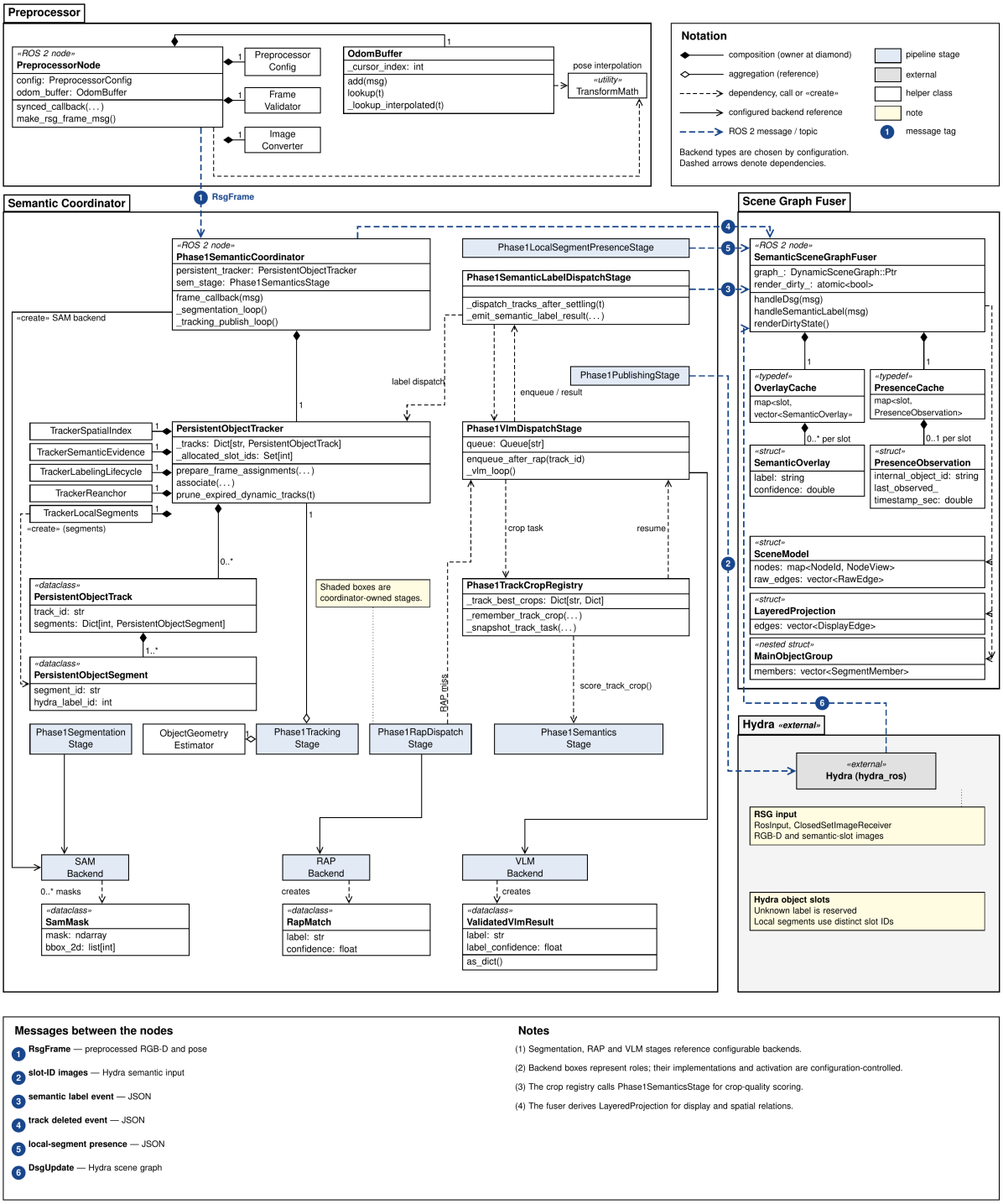


Figure 4.1.: Class overview of the implemented framework, showing the main ROS 2 nodes, internal processing stages, supporting classes, and configurable backends.

label space. The launch file `rsg_all.launch.py` selects these files through launch arguments, with one set each for uHumans2 and OpenLORIS. Within the pipeline file, the VLM is chosen by a profile: `phase1.vlm.profiles` defines one entry per model, with its endpoint, model file, server arguments, and memory requirement, and `active_profile` selects the entry used by both the coordinator and the script that starts the llama.cpp server.

4.2.2. Preprocessor

`PreprocessorNode` holds a `PreprocessorConfig` and delegates temporal association to `OdomBuffer`, the validity checks to `FrameValidator`, image handling to `ImageConverter`, and rigid-body arithmetic to the stateless `TransformMath` (Figure 4.1). Figure 4.2 shows the processing of one frame. Odometry messages are appended to `OdomBuffer` as they arrive, and the latest calibration is cached outside the image synchroniser. RGB and depth images are paired by a `message_filters` approximate-time synchroniser. A synchronised pair is rejected if no calibration is available or if the resolutions of RGB and depth differ, and it is skipped if it would exceed the configured maximum output rate.

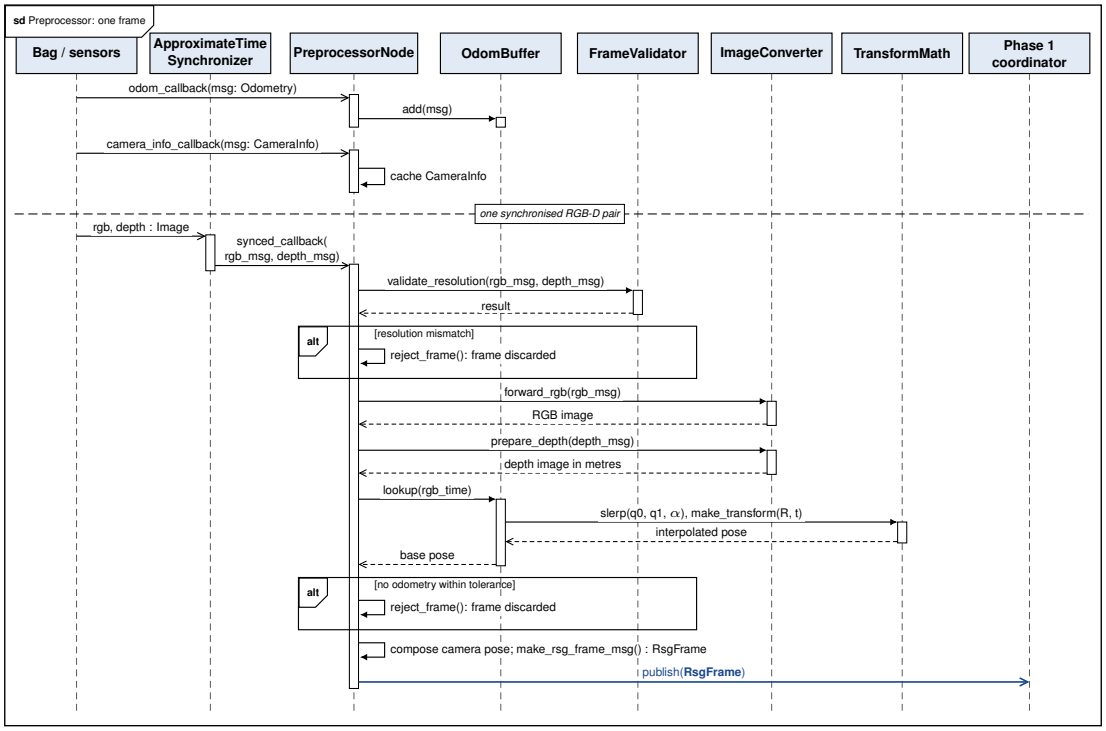


Figure 4.2.: Sequence of one frame through the preprocessor.

Images that already satisfy the output contract (`rgb8`, and `32FC1` in metres) are forwarded by reference; depth recorded in millimetres is converted to metres. Since frames arrive in temporal order, `OdomBuffer` advances a cursor instead of searching the buffer. It interpolates the base pose

linearly in translation and by spherical linear interpolation in rotation, falls back to the nearest sample if no enclosing pair exists, and rejects the frame if no sample lies within the configured time tolerance. The camera pose is obtained by composing the interpolated base pose with the fixed transform $\mathbf{T}_{BC}$, after which the node assembles and publishes one `RsgFrame`. For datasets whose bags contain no dynamic transform stream, the node also broadcasts the camera transform. For replay, QoS override files republish `/tf_static` and the latched calibration as `TRANSIENT_LOCAL`, so that late subscribers such as Hydra still receive them.

4.2.3. Phase 1 Coordinator

`Phase1SemanticCoordinator` loads its configuration, creates the model backends through the factory functions `make_sam_backend`, `make_rap_backend`, and `make_vlm_backend`, and composes one object per pipeline stage (Figure 4.1). Stages and backends exchange data only through the data classes `SamMask`, `RapMatch`, and `ValidatedVlmResult`. The coordinator is organised as a pipeline of threads connected by bounded queues, whose interaction is shown in Figure 4.3. Five threads are active: the ROS executor, a segmentation thread, a tracking/publish thread, a RAP worker, and a VLM worker.

The per-frame path is split into two stages that run on separate threads and form a two-stage pipeline: a segmentation stage and a tracking/publish stage. While the segmentation thread processes frame $n+1$, the tracking/publish thread processes frame n. The split follows the nature of the work: segmentation is GPU-bound and releases the global interpreter lock while it waits for inference, whereas tracking and publication are CPU-bound, so that the two stages overlap in time and the frame rate is limited by the slower stage rather than by the sum of both.

The executor callback wraps each `RsgFrame` and places it in `frame_fifo`, from which the segmentation thread takes it, converts the images and calls the segmentation backend through `Phase1SegmentationStage`, which downscales the image, runs the segmenter, and restores the masks to full resolution. The result is passed through `sam_output_fifo` to the tracking/publish thread. Both queues deliberately hold a single entry and drop the oldest one when full: if the receiving thread is still busy, the waiting frame is replaced by the newer one, so that only the most recent frame is processed. Larger queues would avoid these drops, but every frame they buffer would delay the output passed to Hydra, so that the latency of the map would grow whenever processing falls behind the frame rate and near-real-time operation would be lost. The tracking/publish thread first computes a 3D observation for every mask with `ObjectGeometryEstimator`, which back-projects a subsample of the pixels of the largest connected region within the valid depth range and discards masks with too few valid points. It then assigns all observations of the frame jointly with `PersistentObjectTracker` and offers each crop to `Phase1TrackCropRegistry`, which keeps the best crop of each track according to Equation 3.5.

Before the frame is published to Hydra, the same thread performs the bookkeeping of the asynchronous path. `Phase1LocalSegmentPresenceStage` publishes the heartbeat of the slots observed in the frame, `Phase1SemanticLabelDispatchStage` schedules tracks whose settling period has elapsed, weak crops whose waiting time has expired are released to the VLM, and

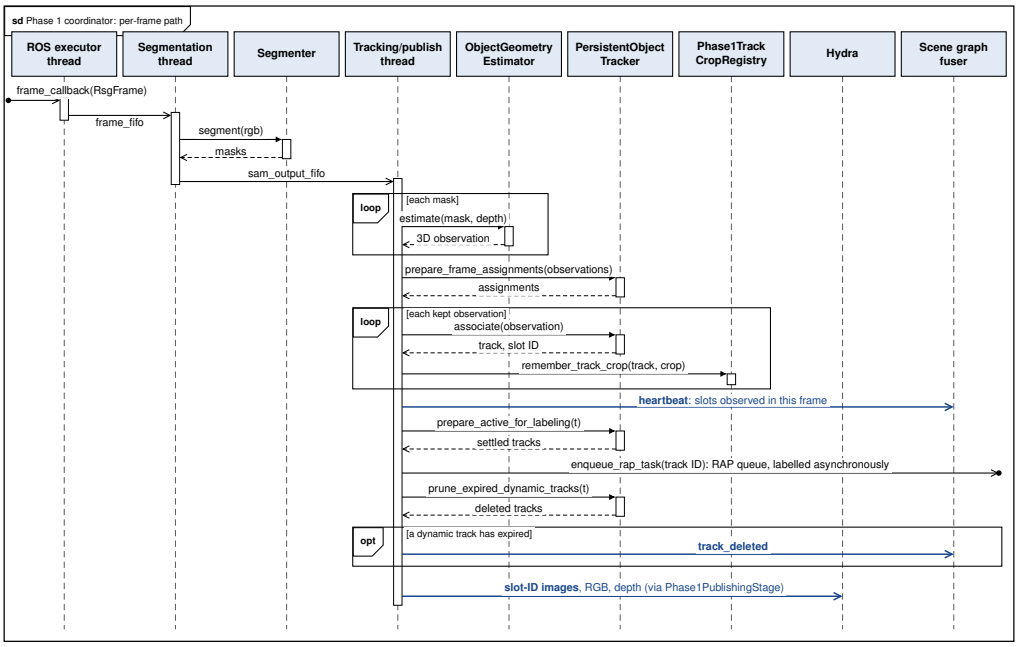
confirmed dynamic tracks whose presence confidence has decayed are deleted, which is announced to the fuser as a `track_deleted` event on the label topic. Finally, `Phase1PublishingStage` renders the masks into `16UC1` semantic and instance images of slot identifiers in decreasing order of mask area, sets depth and label to zero outside the valid depth range, and publishes RGB, depth, label images, and calibration to Hydra with the original timestamp. The tracker lock and a re-entrant lock on the crop registry guard the state shared with the RAP and VLM workers. Python threads suffice, since TensorRT inference and blocking HTTP requests release the global interpreter lock.

The track state of Section 3.3.3 is held in `PersistentObjectTrack` objects, each of which owns one or more `PersistentObjectSegment` objects whose field `hydra_label_id` stores the slot ID. The joint assignment of Equation 3.4 is implemented in `prepare_frame_assignments`, which selects the track for each observation with `_greedy_maximize`. For persistence across sessions (Section 3.6.2), the module `tracker_state_store` writes the tracker state atomically to a versioned JSON file; on restoration, the timestamps are shifted into the past, so that restored tracks are matched in revisit mode.

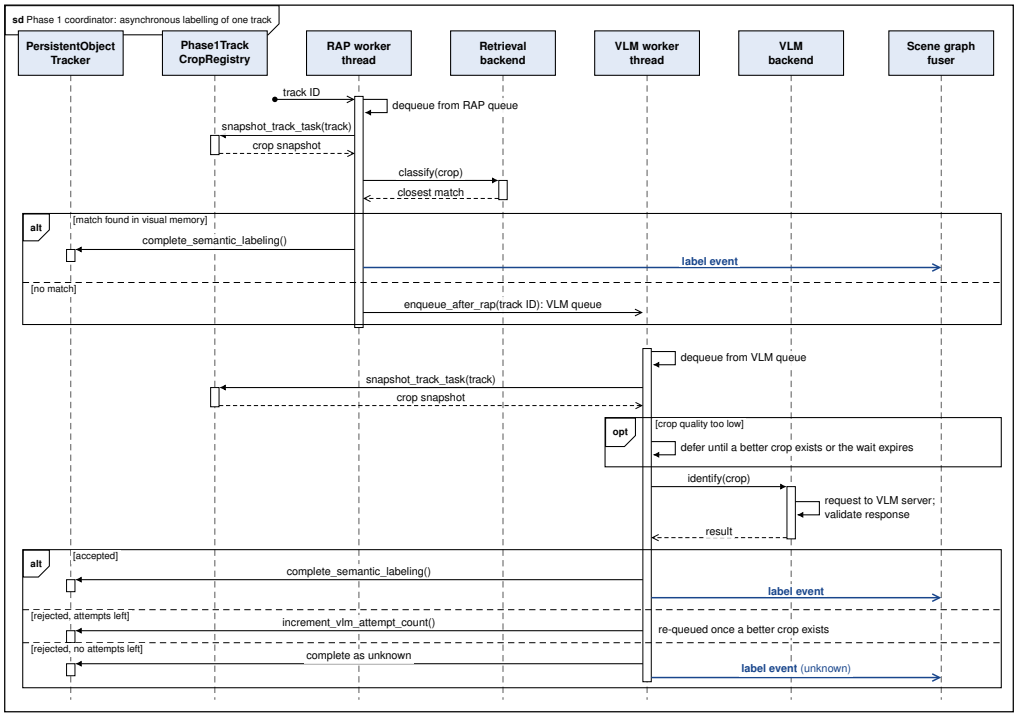
The labelling scheme of Section 3.1 is realised by `Phase1SemanticLabelDispatchStage` together with `Phase1RapDispatchStage` and `Phase1VlmDispatchStage` (Figure 4.3b), and its progress is recorded in the `labeling_status` of each track. When a worker queue is full, identifiers are deferred rather than dropped, so that no track loses its labelling request. Before a VLM request, the crop additionally passes a quality gate based on size, border contact, and crop score; a track that fails it waits in `vlm_waiting_for_better_crop` until a better crop is registered or a maximum waiting time has passed. Validation is implemented in `validate_vlm_response`, and an accepted result is published as one label event per slot of the track.

4.2.4. Hydra Integration and Session Persistence

Hydra is configured with the frozen label space `rsg_slot_only_frozen_label_space.yaml`, which contains the background label and the slots 1 to 10 000, all declared as object labels, with a dataset configuration that holds the map parameters, and with an input configuration that connects Hydra's image receiver to the Phase 1 topics. Figure 4.4 shows the resulting sequence. Since every slot is an object label, Hydra clusters one object per slot, so that object identity is decided by Phase 1 rather than by Hydra's class labels. The modifications of Section 3.1.1 are implemented as follows. Both the receiver queue and the input queue of the active window are bounded to five entries and filled without blocking, so that frames are dropped when reconstruction falls behind instead of accumulating a backlog. Since segmentation often detects an object only after it has been seen in several frames, its voxels first receive evidence for the background label. Hydra's maximum-likelihood semantic integrator accumulates this evidence without decay, so that such a head start could prevent the object's slot label from ever prevailing and the object would never form a node. The integrator was therefore modified to ignore the background label: a voxel that has only been observed as background remains undetermined, while competition between object labels is unaffected.



(a) Synchronous per-frame path.



(b) Asynchronous labelling of one track by the RAP and VLM workers.

Figure 4.3.: Sequence of the Phase 1 coordinator.

For resumption, the saved scene graph and mesh are loaded into the back-end before integration, restored nodes are archived so that re-observations are merged into them, node identifiers continue after the restored ones, and back-end object merging is enabled. At shutdown, the back-end writes the scene graph, the mesh, and the deformation graph, and the active window writes its volumetric map. Session state is kept in three stores under `memory/` (tracker JSON, Hydra back-end files, Chroma directory), which are controlled by one switch and cleared by `clear_memory.py`; in the evaluated configuration, this switch is disabled, so that every run starts from an empty map. For loop closure, the coordinator latches the `map`-to-`odom` transform from `/tf`, ignores changes below configured translation and rotation thresholds, and otherwise calls `reanchor_all` and `merge_reanchor_duplicates`; with ground-truth odometry, this hook remains inert.

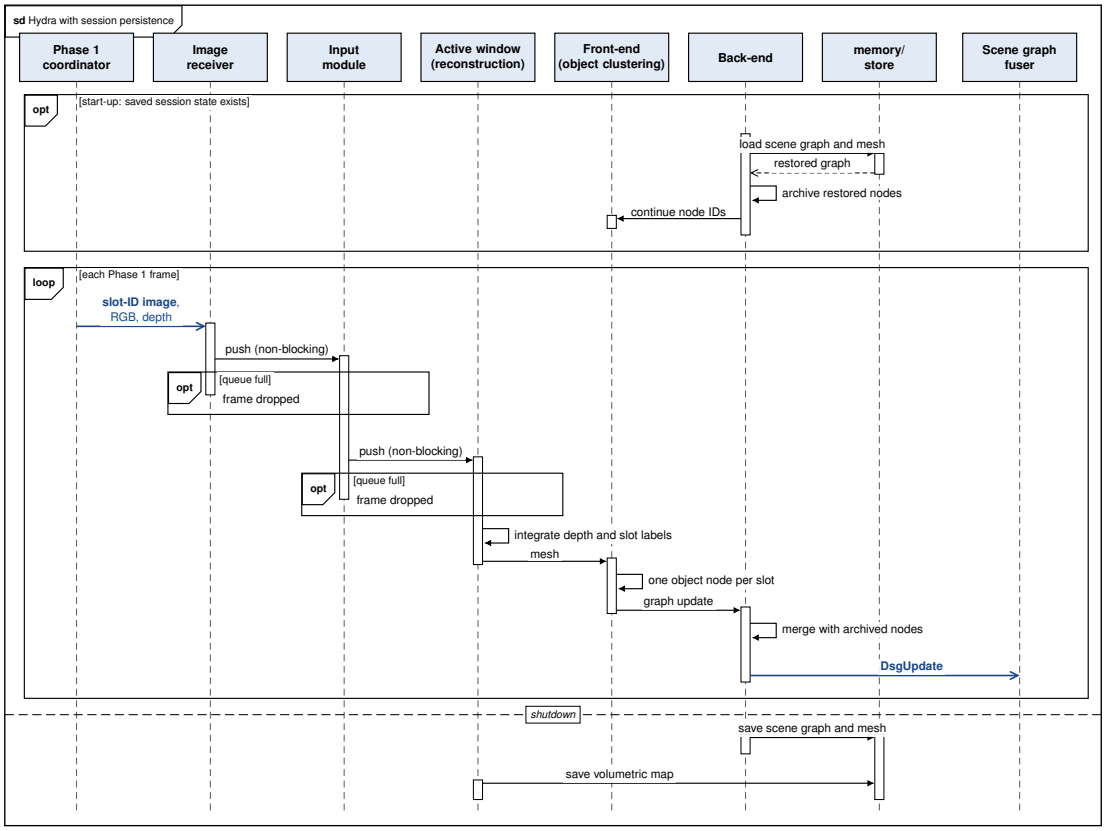


Figure 4.4.: Sequence of the modified Hydra pipeline with session persistence.

4.2.5. Scene Graph Fuser

The scene graph fuser, implemented as the class `SemanticSceneGraphFuser`, joins Hydra's geometry and the semantic events of Phase 1 by slot ID. Since the two streams arrive independently, for instance a label before its object node exists, the fuser does not combine them on arrival but stores them in shared state: a local copy of Hydra's scene graph, the overlay and presence

caches of each slot, the set of deleted slots, and the flag `render_dirty`, all of which are protected against simultaneous access. Three concurrent callback groups interact only through this state (Figure 4.5). The graph callback group applies Hydra's `DsgUpdate` messages to the local graph, either replacing it or applying an incremental update with its deletions. The semantic callback group stores label events and `track_deleted` events in the overlay cache and the set of deleted slots, and heartbeats in the presence cache. Both groups merely store the data and set the flag, so that ingestion is never blocked by rendering.

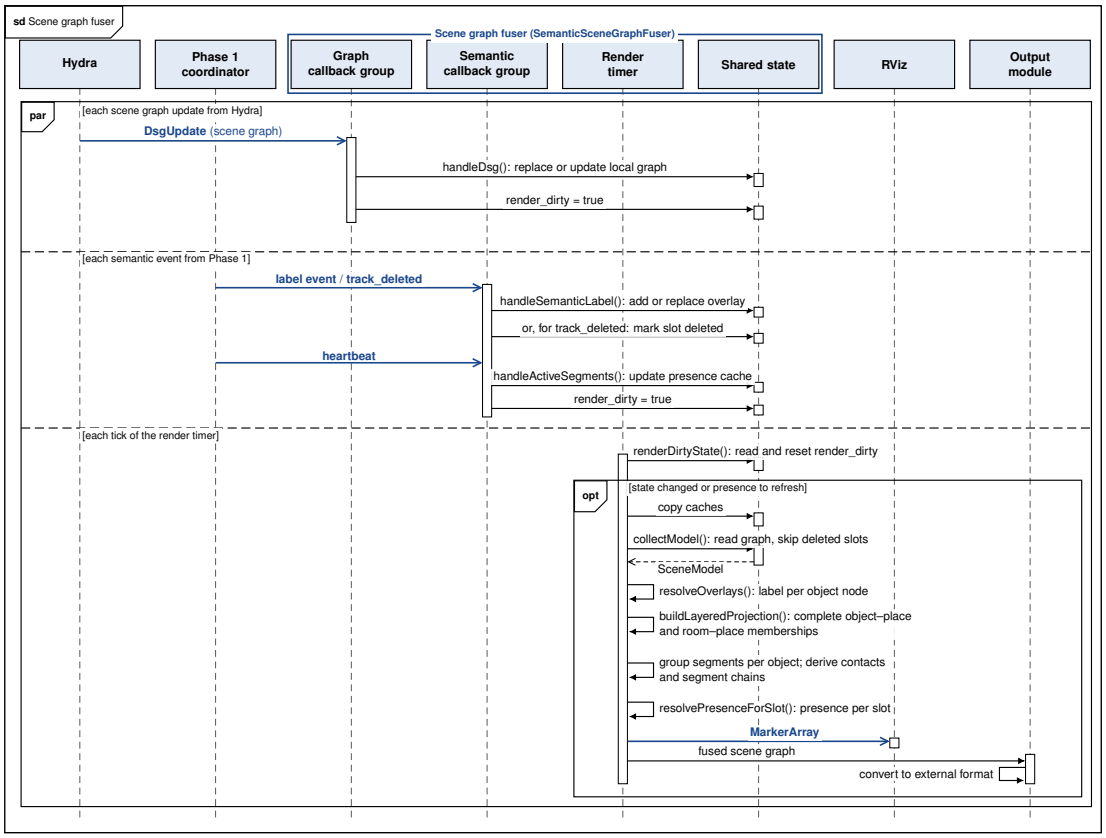


Figure 4.5.: Sequence of the scene graph fuser.

The third group, a periodic timer, renders only if the flag is set or presence confidences have to be refreshed, so that several messages within one period cause a single render. It first copies the caches, so that it works on a consistent snapshot while new messages continue to arrive. From this snapshot and the local graph, it derives the scene model without the deleted slots and assigns a label to each object node, so that a label that arrived early is applied as soon as its node exists. It then completes missing object–place memberships, validated against the mesh, and room–place memberships, groups the segments of each physical object to derive contacts between objects and the chains between their segments, and resolves the presence of each slot according to Equation 3.6. The fused scene graph is published as an RViz marker array and passed to an output module, which converts it into a format for external consumers such as a downstream risk assessment.

4.2.6. Portability and Extensibility

The framework separates the models and the dataset from the code, so that each of them can be exchanged through the configuration files described in Section 4.2.1. The pipeline accesses every model only through a small interface, `VlmBackend`, `SamBackend`, or `RapBackend` (Figure 4.1), and a factory function creates the implementation named in the pipeline configuration file at start-up. The following paragraphs describe the steps required to use a new VLM, a new segmentation model, and a new dataset.

New VLM. A VLM that is served through an OpenAI-compatible chat-completions endpoint, as provided by llama.cpp [64], requires no change to the code. A new entry is added under `phase1.vlm.profiles` in the pipeline configuration file, specifying the endpoint, the model name, the model file and its vision projector, the server arguments, and the minimum system memory. Setting `phase1.vlm.active_profile` to this entry selects the model: at start-up, the launcher `rsg_vlm_server` checks the available memory and starts the server with the model, and the coordinator sends its requests to the same endpoint. The prompt and the validation thresholds are defined independently of the profiles and therefore remain unchanged; the new model only has to return the same structured output, and its labelling accuracy should be re-validated with the existing prompt. Only a VLM with a different interface requires a new implementation of `VlmBackend` and an entry in `make_vlm_backend`.

New segmentation model. The segmentation model is selected by `phase1.sam.backend`, and the files of each backend are specified in its own configuration block. A model that is compatible with an existing backend, such as another SAM checkpoint for `RealSamBackend` or other TensorRT engines for `NanoSamBackend`, is therefore used by changing these file paths. A model with a different interface requires a new class that implements `segment(rgb)` and returns the masks as `SamMask` objects, an entry in `make_sam_backend` under a new backend name, and a configuration block for its files. Since all later stages consume only `SamMask` objects, no other part of the pipeline is affected; `NanoSamBackend` was added alongside `RealSamBackend` in exactly this way.

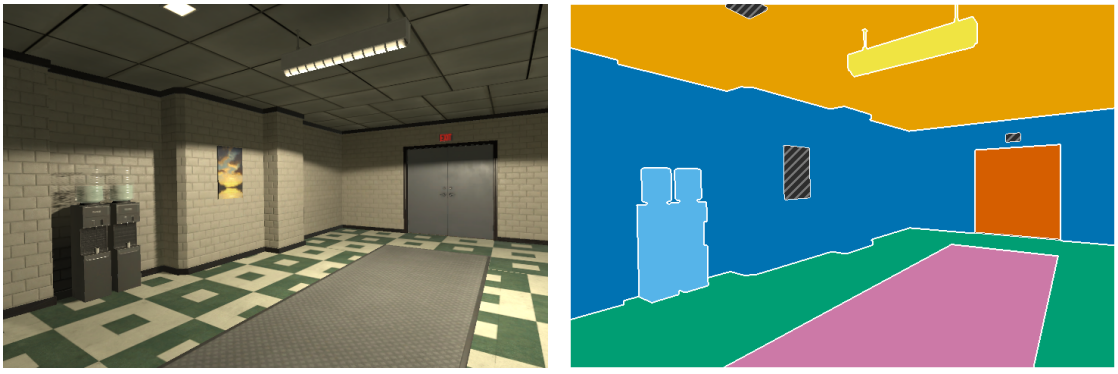
New dataset. A new dataset or robot requires only a new set of configuration files and no change to the code. The pipeline configuration file specifies the input topics, the coordinate frames, the fixed transform between the robot base and the camera, and the depth scale of the sensor; the fuser configuration file specifies the parameters of the fused graph; Hydra's input configuration connects its image receiver to the Phase 1 topics and sets the sensor range; and Hydra's dataset configuration contains the map parameters, which have to be tuned to the new sensor. If the data are replayed from bags, a QoS override file republishes static transforms and calibration so that late subscribers still receive them. The new files are passed to `rsg_all.launch.py` through the launch arguments `pipeline_config`, `fuser_config`, `input_config`, and `dataset`. OpenLORIS was added to the framework in this way, without any change to the code used for uHumans2 (Section 4.1.3).

5. Results

This chapter reports the experimental results in the order of the processing pipeline. Section 5.1 examines the segmentation backend and its configuration, Section 5.2 object tracking and crop selection, Section 5.3 the comparison of vision-language models and prompts, Section 5.4 the resulting scene graphs across environments, and Section 5.5 the runtime performance of the complete pipeline. Since the experiments differ in data, configuration and purpose, each section first states its own experimental setup.

5.1. Segmentation Backend and Configuration

The segmentation models for object proposals (Section 3.3.1) were evaluated offline on stored frames, outside the running pipeline, so that their masks could be compared with ground truth and their latency measured in isolation. The benchmark consists of 500 frames of the uHumans2 office scene [11] at 720×480 pixels, sampled about every 0.75 s. The simulator provides a segmentation image that labels each pixel with a class but does not distinguish objects of the same class. Each connected region of one class was therefore taken as one ground-truth object; as a consequence, touching objects of the same class, such as the two water dispensers in Figure 5.1, form a single object. Regions smaller than 3500 pixels were ignored in the evaluation. This gives 5231 objects, on average 10.5 per frame (Figure 5.1).



(a) RGB image. (b) Ground-truth objects.

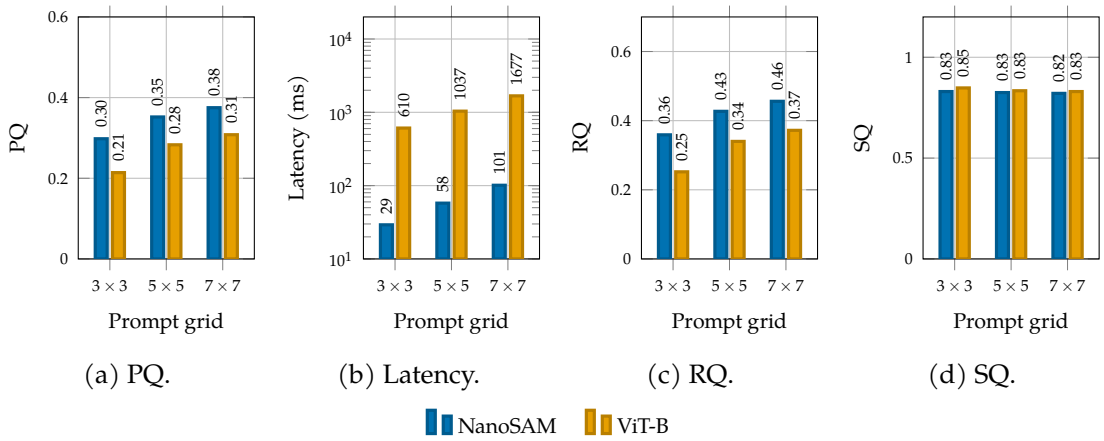
Figure 5.1.: Example benchmark frame. Each colour is one ground-truth object; hatched regions are below 3500 pixels and ignored.

The masks were scored with the panoptic quality (PQ) of Kirillov et al. [65], applied without class labels. A mask p and a ground-truth object g match if their intersection over union (IoU) exceeds 0.5; since neither masks nor objects overlap, each object has at most one such mask, and the result is independent of the order of the masks. Unmatched objects count as false negatives (FN), unmatched masks as false positives (FP). Pooled over all frames,

$$\mathrm{PQ} = \underbrace{\frac{\sum_{(p,g)\in TP} \mathrm{IoU}(p,g)}{|TP|}}_{\text{segmentation quality (SQ)}} \times \underbrace{\frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}}_{\text{recognition quality (RQ)}}, \tag{5.1}$$

where TP is the set of matched pairs. SQ measures how closely the found objects are outlined, RQ, an F1 score, how many objects are found without spurious masks. All runs were performed on a Jetson AGX Thor (JetPack 7.2, TensorRT 10.16) in its maximum power mode without other workload. The latency covers the complete processing of a frame, from the image in memory to the final masks, and is the median over three passes after ten warm-up frames.

In a first experiment, NanoSAM [33] was compared with SAM with a ViT-B encoder [30] (PyTorch, full precision), both prompted with 3×3, 5×5 and 7×7 grids and post-processed identically: single-mask output per prompt, logit threshold 0.80, minimum mask size 3500 pixels and non-maximum suppression (NMS) at an IoU of 0.10. At every grid density, NanoSAM is more accurate and about ten times faster than ViT-B (Figure 5.2). The difference comes from how many objects are found, not from how well they are outlined. Both models produce about the same number of masks per frame, and the objects that ViT-B does find are outlined just as well. However, fewer of its masks correspond to a real object: they more often cover only part of an object or spill over onto its neighbours. Denser grids help both models with diminishing returns, as many prompts fall on the same large surfaces, and even the densest grid finds fewer than half of the objects. These results confirm the choice of NanoSAM (Section 2.7).



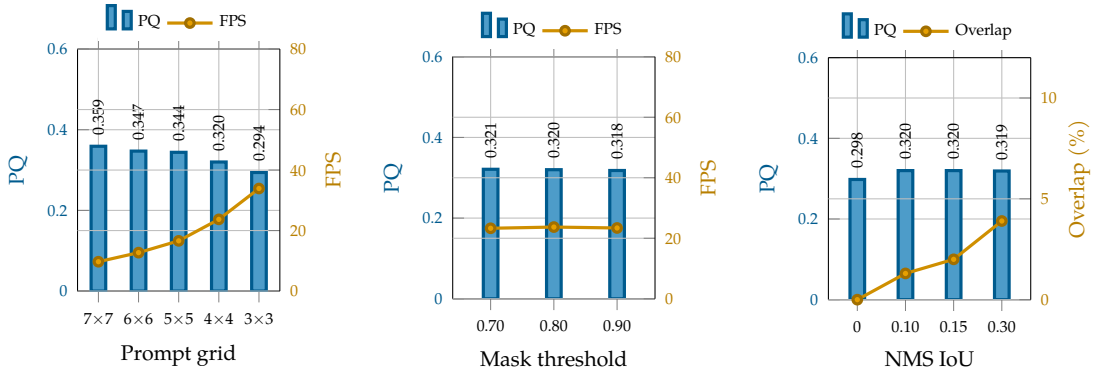
(a) PQ. (b) Latency. (c) RQ. (d) SQ.

Figure 5.2.: NanoSAM against SAM ViT-B on the benchmark: (a) PQ, (b) median latency (log scale), (c) RQ, (d) SQ.

A second experiment aimed to find a NanoSAM configuration that is fast enough for the pipeline while keeping as much of the segmentation quality as possible, and to check whether the overlap

between the generated masks can be reduced without losing accuracy. It used the same frames, scoring and timing as the first experiment, but took the input scale of 0.85, the cap of 8 masks and the minimum mask size of 3500 pixels from the pipeline configuration; the two experiments are therefore not directly comparable. Three parameters were varied one at a time, each step keeping the values chosen before. The grid density was varied from 7×7 to 3×3 and chosen as a trade-off between PQ and latency. The mask threshold (0.70, 0.80, 0.90) was then tested for its effect on PQ. Finally, the NMS IoU (0, 0.10, 0.15, 0.30) was varied to determine how strongly overlapping masks can be suppressed without reducing PQ, since overlapping masks lead to overlapping object crops.

Sparser grids lose PQ almost entirely through recall (Figure 5.3), while SQ and precision stay nearly constant: fewer prompts miss objects rather than producing poorer masks. Since the encoder takes a constant 11 ms, the latency is governed by the number of prompts. The loss of PQ steepens below 5×5, and a 4×4 grid was chosen as the balance, with a 95th-percentile latency of only 44 ms. The mask threshold has almost no effect on PQ, so a conservative value of 0.80 was chosen. For the NMS, only the complete suppression of overlaps costs accuracy, because it removes about one valid mask per frame; any IoU from 0.10 upwards gives the same PQ, and 0.30 was chosen, since it keeps slightly more masks per frame while the overlap remains a few per cent of the pixels.



(a) Grid density. (b) Mask threshold. (c) NMS IoU.

Figure 5.3.: Parameter study of NanoSAM (input scale 0.85, at most 8 masks): (a) grid density at threshold 0.80 and NMS IoU 0.10, (b) mask threshold with a 4×4 grid, (c) NMS IoU with a 4×4 grid and threshold 0.80. Overlap is the share of pixels covered by more than one mask.

The pipeline itself uses a 3×3 grid instead of the selected 4×4; all other values agree with the study. Apart from the NMS IoU, which has no measurable effect in this range, this corresponds to the 3×3 point of Figure 5.3a: it gives up 0.026 in PQ for about 13 ms less per frame, in order to leave more GPU time to Hydra and the vision-language model, with which segmentation shares the device (Section 5.5). The mask threshold of 0.80 is additionally supported by the tracking investigation, since a lower value enlarges the masks of touching objects and reduces the separation between their tracks. The consequences of the lower recall for tracking, in particular masks that merge touching surfaces, are examined in Section 5.2.

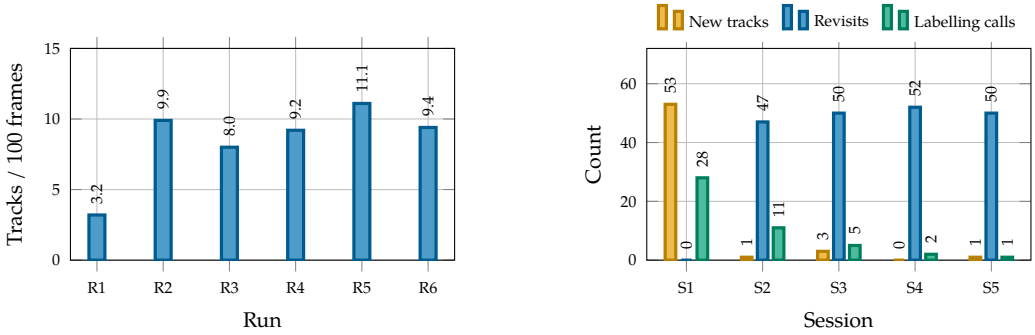
5.2. Object Tracking and Crop Selection

Tracking and view selection (Sections 3.3 and 3.4.1) were evaluated online, in the running pipeline, since both depend on the sequence of observations rather than on individual frames. All experiments replay the office sequence of uHumans2 [11] with ground-truth odometry at a rate of 0.1 (about 1.6 fps camera input) on a Jetson AGX Orin, using the implementation of Section 4.2.3. Because the odometry is exact, the map and odometry frames coincide and loop-closure re-anchoring is never triggered, so that the results reflect the association alone. Two records were used throughout: a log with one entry per mask-to-track decision, containing the association mode and the individual cue scores, and crops of each track saved at regular intervals, which show the raw single-frame mask. Where runs of different length are compared, the number of tracks is normalised per 100 processed frames.

To evaluate the quality of the tracking, an inspection setup was first established: after a run on a 300 s segment, the stored crops of all tracks were arranged in a gallery, one row per track, so that the tracks could be compared visually. For the initial configuration (A1), this inspection revealed duplicate tracks, that is, several tracks for one object seen from different viewpoints. A first experiment therefore tuned the parameters of the association score (Equation 3.3, Section 3.3.2) to remove them. Since every run replays the same segment, the total number of tracks served as the measure. The parameters were changed in five steps, A2 to A6; after each step, the number of tracks was recorded, the association log showed in which mode, recent or revisit, the duplicates had been evaluated, and the gallery was inspected again. The first two steps addressed the revisit mode: lowering its threshold $s_{\min}$ from 0.75 to 0.40 (A2) and raising its image weight to 0.35 (A3) raised the number of tracks from 86 to 93 and 92. The log showed why: the duplicates were created about 0.5 s after the last update of the original track, within the continuation window of 15 s, and were therefore evaluated in the recent mode. The remaining steps addressed this mode. Lowering its threshold to 0.40 and raising its image weight from 0.25 to 0.35 (A4) gave 79 tracks; lowering the threshold to 0.30 and shortening the continuation window to 8 s (A5) gave 64, 26 % below the initial value; and rebalancing the weights towards the volumetric overlap (A6) gave 63. The recent-mode weights w_k finally adopted are 0.70 for the volumetric overlap, 0.30 for the centroid and 0.45 for the image overlap.

The number of tracks, however, does not guarantee that each track contains only one object. A second experiment therefore inspected the content of the tracks, with the aim of finding nodes that combine two physical objects and removing their cause. For every track, the crop of every tenth observation was saved with the contour of that frame's own mask, together with the 3D box of the observation and the accumulated box of the track, and inspected manually. If the mask of a single frame already outlined two objects, the error came from the segmentation: in this *mask fusion*, anticipated in Section 5.1, NanoSAM returns one mask covering two touching surfaces, which the tracker cannot separate. If every mask was correct but the box of the track had grown around a second object, the association had added the mask of another object to the track; the frame at which this happened was located in the decision log by a sudden jump of the object position and an abrupt change of the mask size. Each inspected observation was manually marked as a wrong merge or a correct continuation. The log showed why such *cross-track merges* occurred: two of the five tests of Section 3.3.2 sufficed for a match, and two of them, height compatibility and box overlap, are passed by almost any object standing on a large surface,

so the centroid distance and the image overlap, the only tests that distinguish a surface from an object resting on it, had no influence. Since changing the weights did not prevent this, the acceptance rule was made stricter in successive runs, R1 to R6, each building on the previous one (Figure 5.4a): R2 required three agreeing tests and a stricter centroid limit, R3 repeated this setting, R4 raised the required box overlap, R5 disabled a shortcut that counted touching boxes as overlapping, and R6 discarded masks without enough valid depth for a 3D box. Each change was kept only if, replayed on the logged decisions, it rejected all marked wrong merges and accepted all marked correct continuations. After R6, the crops showed none of the confirmed merges, and its configuration was adopted as final. Following the asymmetric-error principle of Section 3.3, the resulting fragmentation was accepted: it rose from 3.2 to at most 11.1 tracks per 100 frames and settled at 9.4.



(a) Tightening runs.

(b) Sessions with persistence.

Figure 5.4.: Tracking results: (a) fragmentation over the tightening runs R1 to R6 of the second experiment; (b) new tracks, revisit associations and labelling calls over five sessions.

A third experiment measured how reliably objects are re-identified when the pipeline is restarted in an environment it has already mapped. The same 300 s segment, corresponding to 147 to 148 processed frames, was replayed in five consecutive sessions with the final association configuration. Session persistence (Section 3.6.2) was enabled for this experiment only, and the memory was cleared only before the first session. Each later session resumed from the state saved by the previous one, with the stored timestamps shifted so that restored tracks were first evaluated in the revisit mode. A re-encountered object counts as re-identified if it is associated with a stored track in the revisit mode instead of opening a new track. As Figure 5.4b shows, the first session created all tracks, whereas the following sessions created almost none and associated nearly every re-encountered object as a revisit. In total, 97.5 % of the re-encountered objects were re-identified; three of the five misses lie close together above 3.3 m and probably belong to one tall structure whose centroid shifts beyond the centroid limit as the viewpoint changes. Since restored tracks keep their labels, the number of labelling calls dropped to nearly zero, while the throughput remained unchanged across sessions (Section 5.5).

To evaluate the view selection of Section 3.4.1, which chooses the crop passed to the vision-language model for each track, the crops selected during a run were logged together with the individual terms of their score. A fourth experiment used this log to tune the additive score

of Equation 3.5, a weighted sum of pixel count, sharpness, measured as the variance of the Laplacian [61], and margin. The margin term was first redefined to count object pixels in a 3 px edge zone instead of on the exact crop border, which spread its values from 0.943–0.991 to 0.476–0.967 and thus made it able to separate tightly cut views from well-framed ones. The weights were then adjusted qualitatively on the crop log of one run with 16 tracks. Since the log contains the three terms of every selected crop, each candidate weighting could be replayed offline, and the replacements it would have accepted were inspected visually. Six ratios $w_P : w_S : w_M$ were compared, namely 1:1:1, 3:1:1, 4:1:1, 5:1:1, 2:2:1 and 2:2:0.5, using two criteria: a replacement that clearly improves the view must still be accepted, such as a gain of 0.59 % in Track 1 caused by better framing, whereas replacements between nearly identical views from consecutive frames should be avoided. Equal weights met the first criterion but not the second, as they still accepted many replacements between nearly identical views. Ratios dominated by the pixel term (3:1:1 to 5:1:1) met the second but not the first: they penalised small objects strongly and rejected genuine improvements, including the one in Track 1. The ratios 2:2:1 and 2:2:0.5 met both criteria and selected almost identical crops. The ratio 2:2:1, that is, $w_P = w_S = 0.4$ and $w_M = 0.2$, was adopted. It gives pixel count and sharpness equal and dominant weight, since both determine how much the vision-language model can see, while the margin term acts as a secondary criterion that favours better framing without rejecting a crop that is large and sharp but tightly cut; of the two equivalent ratios, the round value 1 for the margin was kept for clarity. The score takes about 1 ms per crop.

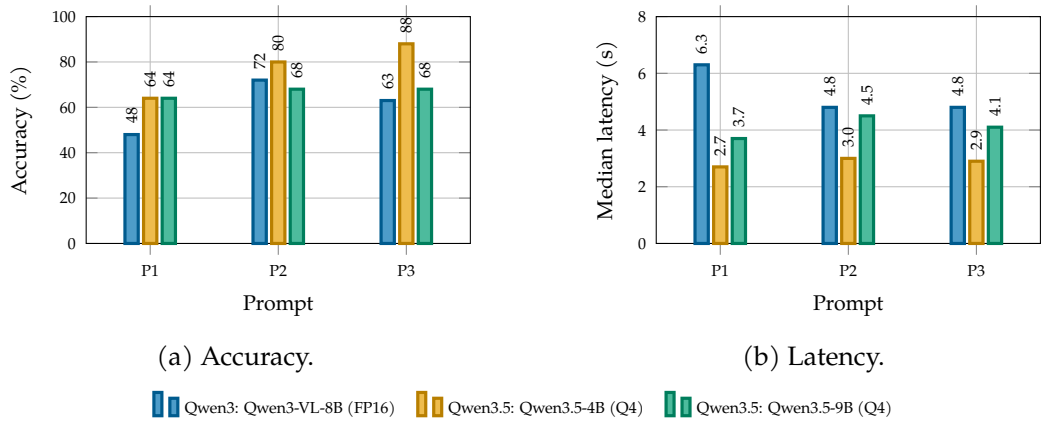
The pipeline uses the final association configuration of the second experiment (quorum of three, centroid gate 0.60 m, overlap threshold 0.50, containment threshold 0.92, touch shortcut disabled) and the crop score with $h = 0.005$. Session persistence is disabled by default, so that every run in the following sections starts from an empty map. The selected crops form the input of the labelling evaluated in Section 5.3; tracks affected by mask fusion receive a crop that shows two objects.

5.3. VLM Model and Prompt Comparison

The vision-language model and its prompt were evaluated online, in the running pipeline, so that the model received exactly the crops produced by the tracking and view selection of Section 5.2. Every run replayed the office sequence of uHumans2 [11] at a rate of 0.1 on a Jetson AGX Orin and was stopped after 700 s of wall-clock time, about 70 s of scene time, so that all runs covered the same part of the scene and produced about 50 object crops each. Retrieval was disabled and its memory cleared before every run, so that every unlabelled object was sent to the vision-language model. All other settings were those of the production configuration; only the model and the prompt were changed. The benchmark concerns the identification of objects only: the prompts compared request the label, the mobility class and their confidences, but no risk-relevant information, so the visual-state description of Section 3.4.2 was not part of it.

For every labelling call, the pipeline saved the crop as sent to the model, together with the returned label, mobility class and confidences, the validation result and the inference time reported by llama.cpp [64]. After each run, the crops were inspected manually, the correct label was recorded

and every wrong answer was assigned to an error type. Accuracy is strict: an answer counts as correct only if both label and mobility class are right, although, since all objects in this scene are static, mobility never cost a point.



(a) Accuracy. (b) Latency.

Figure 5.5.: Comparison of the three prompts on models of two generations, Qwen3 (blue) and Qwen3.5 (orange and green), on the office sequence ($N \approx 50$ crops per run): (a) strict accuracy, (b) median inference latency.

A first experiment used Qwen3-VL-8B at full precision (FP16) [53], the model of the baseline pipeline. Three prompts of increasing complexity, P1 to P3, were compared; they are given in Appendix A. P2 and P3 were each revised once against the errors of the preceding prompt on this model and then fixed. As the Qwen3 bars in Figure 5.5a show, P2 raised the accuracy from 48 to 72 %, whereas P3 fell back to 63 %. The rules and examples of P2 removed the main errors of P1, in which the model named an object outside the contour or a part instead of the whole object, whereas the additional guidance of P3 introduced new confusions between similar classes rather than resolving the remaining errors. P1 also yielded a nearly constant confidence on every crop, wrong answers included, so the validation gate never fired; only P3 produced values below 0.80, and both answers rejected for this reason were correct. At 4.8 to 6.3 s per call, the model was also slow.

A second experiment repeated the unchanged prompts with Qwen3.5-4B and Qwen3.5-9B [54], both quantised to 4 bits (Q4_K_M) and run with reasoning disabled, since the thinking block otherwise consumed the token budget. As the Qwen3.5 bars in Figure 5.5 show, the accuracy of the 4B model rose with every more detailed prompt, from 64 % with P1 to 88 % with P3, whereas the 9B model reached only 64 to 68 % with all three prompts. The larger model was thus not more accurate. On both Qwen3.5 models, P2 removed the main errors of P1 and P3 removed most of the remaining ones. The 9B, however, kept confusing visually similar classes whatever the prompt, which indicates a limitation of the model itself. As with the 8B, only P3 led the models to report confidences below the threshold of 0.80. One problem occurred only on the smaller models: with the first version of P3, the 4B wrote out its step-by-step reasoning in the reply and used up the token budget before giving the JSON answer. A rule at the beginning of the prompt stating that the reply must consist of exactly one JSON object, illustrated by a good and a bad example, solved this, and all answers were then valid.

The 4B model with the format-hardened P3 reached 88 % at a median of 2.9 s and is the most accurate, fastest and least memory-intensive pair. The pipeline therefore uses the profile `qwen3_5_4b_q4` with reasoning disabled, a temperature of 0, 512 tokens and P3 as the basis of its prompt; the minimum label confidence was kept at 0.80, so that uncertain labels are retried with a better crop. Since Qwen3.5-4B was at least as accurate as the baseline Qwen3-VL-8B with every prompt and 1.6 to 2.3 times faster, and was adopted by changing only the model profile and the prompt, the comparison supports the expectation of Section 2.9.3 that newer model generations improve the pipeline without changes to its other components. Its latency in the complete pipeline is reported in Section 5.5.

The risk-relevant information was built on top of the prompt selected in this comparison. For the risk assessment, the deployed prompt asks the model, in addition to the label and the mobility class, for the visual-state description of Section 3.4.2, a short phrase stating what is visibly particular about the object at the time of observation, for instance items placed on a table or whether a monitor is switched on. The description was added as a separate section appended to P3, so that the selected prompt itself remains unchanged and only the output contract is extended by a fifth field; the complete active prompt is given in Appendix A.4. Since this extension was introduced after the comparison, neither its effect on the label accuracy reported above nor the quality of the descriptions has been measured.

5.4. Visual Results across Environments

The preceding sections evaluated the components of the pipeline individually. This section shows the scene graphs that the complete pipeline produces in its final configuration and examines qualitatively what they contain and where they are wrong. The office and apartment sequences of uHumans2 [11] and the office1-1 sequence of OpenLORIS-Scene [12] were each replayed once at a rate of 0.1, which corresponds to a camera input of about 1.6 Hz for uHumans2 and about 3 Hz for OpenLORIS, on a **[TODO: platform; Jetson AGX Thor, to be confirmed from the stats files]**. All runs used the final configuration of Sections 5.1 to 5.3: NanoSAM with a 3×3 prompt grid, the final association configuration, and Qwen3.5-4B with prompt P3 for labelling, with retrieval and the risk VLM disabled. For OpenLORIS, the camera pose is obtained from the wheel odometry recorded with the dataset and the calibrated base-to-camera transform (Section 4.1.3). The images were taken in RViz at the end of each run, after all pending labelling requests had been processed; Hydra [7] provides the spatial layers, and the Fuser (Section 3.5.1) the labels, relations and presence confidences shown on them.

Since not all of this information can be read from the images, the content of the visualisation is summarised here. The graph is drawn in three layers that keep the horizontal position of every node and differ only in height. The lowest layer contains the *object nodes* at their reconstructed position. Each object is a small sphere, or a cube if its mobility class is dynamic, coloured by its label; objects that have not yet received a label are grey and carry the text `object_unknown`. The opacity of an object follows its presence confidence (Equation 3.6), so that objects not seen for some time fade but are never removed, and a translucent grey halo marks objects observed within the last few seconds. The text beside each object gives the label with its confidence, the mobility

class with its confidence and, where available, the visual-state description (Section 3.4.2). The middle layer, 10 m above the objects, contains the *place nodes* of Hydra [7], which represent the free space through which the robot can move; each place takes the colour of the room it belongs to, or grey if it belongs to none. The top layer, 20 m above the objects, contains the *room nodes*, larger spheres with one distinct colour per room and the labels Room1, Room2 and so on. The edges between these nodes are listed in Table 5.1.

Table 5.1.: Edges in the visualisation. *Completed by the Fuser where missing, if no wall lies between the nodes (Section 3.5.1).

Edge	Meaning	Source	Drawn as
Room–place	Place in room	Hydra*	Black line
Place–object	Object at place	Hydra*	Black line
Place–place	Free-space link	Hydra	Black line
Object–object	Hydra adjacency	Hydra	Black line
Room–room	Adjacent rooms	Hydra	Black line
Contact	Objects touch	Fuser	Amber double arrow
Identity	Same physical object	Fuser	Blue dotted line

[TODO: office paragraph; requires res_office_overview/layers/closeup/camera.png and res_office_stats.txt.]

[TODO: apartment paragraph; requires res_apartment_overview/closeup.png and res_apartment_stats.txt.]

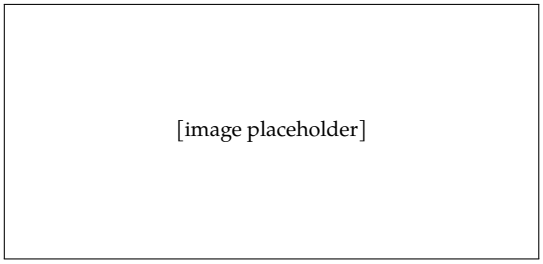
[TODO: OpenLORIS paragraph; requires res_openloris_overview/closeup.png and res_openloris_stats.txt.]

[TODO: comparison paragraph; requires the three stats files.]

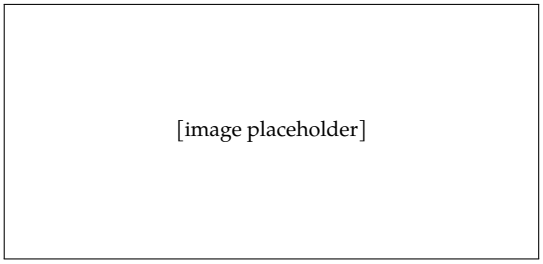
5.5. Runtime Performance

This section quantifies the throughput, per-frame latency and resource demand of the pipeline described in Chapter 4 on the two embedded platforms. In all runs, the uHumans2 office bag [11] (`office_s1_00h_v2`, 506 s, RGB at 16.41 Hz) was replayed at a rate of 1.0, so that the input exceeds the processing capacity of either platform, frames are dropped and the maximum sustained throughput is measured. The Jetson AGX Thor [58] (power mode MAXN) was evaluated in three runs of 300 s, each starting from an empty memory, in the final configuration (object VLM Qwen3.5-4B [54], Q4_K_M, prompt P3); the first 20 frames of each run are excluded as warm-up. The Jetson AGX Orin [56] (power mode MODE_50W) serves as the reference. Timings come from the per-frame trace of the pipeline, resource figures from `tegrastats` and `nvidia-smi`.

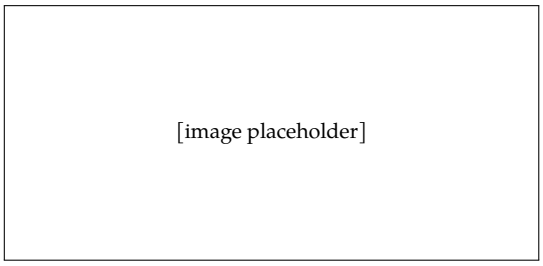
Only the throughput figures are strictly like-for-like, since both platforms ran the same final configuration. Thor processes about twice as many frames per second as Orin (factor 2.03, Figure 5.7a), that is, about 30 % of the input frames instead of 14 %; the variation across the three Thor runs is small, all dropped frames were discarded by the drop-oldest policy of the input



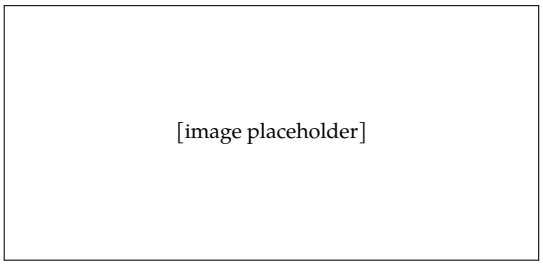
(a) Office: fused scene graph.



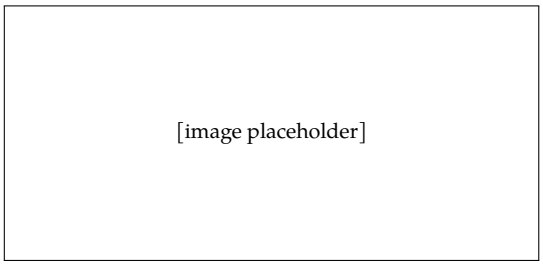
(b) Office: Hydra layers.



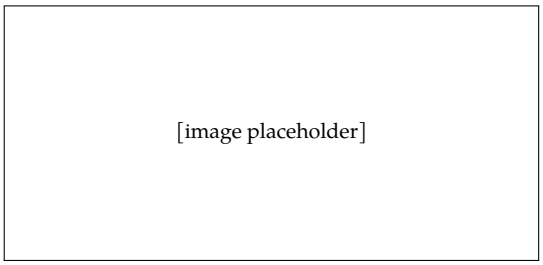
(c) Office: detail.



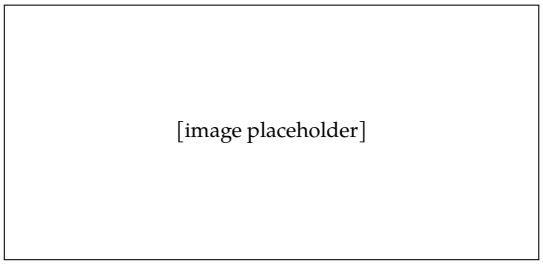
(d) Office: camera view.



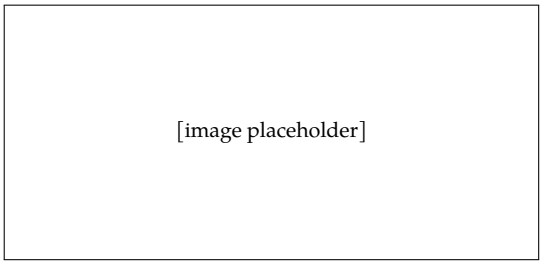
(e) Apartment: fused scene graph.



(f) Apartment: detail.



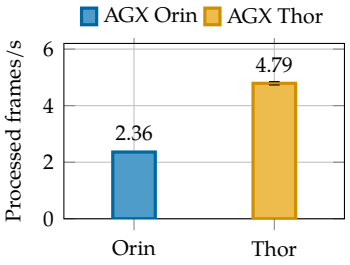
(g) OpenLORIS office1-1: fused scene graph.



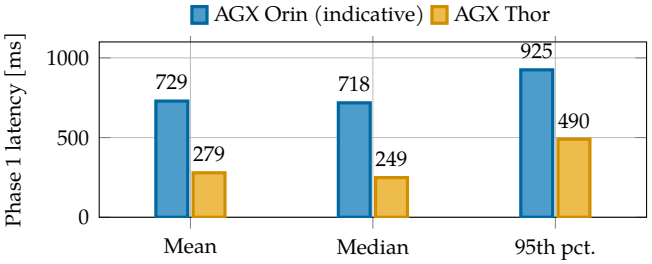
(h) OpenLORIS office1-1: detail.

Figure 5.6.: Scene graphs of the uHumans2 office (a–d) and apartment (e, f) sequences and of the OpenLORIS-Scene office1-1 sequence (g, h) at the end of each run, displayed in RViz. From bottom to top: objects, places (+10 m) and rooms (+20 m). Objects are coloured by label (grey: unlabelled), their opacity follows the presence confidence, and a grey halo marks currently observed objects. Black lines: hierarchy and Hydra edges; amber double arrows: contact relations; blue dotted lines: identity relations. **[TODO: adapt to what the panels actually show.]**

queue, and no frame failed. The per-frame Phase 1 latency, measured from frame arrival until publication to Hydra (Section 4.2.3), is reduced by a factor of about 2.6 on average (Figure 5.7b).



(a) Throughput, final configuration



(b) Per-frame latency, frame arrival to Hydra input

Figure 5.7.: Runtime at saturated input: (a) throughput (Thor: mean ± std of three runs), (b) Phase 1 latency (Orin indicative).

Figure 5.8 decomposes the mean frame time into five stage groups. Segmentation with NanoSAM [33] dominates on both platforms, with about two thirds of the frame time, and is about 2.6 times faster on Thor, almost entirely through faster inference. Since segmentation sets the frame time, the end-to-end reduction of the latency essentially follows the segmentation gain. The CPU-side stages shrink more strongly, by factors of about 4.7 to 7.

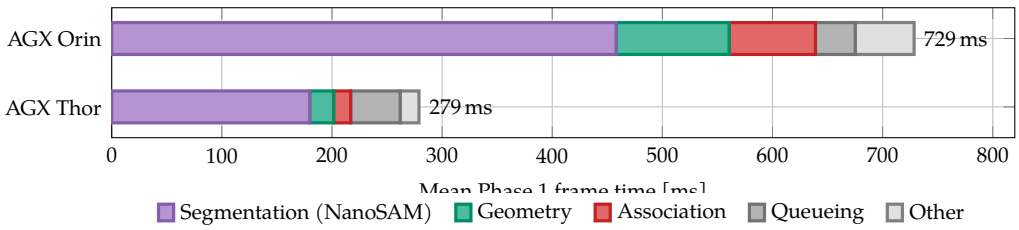
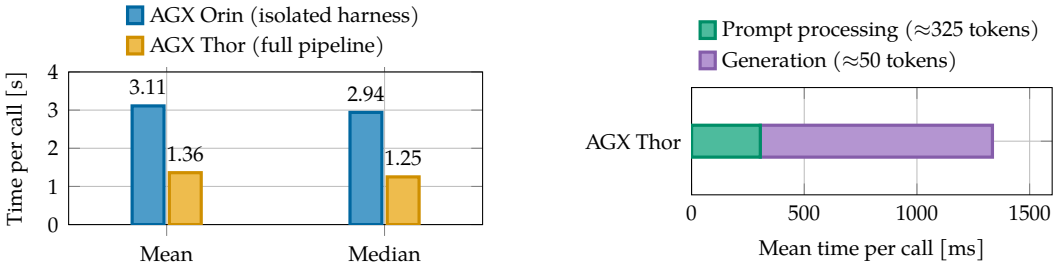


Figure 5.8.: Mean Phase 1 frame time by stage group ("Other": message construction, crop scoring and minor stages; Orin indicative).

With the same model and prompt, an object-labelling call on Thor is about 2.3 times faster than in the Orin measurements of Section 5.3, although on Thor it shares the GPU with segmentation (Figure 5.9a). Most of a call is spent generating the answer, at about 49 tokens/s, rather than processing the prompt (Figure 5.9b), as reported by the llama.cpp server [64]. Even at this speed, the VLM limits the labelling path on a first visit. Each run issues 463 to 492 calls, more than the server can handle as they arrive: the request queue, with a capacity of 300, stays close to full and the server is busy about 98 % of the time. A request therefore waits 199 to 210 s on average, and the remaining backlog needs a further 330 to 380 s to clear after the bag has ended. As a result, semantic labels lag the geometry by several minutes on a first pass. This delay is limited to first visits: objects that are already known keep their labels, so the number of calls falls sharply on revisits (from 28 to 1 over five sessions, Section 5.2). A mitigation for first visits is discussed in Chapter **??**.



(a) Per-call latency

(b) Split of a Thor call

Figure 5.9.: Object-labelling VLM (Qwen3.5-4B, Q4_K_M, prompt P3). (a) Time per call on Orin without segmentation load and on Thor within the running pipeline. (b) Split of a Thor call into prompt processing and token generation, taken from the server log.

The pipeline is GPU-bound on Thor. During processing, the GPU was busy about 90 % of the time on average, whereas the CPU load stayed at about 40 %. Memory use remained at about 24 GiB, a fraction of the 122 GiB available. The board drew about 81 W on average and at most 109 W, which corresponds to about 17 J per processed frame, and the GPU temperature did not exceed 63 °C. Downstream of Phase 1, the update rates were measured on the published topics, so they are output rates rather than internal computation times. Hydra [7] (Section 4.2.4) published a new scene graph at about 1.4 Hz, and a processed frame typically appeared in the next graph within about 0.4 s (median). The Fuser updated its output at only about 0.3 Hz instead of the configured 1 Hz. The risk layer thus receives a refreshed scene graph at about 1.4 Hz, but a fused, risk-annotated view only about every 3 to 4 s. Overall, the Jetson AGX Thor processes about 4.8 frames per second, roughly twice the rate of the Orin, and geometry and object tracking keep pace with this rate, so that each processed frame reaches Hydra within about 0.3 s on average. Semantic labelling, in contrast, lags considerably on a first visit and catches up only once objects are revisited, which defines the principal runtime limitation discussed in Chapter **??**.

6. Conclusions

Bibliography

[1] André Schiel, "Online Risk-Annotated Scene Graohs for Autonomous Systems," Master's thesis, University of Stuttgart, Stuttgart, Germany, 2026.

[2] International Federation of Robotics. "Service robots see global growth boom." Press release on World Robotics 2025 – Service Robots. (Oct. 7, 2025), [Online]. Available: https://ifr.org/ifr-press-releases/news/service-robots-see-global-growth-boom (visited on 09/28/2026).

[3] International Federation of Robotics, *World robotics 2025: Press conference presentation*, 2025. [Online]. Available: https://ifr.org/downloads/press_docs/PressConference2025_presentation.pdf.

[4] A. V. Silva Neto, J. B. Camargo, J. R. Almeida, and P. S. Cugnasca, "Safety assurance of artificial intelligence-based systems: A systematic literature review on the state of the art and guidelines for future work," *IEEE Access*, vol. 10, pp. 130 733–130 770, 2022. DOI: 10.1109/ACCESS.2022.3229233.

[5] International Organization for Standardization, *Iso 12100: Safety of machinery – general principles for design – risk assessment and risk reduction*, International Organization for Standardization, 2010.

[6] P. Grimmeisen, R. Golwalkar, Y. Ma, and A. Morozov, "Automated and continuous risk assessment for ros-based software-defined robotic systems," in *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, IEEE, 2023, pp. 1–7. DOI: 10.1109/CASE56687.2023.10260416.

[7] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3d scene graph construction and optimization," in *Robotics: Science and Systems*, New York City, NY, USA, 2022.

[8] Q. Gu, A. Kuwajerwala, S. Morin, *et al.*, "Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning," in *2024 IEEE International Conference on Robotics and Automation*, IEEE, 2024, pp. 5021–5028. DOI: 10.1109/ICRA57147.2024.10610243.

[9] J. F. J. Mullen, P. Goyal, R. Piramuthu, M. Johnston, D. Manocha, and R. Ghanadan, ""don't forget to put the milk back!" dataset for enabling embodied agents to detect anomalous situations," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 9087–9094, 2024. DOI: 10.1109/LRA.2024.3430129.

[10] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074.

[11] A. Rosinol, A. Violette, M. Abate, N. Hughes, Y. Chang, J. Shi, A. Gupta, and L. Carlone, "Kimera: From slam to spatial perception with 3d dynamic scene graphs," *The International Journal of Robotics Research*, vol. 40, no. 12–14, pp. 1510–1546, 2021. DOI: 10.1177/02783649211056674.

[12] X. Shi, D. Li, P. Zhao, Q. Tian, Y. Tian, Q. Long, C. Zhu, J. Song, F. Qiao, L. Song, *et al.*, "Are we ready for service robots? The OpenLORIS-Scene datasets for lifelong SLAM," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2020, pp. 3139–3145. DOI: 10.1109/ICRA40945.2020.9196638.

[13] M. Osborne, R. D. Hawkins, and J. A. McDermid, "Analysing the safety of decision-making in autonomous systems," in *Computer Safety, Reliability, and Security*, ser. Lecture Notes in Computer Science, vol. 13414, Springer, 2022, pp. 3–16. DOI: 10.1007/978-3-031-14835-4_1.

[14] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004. DOI: 10.1109/TDSC.2004.2.

[15] International Organization for Standardization, *Iso 10218-1:2025 robotics – safety requirements – part 1: Industrial robots*, International Organization for Standardization, 2025. [Online]. Available: https://www.iso.org/standard/73933.html (visited on 05/25/2026).

[16] International Organization for Standardization, *Iso 10218-2:2025 robotics – safety requirements – part 2: Industrial robot applications and robot cells*, International Organization for Standardization, 2025. [Online]. Available: https://www.iso.org/standard/73934.html (visited on 05/25/2026).

[17] International Organization for Standardization, *Iso/ts 15066:2016 robots and robotic devices – collaborative robots*, International Organization for Standardization, 2016. [Online]. Available: https://www.iso.org/standard/62996.html (visited on 05/25/2026).

[18] European Commission, "Industry 5.0: Towards a sustainable, human-centric and resilient european industry," European Commission, 2021. [Online]. Available: https://op.europa.eu/en/publication-detail/-/publication/468a892a-5097-11eb-b59f-01aa75ed71a1 (visited on 05/25/2026).

[19] S. P. Kumar, C. Savur, and F. Sahin, "Survey of human–robot collaboration in industrial settings: Awareness, intelligence, and compliance," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 1, pp. 280–297, 2021. DOI: 10.1109/TSMC.2020.3041231.

[20] J. A. Marvel, "Performance metrics of speed and separation monitoring in shared workspaces," *IEEE Transactions on Automation Science and Engineering*, vol. 10, no. 2, pp. 405–414, 2013. DOI: 10.1109/TASE.2013.2237904.

[21] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3d scene graph: A structure for unified semantics, 3d space, and camera," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 5664–5673.

[22] S.-C. Wu, J. Wald, K. Tateno, N. Navab, and F. Tombari, "Scenegraphfusion: Incremental 3d scene graph prediction from rgb-d sequences," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7515–7525.

[23] Y. Deng, J. Wang, J. Zhao, X. Tian, G. Chen, Y. Yang, and Y. Yue, "Opengraph: Open-vocabulary hierarchical 3d graph representation in large-scale outdoor environments," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8402–8409, 2024. DOI: 10.1109/LRA.2024.3439122.

[24] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: An open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.

[25] L. Puck, P. Keller, T. Schnell, C. Plasberg, A. Tanev, G. Heppner, A. Roennau, and R. Dillmann, "Performance evaluation of real-time ros2 robotic control in a time-synchronized distributed network," in *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*, IEEE, 2021, pp. 1670–1676. DOI: 10.1109/CASE49439.2021.9551447.

[26] J. Johnson, R. Krishna, M. Stark, L.-J. Li, D. A. Shamma, M. S. Bernstein, and L. Fei-Fei, "Image retrieval using scene graphs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3668–3678. DOI: 10.1109/CVPR.2015.7298990.

[27] R. Krishna, Y. Zhu, O. Groth, *et al.*, "Visual genome: Connecting language and vision using crowdsourced dense image annotations," *International Journal of Computer Vision*, vol. 123, pp. 32–73, 2017. DOI: 10.1007/s11263-016-0981-7.

[28] J. Wang, K. Sun, T. Cheng, *et al.*, "Deep high-resolution representation learning for visual recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3349–3364, 2021. DOI: 10.1109/TPAMI.2020.2983686.

[29] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Scene parsing through ade20k dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 633–641. DOI: 10.1109/CVPR.2017.544.

[30] A. Kirillov, E. Mintun, N. Ravi, *et al.*, "Segment anything," *arXiv preprint arXiv:2304.02643*, 2023. arXiv: 2304.02643 [cs.CV].

[31] FastAPI. "Fastapi documentation." (2026), [Online]. Available: `https://fastapi.tiangolo.com/` (visited on 05/28/2026).

[32] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using networkx," in *Proceedings of the 7th Python in Science Conference*, Pasadena, CA, USA, 2008, pp. 11–15.

[33] NVIDIA-AI-IOT. "Nanosam: A distilled segment anything model for real-time inference on nvidia jetson." (2023), [Online]. Available: `https://github.com/NVIDIA-AI-IOT/nanosam` (visited on 10/02/2026).

[34] NVIDIA. "Nvidia tensorrt documentation." (2026), [Online]. Available: `https://docs.nvidia.com/deeplearning/tensorrt/` (visited on 10/02/2026).

[35] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[36] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 16 000–16 009. DOI: `10.1109/CVPR52688.2022.01553`.

[37] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in *NIPS Deep Learning and Representation Learning Workshop*, arXiv:1503.02531, 2015.

[38] C. Zhang, D. Han, Y. Qiao, J. U. Kim, S.-H. Bae, S. Lee, and C. S. Hong, "Faster segment anything: Towards lightweight SAM for mobile applications," *arXiv preprint arXiv:2306.14289*, 2023.

[39] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu, and L. Yuan, "TinyViT: Fast pretraining distillation for small vision transformers," in *European Conference on Computer Vision*, 2022, pp. 68–85.

[40] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778. DOI: `10.1109/CVPR.2016.90`.

[41] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision*, 2014, pp. 740–755.

[42] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[43] J. McCormac, R. Clark, M. Bloesch, A. J. Davison, and S. Leutenegger, "Fusion++: Volumetric object-level slam," in *2018 International Conference on 3D Vision (3DV)*, IEEE, 2018, pp. 32–41. DOI: `10.1109/3DV.2018.00015`.

[44] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969. DOI: 10.1109/ICCV.2017.322.

[45] A. Radford, J. W. Kim, C. Hallacy, *et al.*, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 8748–8763.

[46] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[47] Chroma. "Chroma documentation." (2026), [Online]. Available: https://docs.trychroma.com/ (visited on 10/03/2026).

[48] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[49] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J.-R. Wen, "Evaluating object hallucination in large vision-language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 292–305.

[50] M. Xiong, Z. Hu, X. Lu, Y. Li, J. Fu, J. He, and B. Hooi, "Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs," in *International Conference on Learning Representations*, 2024.

[51] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, *et al.*, "Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution," *arXiv preprint arXiv:2409.12191*, 2024.

[52] S. Bai, K. Chen, X. Liu, *et al.*, "Qwen2.5-vl technical report," *arXiv preprint arXiv:2502.13923*, 2025. arXiv: 2502.13923 [cs.CV].

[53] Qwen Team, "Qwen3-VL technical report," *arXiv preprint arXiv:2511.21631*, 2025.

[54] Qwen Team. "Qwen3.5: Towards native multimodal agents." (2026), [Online]. Available: https://qwen.ai/blog?id=qwen3.5 (visited on 10/03/2026).

[55] Artificial Analysis. "Qwen3.5 small models: Everything you need to know." (2026), [Online]. Available: https://artificialanalysis.ai/articles/qwen3-5-small-models (visited on 10/03/2026).

[56] NVIDIA. "Jetson agx orin for next-gen robotics." (2026), [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/ (visited on 06/01/2026).

[57] A. Schiel, "Online risk-annotated scene graphs for autonomous systems," M.S. thesis, University of Stuttgart, Stuttgart, Germany, 2026.

[58] NVIDIA. "Jetson thor: Advanced ai for physical robotics." (2026), [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-thor/ (visited on 06/01/2026).

[59] NVIDIA Jetson AI Lab. "Jetson ai lab models." (2026), [Online]. Available: https://www.jetson-ai-lab.com/models/ (visited on 06/01/2026).

[60] L. Keselman, J. I. Woodfill, A. Grunnet-Jepsen, and A. Bhowmik, "Intel RealSense stereoscopic depth cameras," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017, pp. 1–10. DOI: 10.1109/CVPRW.2017.167.

[61] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: A comparative study," in *Proceedings of the 15th International Conference on Pattern Recognition (ICPR)*, vol. 3, IEEE, 2000, pp. 314–317. DOI: 10.1109/ICPR.2000.903548.

[62] Intel Corporation. "Intel realsense depth camera d455." (2026), [Online]. Available: https://www.intelrealsense.com/depth-camera-d455/ (visited on 10/11/2026).

[63] Unitree Robotics. "Unitree go1." (2026), [Online]. Available: https://www.unitree.com/go1 (visited on 10/11/2026).

[64] G. Gerganov and llama.cpp contributors. "Llama.cpp: LLM inference in C/C++." (2026), [Online]. Available: https://github.com/ggml-org/llama.cpp (visited on 10/11/2026).

[65] A. Kirillov, K. He, R. Girshick, C. Rother, and P. Dollár, "Panoptic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. DOI: 10.1109/CVPR.2019.00963.

A. Prompts of the VLM Comparison

This appendix lists the three prompts compared in Section 5.3 verbatim. All three request the label, its confidence and the mobility class with its confidence; none requests the risk-relevant visual-state description of Section 3.4.2. The prompt currently active in the pipeline, which adds this description to P3, is given in Section A.4. The cyan boundary referred to in all prompts is the contour drawn around the target in the crop (Section 3.4.1).

A.1. Zero-Shot Prompt (P1)

```
Classify the single SAM-segmented target in this indoor crop.

The target object is the one outlined by the cyan boundary. Identify only this object. The
     surrounding area may be used as context to help identify it.

Assume an indoor setting.

Name the object with a concise, singular, lowercase snake_case label that best describes what it
     actually is. There is no fixed list of labels. Do not describe colour, material, position,
     condition or activity. Use a confidence above 0.90 only when the identification is
     unmistakable. If the target is unclear, truncated or unidentifiable, use the label
     VLM_no_result with confidence 0.0.

Mobility:
- "dynamic": a human, an animal, or a self-propelled robot only
- "static": everything else
- "unknown": when the label is unknown or mobility cannot be determined

Output exactly one JSON object:
{"label": "<label>", "label_confidence": <0-1>, "mobility_class": "<static|dynamic|unknown>",
     "mobility_confidence": <0-1>}

Return only the JSON object. No markdown, explanations, or extra text.
```

A.2. Example-Driven Prompt (P2)

```
Identify the object that the cyan boundary encloses in this indoor crop.

WHICH OBJECT TO NAME
```

```
- The cyan boundary marks one region of the image. Name only what lies inside that boundary.
- Everything outside the boundary is background. A large, obvious object outside the boundary -
    a chair, a shelf, a bookcase, a whiteboard, a plant - is NOT the answer. Use the
    surroundings only to understand what the enclosed region is, never as the label itself.
- First decide what the enclosed region mainly shows, then name that.

HOW TO CHOOSE THE LABEL
- Name the largest, most complete thing the boundary covers. If the boundary encloses only part
    of a bigger structure, name the bigger structure.
- If the boundary spans many repeated units of one surface - several ceiling tiles or panels, a
    run of floor, a stretch of wall - name the surface itself ("ceiling", "floor", "wall"), not
    a single unit ("ceiling_tile").
- If the boundary covers a surface that carries a smaller mounted or attached item - a wall
    bearing a whiteboard, sign, monitor or vent; a ceiling holding a light or diffuser; a floor
    with a rug on it - name the surface, not the attached item. Name the attached item only
    when it clearly fills the boundary on its own.
- When you are torn between a specific object and the surface or structure behind it, choose the
    surface or structure.
- Use a concise, singular, lowercase label that best fits what you actually see. There is no
    fixed list of labels - name whatever it is.

CONFIDENCE ("label_confidence") - be honest, do not default to 0.95
- 0.90-1.00: unmistakable; the enclosed region can only be this.
- 0.60-0.85: probable, but a competing reading exists.
- 0.30-0.55: genuinely ambiguous, low detail, or a guess.
- 0.0 with label VLM_no_result: too blurred, dark, or truncated to identify at all.
Do not report 0.90+ on a crop you would not bet on.

Mobility:
- "dynamic": a human, an animal, or a self-propelled robot only
- "static": everything else
- "unknown": when the label is unknown or mobility cannot be determined

Output exactly one JSON object:
{"label": "<label>", "label_confidence": <0-1>, "mobility_class": "<static|dynamic|unknown>",
    "mobility_confidence": <0-1>}

WORKED EXAMPLES (these show the reasoning, not a list of allowed labels):
- Boundary is on a wall plane; an armchair sits in front of it, outside the boundary ->
    {"label": "wall", "label_confidence": 0.86, "mobility_class": "static",
    "mobility_confidence": 0.99}
- Boundary is on the floor; a shelf unit stands beyond it, outside the boundary -> {"label":
    "floor", "label_confidence": 0.9, "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary spans a panelled ceiling covering many tiles and a light fixture -> {"label":
    "ceiling", "label_confidence": 0.92, "mobility_class": "static", "mobility_confidence":
    0.99}
- Boundary covers a wall area that has a whiteboard mounted on part of it -> {"label": "wall",
    "label_confidence": 0.85, "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary covers a single whiteboard that fills almost the whole region -> {"label":
    "whiteboard", "label_confidence": 0.9, "mobility_class": "static", "mobility_confidence":
    0.99}
- Boundary is on a flat vertical surface with little detail - could be a cabinet front, a pillar
    or a wall -> {"label": "wall", "label_confidence": 0.4, "mobility_class": "static",
    "mobility_confidence": 0.9}
- Boundary shows a reflective rectangular panel on a wall - could be a mirror or a whiteboard ->
    {"label": "wall", "label_confidence": 0.4, "mobility_class": "static",
    "mobility_confidence": 0.9}
- Boundary encloses a recessed light set into a ceiling panel -> {"label": "ceiling",
    "label_confidence": 0.9, "mobility_class": "static", "mobility_confidence": 0.99}
```

```
- Boundary encloses a floor with a rug lying on it -> {"label": "floor", "label_confidence":
    0.9, "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary encloses a potted plant that fills the region -> {"label": "potted_plant",
    "label_confidence": 0.88, "mobility_class": "static", "mobility_confidence": 0.95}
- Boundary encloses a door and its frame -> {"label": "door", "label_confidence": 0.92,
    "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary encloses a person standing in the space -> {"label": "person", "label_confidence":
    0.93, "mobility_class": "dynamic", "mobility_confidence": 0.98}
- Boundary region is blurred, dark or cut off with no identifiable object -> {"label":
    "VLM_no_result", "label_confidence": 0.0, "mobility_class": "unknown",
    "mobility_confidence": 0.0}

Return only the JSON object. No markdown, explanations, or extra text.
```

A.3. Structural-Priority Prompt (P3)

The prompt is given in its final, format-hardened version, as used in runs R6 and R9. The run with Qwen3-VL-8B (R3) used the earlier version, which did not yet contain the output rule at the beginning, the note to reason silently in the heading of the approach, or the pair of good and bad example replies; its instructions and examples were otherwise the same.

```
OUTPUT RULE -- READ THIS FIRST
Your entire reply is ONE JSON object and nothing else. The first character you output is "{" and
    the last is "}". No preamble, no analysis, no numbered steps, no sentences such as "Based
    on the visual evidence...", and no markdown code fences. Work through the approach below
    silently and output only the result.

This is an indoor object-classification task. Identify the object that the cyan boundary
    encloses.

APPROACH (reason through this silently - do not write any of it)
1. Look only inside the cyan boundary. Everything outside it is background - never name it,
    however large or obvious. Use the surroundings only to work out what the enclosed region is.
2. Decide what the enclosed region mainly shows.
3. Choose the label using the priority order below.

PRIORITY ORDER
a. Room-defining structure over incidental detail. The large planes that form the room -
    overhead, underfoot, the vertical sides - and the openings and supports in them take
    priority over small things mounted on, set into, or resting against them. If the boundary
    spans many repeated units of one surface (several ceiling tiles or panels, a run of floor,
    a stretch of wall), name the surface itself ("ceiling", "wall", "floor"), not one unit
    ("ceiling_tile").
b. BUT distinctive infrastructure that fills the boundary is named, not generalised. If exposed
    pipes, ductwork, a large air vent or diffuser, a fire hose reel, a sprinkler main or a big
    fixture dominates the enclosed region, name that thing. Collapsing to "ceiling" or "wall"
    is only right when the fixture is small or incidental and the surface is what the boundary
    mostly shows.
c. Furniture over the objects on it - but only with a furniture cue. A freestanding piece of
    furniture outranks the smaller things on it. A large flat vertical surface with no handle,
    seam, gap, toe-kick or visible depth is a wall or a pillar, not a cabinet - do not upgrade
    a plain surface to furniture without a cue.
d. When two readings compete, choose the larger, more dominant, more structural one.
```

```
MATERIAL AND AMBIGUITY CUES
- A transparent or translucent panel, a framed glass screen, a glazed partition or a
    glass-panelled door -> name it as glass (for example "glass_wall", "glass_partition",
    "glass_door"). Offices often divide space with glass rather than solid wall.
- Judge transparency directly: if the room, objects, people or light behind the panel are
    visible through it - even dimly, even with a frame, mullions or a faint surface reflection
    - it is glass; a panel you cannot see through at all is solid (wall, door, cabinet).
- A rectangular panel that reflects the room or shows a mirror image -> mirror. A plain matte
    white or lightly-marked panel -> whiteboard. If you cannot tell, name the wall it sits on
    at low confidence.
- Low detail, deep shadow, motion blur or heavy truncation -> do not commit to a specific
    object; give the structural reading (wall / floor / ceiling) at low confidence, or
    VLM_no_result if nothing is identifiable.

LABEL FORM
- A concise, singular, lowercase label that best describes what it actually is. There is no
    fixed list of labels - name whatever it is. Do not describe colour, condition or activity.

CONFIDENCE CALIBRATION for "label_confidence" - report it honestly
- 0.90-1.00: unmistakable; the enclosed region can only be this.
- 0.70-0.90: clear, minor uncertainty.
- 0.45-0.70: plausible, but a competing reading exists - the right band for cabinet-vs-wall and
    mirror-vs-whiteboard.
- 0.20-0.45: genuinely ambiguous, or a guess.
- 0.0 with label VLM_no_result: incomprehensible crop.
A low confidence on a hard crop is better than a confident wrong answer. Do not default to 0.90+.

Mobility:
- "dynamic": only a human, an animal, or a self-propelled moving object
- "static": any stationary object, including all surfaces, structures, furniture, glass and
    fixtures
- "unknown": when the label is unknown

OUTPUT - exactly one JSON object, with nothing before it or after it:
{"label": "<label>", "label_confidence": <0-1>, "mobility_class": "<static|dynamic|unknown>",
    "mobility_confidence": <0-1>}

GOOD - this is the whole reply:
{"label": "wall", "label_confidence": 0.85, "mobility_class": "static", "mobility_confidence":
    0.99}

BAD - never reply like any of these:
- Based on the visual evidence within the cyan boundary: 1. Analysis: ... then the JSON
- The object is a wall. {"label": "wall", ...}
- the JSON object wrapped in triple-backtick code fences

WORKED EXAMPLES (illustrating the approach, not a list of allowed labels):

Room-defining structure over incidental detail:
- Boundary spans a panelled ceiling covering many tiles and a recessed light -> {"label":
    "ceiling", "label_confidence": 0.92, "mobility_class": "static", "mobility_confidence":
    0.99}
- Boundary covers a wall area with a whiteboard mounted on part of it -> {"label": "wall",
    "label_confidence": 0.85, "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary on a wall plane; an armchair sits in front of it, outside the boundary -> {"label":
    "wall", "label_confidence": 0.86, "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary on a floor area with a rug on it -> {"label": "floor", "label_confidence": 0.9,
    "mobility_class": "static", "mobility_confidence": 0.99}
```

```
Distinctive infrastructure that fills the boundary - name it:
- Exposed pipes or ductwork running across the ceiling, filling the boundary -> {"label":
     "pipes", "label_confidence": 0.8, "mobility_class": "static", "mobility_confidence": 0.99}
- A large air vent or diffuser that dominates the enclosed region -> {"label": "air_vent",
     "label_confidence": 0.82, "mobility_class": "static", "mobility_confidence": 0.99}
- A fire hose reel on the wall, filling the boundary -> {"label": "fire_hose_reel",
     "label_confidence": 0.85, "mobility_class": "static", "mobility_confidence": 0.99}

Glass and reflective surfaces:
- A framed glass partition dividing two areas -> {"label": "glass_wall", "label_confidence":
     0.8, "mobility_class": "static", "mobility_confidence": 0.99}
- A glass-panelled door in a frame -> {"label": "glass_door", "label_confidence": 0.82,
     "mobility_class": "static", "mobility_confidence": 0.99}
- A reflective rectangular panel showing a mirror image of the room -> {"label": "mirror",
     "label_confidence": 0.75, "mobility_class": "static", "mobility_confidence": 0.99}
- A plain white rectangular panel on a wall, no reflection -> {"label": "whiteboard",
     "label_confidence": 0.7, "mobility_class": "static", "mobility_confidence": 0.99}

Furniture vs plain surface:
- An office chair filling most of the frame -> {"label": "office_chair", "label_confidence":
     0.9, "mobility_class": "static", "mobility_confidence": 0.99}
- A desk seen clearly, with items on top -> {"label": "desk", "label_confidence": 0.9,
     "mobility_class": "static", "mobility_confidence": 0.99}
- A flat vertical surface with a visible handle, seam and toe-kick -> {"label": "cabinet",
     "label_confidence": 0.8, "mobility_class": "static", "mobility_confidence": 0.99}
- A flat vertical surface with no handles, seams or depth cues -> {"label": "wall",
     "label_confidence": 0.5, "mobility_class": "static", "mobility_confidence": 0.95}

Moving agents:
- A person in the space -> {"label": "person", "label_confidence": 0.93, "mobility_class":
     "dynamic", "mobility_confidence": 0.98}

Unclear:
- A blurred, dark or truncated region with no identifiable object -> {"label": "VLM_no_result",
     "label_confidence": 0.0, "mobility_class": "unknown", "mobility_confidence": 0.0}

Return only the JSON object - nothing before "{", nothing after "}". No markdown, no
     explanation, no numbered analysis.
```

A.4. Deployed Prompt with Visual-State Description

The listing below gives the complete prompt currently active in the pipeline. It consists of the structural-priority prompt P3 of Section A.3, unchanged, followed by an appended section that requests the visual-state description as a fifth output field. This extension was introduced after the comparison of Section 5.3 and was not evaluated there.

```
OUTPUT RULE -- READ THIS FIRST
Your entire reply is ONE JSON object and nothing else. The first character you output is "{" and
     the last is "}". No preamble, no analysis, no numbered steps, no sentences such as "Based
     on the visual evidence...", and no markdown code fences. Work through the approach below
     silently and output only the result.
```

```
This is an indoor object-classification task. Identify the object that the cyan boundary
    encloses.

APPROACH (reason through this silently - do not write any of it)
1. Look only inside the cyan boundary. Everything outside it is background - never name it,
    however large or obvious. Use the surroundings only to work out what the enclosed region is.
2. Decide what the enclosed region mainly shows.
3. Choose the label using the priority order below.

PRIORITY ORDER
a. Room-defining structure over incidental detail. The large planes that form the room -
    overhead, underfoot, the vertical sides - and the openings and supports in them take
    priority over small things mounted on, set into, or resting against them. If the boundary
    spans many repeated units of one surface (several ceiling tiles or panels, a run of floor,
    a stretch of wall), name the surface itself ("ceiling", "wall", "floor"), not one unit
    ("ceiling_tile").
b. BUT distinctive infrastructure that fills the boundary is named, not generalised. If exposed
    pipes, ductwork, a large air vent or diffuser, a fire hose reel, a sprinkler main or a big
    fixture dominates the enclosed region, name that thing. Collapsing to "ceiling" or "wall"
    is only right when the fixture is small or incidental and the surface is what the boundary
    mostly shows.
c. Furniture over the objects on it - but only with a furniture cue. A freestanding piece of
    furniture outranks the smaller things on it. A large flat vertical surface with no handle,
    seam, gap, toe-kick or visible depth is a wall or a pillar, not a cabinet - do not upgrade
    a plain surface to furniture without a cue.
d. When two readings compete, choose the larger, more dominant, more structural one.

MATERIAL AND AMBIGUITY CUES
- A transparent or translucent panel, a framed glass screen, a glazed partition or a
    glass-panelled door -> name it as glass (for example "glass_wall", "glass_partition",
    "glass_door"). Offices often divide space with glass rather than solid wall.
- Judge transparency directly: if the room, objects, people or light behind the panel are
    visible through it - even dimly, even with a frame, mullions or a faint surface reflection
    - it is glass; a panel you cannot see through at all is solid (wall, door, cabinet).
- A rectangular panel that reflects the room or shows a mirror image -> mirror. A plain matte
    white or lightly-marked panel -> whiteboard. If you cannot tell, name the wall it sits on
    at low confidence.
- Low detail, deep shadow, motion blur or heavy truncation -> do not commit to a specific
    object; give the structural reading (wall / floor / ceiling) at low confidence, or
    VLM_no_result if nothing is identifiable.

LABEL FORM
- A concise, singular, lowercase label that best describes what it actually is. There is no
    fixed list of labels - name whatever it is. Do not describe colour, condition or activity.

CONFIDENCE CALIBRATION for "label_confidence" - report it honestly
- 0.90-1.00: unmistakable; the enclosed region can only be this.
- 0.70-0.90: clear, minor uncertainty.
- 0.45-0.70: plausible, but a competing reading exists - the right band for cabinet-vs-wall and
    mirror-vs-whiteboard.
- 0.20-0.45: genuinely ambiguous, or a guess.
- 0.0 with label VLM_no_result: incomprehensible crop.
A low confidence on a hard crop is better than a confident wrong answer. Do not default to 0.90+.

Mobility:
- "dynamic": only a human, an animal, or a self-propelled moving object
- "static": any stationary object, including all surfaces, structures, furniture, glass and
    fixtures
- "unknown": when the label is unknown
```

```
OUTPUT - exactly one JSON object, with nothing before it or after it:
{"label": "<label>", "label_confidence": <0-1>, "mobility_class": "<static|dynamic|unknown>",
     "mobility_confidence": <0-1>}

GOOD - this is the whole reply:
{"label": "wall", "label_confidence": 0.85, "mobility_class": "static", "mobility_confidence":
     0.99}

BAD - never reply like any of these:
- Based on the visual evidence within the cyan boundary: 1. Analysis: ... then the JSON
- The object is a wall. {"label": "wall", ...}
- the JSON object wrapped in triple-backtick code fences

WORKED EXAMPLES (illustrating the approach, not a list of allowed labels):

Room-defining structure over incidental detail:
- Boundary spans a panelled ceiling covering many tiles and a recessed light -> {"label":
     "ceiling", "label_confidence": 0.92, "mobility_class": "static", "mobility_confidence":
     0.99}
- Boundary covers a wall area with a whiteboard mounted on part of it -> {"label": "wall",
     "label_confidence": 0.85, "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary on a wall plane; an armchair sits in front of it, outside the boundary -> {"label":
     "wall", "label_confidence": 0.86, "mobility_class": "static", "mobility_confidence": 0.99}
- Boundary on a floor area with a rug on it -> {"label": "floor", "label_confidence": 0.9,
     "mobility_class": "static", "mobility_confidence": 0.99}

Distinctive infrastructure that fills the boundary - name it:
- Exposed pipes or ductwork running across the ceiling, filling the boundary -> {"label":
     "pipes", "label_confidence": 0.8, "mobility_class": "static", "mobility_confidence": 0.99}
- A large air vent or diffuser that dominates the enclosed region -> {"label": "air_vent",
     "label_confidence": 0.82, "mobility_class": "static", "mobility_confidence": 0.99}
- A fire hose reel on the wall, filling the boundary -> {"label": "fire_hose_reel",
     "label_confidence": 0.85, "mobility_class": "static", "mobility_confidence": 0.99}

Glass and reflective surfaces:
- A framed glass partition dividing two areas -> {"label": "glass_wall", "label_confidence":
     0.8, "mobility_class": "static", "mobility_confidence": 0.99}
- A glass-panelled door in a frame -> {"label": "glass_door", "label_confidence": 0.82,
     "mobility_class": "static", "mobility_confidence": 0.99}
- A reflective rectangular panel showing a mirror image of the room -> {"label": "mirror",
     "label_confidence": 0.75, "mobility_class": "static", "mobility_confidence": 0.99}
- A plain white rectangular panel on a wall, no reflection -> {"label": "whiteboard",
     "label_confidence": 0.7, "mobility_class": "static", "mobility_confidence": 0.99}

Furniture vs plain surface:
- An office chair filling most of the frame -> {"label": "office_chair", "label_confidence":
     0.9, "mobility_class": "static", "mobility_confidence": 0.99}
- A desk seen clearly, with items on top -> {"label": "desk", "label_confidence": 0.9,
     "mobility_class": "static", "mobility_confidence": 0.99}
- A flat vertical surface with a visible handle, seam and toe-kick -> {"label": "cabinet",
     "label_confidence": 0.8, "mobility_class": "static", "mobility_confidence": 0.99}
- A flat vertical surface with no handles, seams or depth cues -> {"label": "wall",
     "label_confidence": 0.5, "mobility_class": "static", "mobility_confidence": 0.95}

Moving agents:
- A person in the space -> {"label": "person", "label_confidence": 0.93, "mobility_class":
     "dynamic", "mobility_confidence": 0.98}
```

```
Unclear:
- A blurred, dark or truncated region with no identifiable object -> {"label": "VLM_no_result",
     "label_confidence": 0.0, "mobility_class": "unknown", "mobility_confidence": 0.0}

Return only the JSON object - nothing before "{", nothing after "}". No markdown, no
     explanation, no numbered analysis.

ADDITIONAL FIELD - object_detail (experimental addition, V1_basic_addition)

Everything above is unchanged: still decide "label", "label_confidence",
"mobility_class" and "mobility_confidence" exactly as instructed.

Now also add a fifth key, "object_detail": a short phrase describing the
object's CURRENT VISIBLE STATE - not its identity, material or colour (the
label already covers that). Describe what is different about this specific
instance right now: items on or with it, its condition, or whether it looks
active or in use.

Examples:
- a table with a monitor and keyboard on it, tidily arranged -> "monitor and keyboard on table,
     organized properly"
- a floor with a wet patch -> "water on floor"
- a floor covered by a rug -> "carpet covering floor"
- a monitor showing an image -> "display on"
- a monitor with a black, blank screen -> "display off"
- nothing notable to report -> "no notable detail"

Keep object_detail to 10 words or fewer. Do not repeat the label inside it.

OUTPUT - exactly one JSON object, with nothing before it or after it:
{"label": "<label>", "label_confidence": <0-1>, "mobility_class": "<static|dynamic|unknown>",
     "mobility_confidence": <0-1>, "object_detail": "<short phrase>"}

GOOD - this is the whole reply:
{"label": "table", "label_confidence": 0.9, "mobility_class": "static", "mobility_confidence":
     0.99, "object_detail": "monitor and keyboard on table, organized properly"}
```

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e. g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

Stuttgart, on the 30 October 2026